CONCURRENCY DESIGN PATTERNS HANDBOOK

Complete Guide for Interview Preparation with Diagrams

Version 1.0

About This Book

Dear Reader,

This book is specially created for developers who want to master concurrency design patterns before their technical interviews. Every concept is explained in simple, clear English with real-world examples. Each pattern includes a diagram to help you visualize the structure and flow, making it easier to understand and remember.

Concurrency patterns are essential for building applications that can efficiently utilize modern multi-core processors and handle multiple tasks simultaneously. Whether you're working with Java threads, Go goroutines, or any other concurrency model, these patterns provide proven solutions to common problems in multi-threaded programming.

This book covers all the essential concurrency patterns you need to know for your interviews. Take your time reading this book. Read one pattern per day. Study the diagrams. Practice explaining each pattern out loud. By the time you finish, you will be completely ready to answer any concurrency design pattern question in your interview.

Let's begin your journey to becoming a concurrency expert.

[Page 2]

TABLE OF CONTENTS

PART ONE: INTRODUCTION TO CONCURRENCY PATTERNS

- Chapter 1: What Are Concurrency Design Patterns?
- Chapter 2: Why Concurrency Patterns Matter
- Chapter 3: Classification of Concurrency Patterns
- Chapter 4: Core Concurrency Concepts

PART TWO: THREAD COORDINATION PATTERNS
- Chapter 5: Mutex Pattern
- Chapter 6: Signaling Pattern
- Chapter 7: Rendezvous Pattern
- Chapter 8: Barrier Pattern
- Chapter 9: Double-Checked Locking Pattern
- Chapter 10: Read-Write Lock Pattern
- Chapter 11: Monitor Object Pattern
- Chapter 12: Guarded Suspension Pattern

PART THREE: TASK DISTRIBUTION PATTERNS
- Chapter 13: Thread Pool Pattern
- Chapter 14: Worker Pool Pattern
- Chapter 15: Active Object Pattern
- Chapter 16: Future / Promise Pattern
- Chapter 17: Generator Pattern
- Chapter 18: Fork-Join Pattern
- Chapter 19: Leader-Followers Pattern

PART FOUR: DATA FLOW PATTERNS
- Chapter 20: Producer-Consumer Pattern
- Chapter 21: Pipeline Pattern
- Chapter 22: Fan-Out / Fan-In Pattern
- Chapter 23: MapReduce Pattern

PART FIVE: EVENT HANDLING PATTERNS
- Chapter 24: Reactor Pattern
- Chapter 25: Proactor Pattern

- Chapter 26: Event Loop Pattern
- Chapter 27: Publish-Subscribe Pattern
- Chapter 28: Half-Sync / Half-Async Pattern

PART SIX: ADVANCED PATTERNS
- Chapter 29: Balking Pattern
- Chapter 30: Thread-Local Storage Pattern
- Chapter 31: Scheduler Pattern
- Chapter 32: Supervisor Pattern
- Chapter 33: Rate Limiting Pattern
- Chapter 34: Singleflight Pattern

PART SEVEN: COMPARISON AND INTERVIEW PREPARATION
- Chapter 35: Pattern Comparison Table
- Chapter 36: How to Answer Concurrency Questions
- Chapter 37: Common Interview Questions with Examples

[Page 3]

PART ONE: INTRODUCTION TO CONCURRENCY PATTERNS

Chapter 1: What Are Concurrency Design Patterns?

Understanding the Basic Concept

In software engineering, concurrency patterns are those types of design patterns that deal with the multi-threaded programming paradigm. They provide proven solutions to common problems that arise when multiple threads or processes execute simultaneously.

The Difference from Regular Design Patterns

Regular design patterns (like Singleton or Factory) focus on how classes and objects are structured and how they interact. Concurrency patterns focus on how threads coordinate, communicate, and synchronize. They address challenges like:

- **Race Conditions:** When multiple threads access shared data simultaneously and the final result depends on the timing of their execution.

- **Deadlocks:** When two or more threads are blocked forever, each waiting for the other to release a resource.

- **Livelocks:** When threads keep changing state in response to each other but never make progress.

- **Starvation:** When a thread never gets access to resources it needs to proceed.

- **Thread Safety:** Ensuring that shared data is accessed correctly by multiple threads.

Think of it this way. Imagine a kitchen with multiple chefs (threads) working together to prepare meals. They need to share resources like knives, cutting boards, and the oven. If they don't coordinate properly:

- Two chefs might grab the same knife at the same time (race condition)
- One chef might wait forever for the oven while another chef never finishes using it (deadlock)
- Some chefs might never get a turn with the oven because others keep using it (starvation)

Concurrency patterns are like kitchen rules and procedures that ensure all chefs work together efficiently without conflicts.

The Four Main Challenges Concurrency Patterns Solve

1. **Mutual Exclusion:** Ensuring that only one thread at a time can access a shared resource.

2. **Synchronization:** Making sure threads wait for appropriate conditions before proceeding.

3. **Communication:** Allowing threads to exchange data safely.

4. **Task Distribution:** Dividing work among multiple threads efficiently.
Example: Without Concurrency Patterns
Consider a simple counter shared by multiple threads:
counter = 0
Thread A: counter = counter + 1
Thread B: counter = counter + 1
Without proper coordination, this might happen:
- Thread A reads counter (0)
- Thread B reads counter (0)
- Thread A adds 1 and writes (1)
- Thread B adds 1 and writes (1)
Final value: 1 (but should be 2). This is a race condition.
Example: With Mutex Pattern
counter = 0
mutex = available
Thread A: acquire mutex, counter++, release mutex
Thread B: acquire mutex, counter++, release mutex
Now the operations are atomic (indivisible), and the result is always 2.
What This Book Covers
This book covers over 30 concurrency patterns, organized into categories based on the problems they solve. Each pattern includes:
- **Definition:** What the pattern is
- **Problem:** What problem it solves
- **Solution:** How it works
- **Diagram:** Visual representation
- **Use Case Examples:** Real-world scenarios
- **Advantages:** Benefits of using the pattern

- **Disadvantages:** Trade-offs and limitations
- **Comparison:** How it differs from similar patterns

[Page 4]

Chapter 2: Why Concurrency Patterns Matter

The Importance of Concurrency in Modern Computing

Concurrency is no longer optional—it's essential. Here's why:

1. Multi-Core Processors

Modern CPUs have multiple cores. A single-threaded application uses only one core, wasting most of the processor's capability. To utilize all cores, you need concurrent programming.

- **2005:** Single-core processors were common
- **2024:** Even phones have 8+ cores
- **Servers:** 64+ cores are common

Without concurrency patterns, you can't take advantage of this hardware capability.

2. Responsive User Interfaces

In GUI applications, the UI must remain responsive while performing long operations. If you perform a long operation on the UI thread, the application freezes. Concurrency patterns help you move long operations to background threads while keeping the UI responsive.

Example: A text editor that saves your file every few minutes. Without concurrency, the editor would freeze during save. With proper concurrency patterns, saving happens in the background while you continue typing.

3. Performance and Throughput

Concurrency patterns allow you to perform multiple operations simultaneously, increasing throughput:

- A web server handling thousands of simultaneous connections
- A batch processing system processing millions of records
- A data pipeline transforming data in real-time

4. Resource Utilization

Many operations involve waiting—for disk I/O, network responses, database queries. Without concurrency, the CPU sits idle during these waits. Concurrency patterns allow other work to proceed while waiting.

Example: A web application making a database query. Without concurrency, the thread blocks, and the CPU does nothing for 50ms. With concurrency, that thread can work on another request while waiting.

5. Scalability

Concurrency patterns enable horizontal scaling—adding more threads or processes to handle more work. This is essential for cloud-native applications.

6. Fault Isolation

Proper concurrency patterns can isolate failures. If one task fails, it doesn't bring down the entire application.

Real-World Consequences of Getting Concurrency Wrong

- **Therac-25 Radiation Machine:** Race conditions led to radiation overdoses
- **Northeast Blackout 2003:** Race condition in alarm system contributed to power grid failure
- **Knight Capital Group:** Concurrency bug caused $440 million loss in 45 minutes

Why Patterns Are Essential for Concurrency

Concurrency is inherently difficult. The human brain thinks sequentially, not in parallel. Concurrency patterns provide:

1. **Proven Solutions:** These patterns have been battle-tested in production systems.

2. **Common Vocabulary:** When you say "we need a producer-consumer queue here," other developers immediately understand the design.

3. **Avoid Pitfalls:** Patterns help you avoid common mistakes like deadlocks and race conditions.

4. **Reasoning Framework:** Patterns give you a way to think about concurrency problems systematically.

5. **Reusable Designs:** You don't have to reinvent the wheel for every concurrent system.

When You Need Concurrency Patterns

- Building web servers or API gateways
- Creating real-time data processing systems
- Developing GUI applications
- Working with databases and connection pools
- Implementing caching systems
- Building microservices that handle many requests
- Creating batch processing jobs
- Developing game engines
- Building operating systems or device drivers

When You Might Not Need Concurrency Patterns

- Simple command-line scripts
- CPU-bound tasks that don't benefit from parallelization
- Applications running on single-core embedded systems
- Prototypes where simplicity is more important than performance

---

[Page 5]

Chapter 3: Classification of Concurrency Patterns

The Six Categories of Concurrency Patterns

Concurrency patterns can be organized into six main categories based on the problems they solve:

Category 1: Thread Coordination Patterns

These patterns manage how threads coordinate their activities, ensuring they don't interfere with each other and that they wait for appropriate conditions.

Key patterns in this category:

- **Mutex:** Provides mutual exclusion for shared resources
- **Signaling:** Allows threads to notify each other about events
- **Rendezvous:** Ensures two threads meet at a specific point
- **Barrier:** Makes multiple threads wait until all reach a point
- **Double-Checked Locking:** Optimizes lazy initialization
- **Read-Write Lock:** Allows concurrent reads, exclusive writes
- **Monitor Object:** Combines mutex and condition variables
- **Guarded Suspension:** Makes threads wait until condition is true

Category 2: Task Distribution Patterns

These patterns focus on distributing work among multiple threads or workers efficiently.

Key patterns in this category:

- **Thread Pool:** Reuses a fixed number of threads for many tasks
- **Worker Pool:** Distributes work among identical workers
- **Active Object:** Decouples method execution from invocation
- **Future / Promise:** Represents a result that will be available later
- **Generator:** Produces values lazily, one at a time
- **Fork-Join:** Divides work and combines results
- **Leader-Followers:** Efficient thread management for events

Category 3: Data Flow Patterns

These patterns manage how data flows between concurrent components, focusing on transformation and processing pipelines.

Key patterns in this category:

- **Producer-Consumer:** Separates production from consumption
- **Pipeline:** Processes data through stages
- **Fan-Out / Fan-In:** Distributes work and collects results
- **MapReduce:** Processes large datasets in parallel

Category 4: Event Handling Patterns

These patterns manage how systems respond to events asynchronously, essential for building responsive applications.

Key patterns in this category:

- **Reactor:** Demultiplexes and dispatches events synchronously
- **Proactor:** Demultiplexes and dispatches events asynchronously
- **Event Loop:** Processes events in a loop
- **Publish-Subscribe:** Decouples event publishers from subscribers
- **Half-Sync / Half-Async:** Separates sync and async processing

Category 5: Advanced Coordination Patterns

These patterns solve specific, more complex coordination problems.

Key patterns in this category:

- **Balking:** Aborts operation if condition not met
- **Thread-Local Storage:** Gives each thread its own data copy
- **Scheduler:** Controls order of thread execution
- **Supervisor:** Monitors and restarts failed threads
- **Rate Limiting:** Controls how fast operations occur
- **Singleflight:** Prevents duplicate concurrent operations

How Patterns Work Together

Patterns are often combined to create robust concurrent systems:

- **Thread Pool + Producer-Consumer:** Web servers use this combination
- **Future + Fork-Join:** Parallel computation frameworks
- **Reactor + Thread Pool:** High-performance network servers
- **Rate Limiting + Worker Pool:** API gateways

Example: Web Server Architecture

A typical web server combines multiple patterns:
- **Reactor/Proactor:** Detects incoming connections
- **Thread Pool:** Handles requests
- **Producer-Consumer:** Queues incoming requests
- **Future:** Represents asynchronous responses
- **Rate Limiting:** Prevents overload

[Page 6]

Chapter 4: Core Concurrency Concepts

Essential Concepts You Must Understand

Before diving into patterns, you need to understand these fundamental concepts. They appear in every pattern and every concurrency discussion.

1. Threads vs Processes

Aspect	Thread	Process
---	---	---
Memory	Shared memory space	Separate memory space
Creation	Fast	Slow
Communication	Easy (shared memory)	Hard (IPC)
Isolation	Low (one thread can crash all)	High
Context Switch	Fast	Slower

- **Thread:** Lightweight unit of execution within a process. Threads share memory, making communication easy but synchronization necessary.

- **Process:** Heavyweight unit with its own memory space. Processes are isolated, making them safer but communication harder.

2. Race Condition

A race condition occurs when multiple threads access shared data concurrently, and the final result depends on the timing of their execution.

Example:

```
Thread A: x = x + 1  // reads x, adds 1, writes back
Thread B: x = x + 1  // reads x, adds 1, writes back
```

If both read x=5 before either writes, both write 6, losing one increment.

3. Critical Section

A critical section is a piece of code that accesses shared resources and must not be executed by more than one thread at a time.

4. Mutual Exclusion

The property that ensures only one thread can execute a critical section at a time. Implemented using locks, semaphores, or monitors.

5. Deadlock

A deadlock occurs when two or more threads are blocked forever, each waiting for a resource held by another.

Four Necessary Conditions for Deadlock:

1. **Mutual Exclusion:** Resources cannot be shared
2. **Hold and Wait:** Thread holds resources while waiting for others
3. **No Preemption:** Resources cannot be forcibly taken
4. **Circular Wait:** Circular chain of threads waiting

Example:

```
Thread A: lock X, then wait for Y
Thread B: lock Y, then wait for X
```

Both wait forever.

6. Starvation

Starvation occurs when a thread never gets access to a resource it needs because other threads always get it first.

Example: A low-priority thread never runs because higher-priority threads always run.

7. Livelock

Livelock occurs when threads keep changing state in response to each other but never make progress.

Example: Two threads each release a lock when they see the other waiting, then both try to acquire again, repeating forever.

8. Atomicity

An operation is atomic if it appears to happen instantaneously—no other thread can see it partially complete.

Hardware atomic operations:

- Compare-and-swap (CAS)
- Test-and-set
- Fetch-and-add

9. Visibility

Changes made by one thread may not be immediately visible to other threads due to CPU caches and compiler optimizations.

Solution: Use memory barriers or volatile variables.

10. Ordering

Instructions may be reordered by the compiler or CPU for optimization. This can cause unexpected behavior in concurrent code.

Solution: Use memory barriers to enforce ordering.

11. Thread Safety

A class or method is thread-safe if it behaves correctly when accessed by multiple threads.

Ways to achieve thread safety:

- **Immutable objects:** Cannot be changed after creation
- **Synchronization:** Using locks to control access
- **Thread-local storage:** Each thread has its own copy
- **Atomic operations:** Hardware-level atomicity

12. Blocking vs Non-Blocking

Type	Description	Example
------	-------------	---------
Blocking	Thread waits until operation completes	Traditional I/O, synchronized methods
Non-Blocking	Thread doesn't wait, may need to retry	CAS operations, async I/O

13. Synchronous vs Asynchronous

Type	Description	Example
------	-------------	---------
Synchronous	Caller waits for result	Regular method call
Asynchronous	Caller continues, result later	Callbacks, futures

14. Concurrency vs Parallelism

- **Concurrency:** Multiple tasks making progress (interleaved execution)
- **Parallelism:** Multiple tasks executing simultaneously (on multiple cores)

Analogy: Concurrency is like a single cook handling multiple orders by switching between them. Parallelism is like multiple cooks each handling one order.

15. Amdahl's Law

The theoretical speedup of a program as a function of the number of processors:

```
Speedup = 1 / (S + (1-S)/N)
```

Where S is the sequential portion and N is the number of processors.

Implication: Even with infinite processors, you're limited by the sequential portion of your code.

16. Context Switching Overhead

Switching between threads has a cost—saving and restoring registers, flushing caches. Too many threads can hurt performance.

17. Thread Safety Levels

1. **Immutable:** Always safe
2. **Thread-Safe:** Safe for concurrent use
3. **Conditionally Safe:** Safe with proper external synchronization
4. **Not Thread-Safe:** Must be synchronized externally

18. Lock Granularity

- **Coarse-grained:** One lock for large data structure (simple but low concurrency)
- **Fine-grained:** Multiple locks for different parts (complex but high concurrency)

19. Lock Types

- **Mutex:** Mutual exclusion lock
- **Semaphore:** Counter that controls access
- **Read-Write Lock:** Multiple readers, single writer
- **Spin Lock:** Busy-waits for lock (good for short waits)

20. Memory Model

Every programming language defines a memory model that specifies how threads interact through memory. Understanding your language's memory model is essential for correct concurrent programming.

Java Memory Model: Guarantees visibility with happens-before relationships

C++ Memory Model: Defines data races and atomic operations

[Page 7]

PART TWO: THREAD COORDINATION PATTERNS

Chapter 5: Mutex Pattern

Detailed Explanation

The Mutex pattern provides mutual exclusion—ensuring that only one thread at a time can access a shared resource. Mutex stands for "MUTual EXclusion."

Understanding the Problem

When multiple threads access shared data simultaneously, race conditions occur. Consider a bank account with a balance of $100. Two threads both try to withdraw $50:

Thread A reads balance ($100)

Thread B reads balance ($100)

Thread A subtracts $50 and writes ($50)

Thread B subtracts $50 and writes ($50)

Final balance: $50 (but should be $0). The account was over-withdrawn by $50.

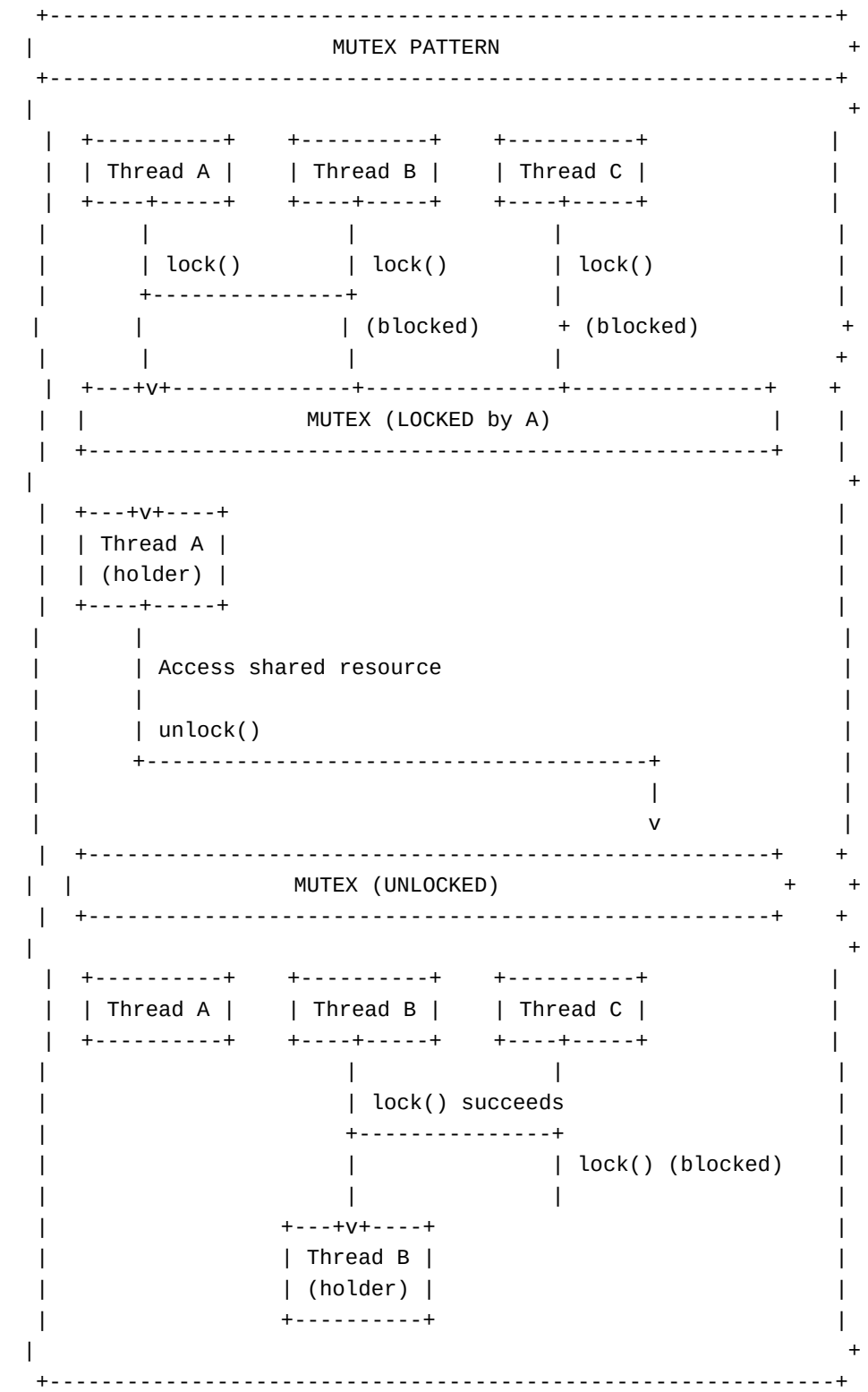
Without mutual exclusion, concurrent access to shared data leads to corruption.

The Solution

The Mutex pattern provides a lock that threads must acquire before accessing shared data. Only one thread can hold the lock at a time. Other threads must wait until the lock is released.

Key Concepts
- **Lock:** An object representing ownership of the mutex
- **Acquire/Lock:** Thread takes ownership (blocks if already held)
- **Release/Unlock:** Thread releases ownership
- **Locked Region:** Code between acquire and release (critical section)

Diagram: Mutex Pattern



Implementation Example

```
public class BankAccount {
    private double balance;
    private final Object lock = new Object();

    public void withdraw(double amount) {
        synchronized(lock) {  // acquire lock
            if (balance >= amount) {
                balance -= amount;
            }
        }  // release lock automatically
    }

    public void deposit(double amount) {
        synchronized(lock) {
            balance += amount;
        }
    }
}
```

When to Use Mutex Pattern

Suitable Situations:

1. When multiple threads access shared data that must remain consistent.

2. When you need to protect critical sections of code.

3. When operations on shared data must appear atomic.

4. In any multi-threaded application with shared mutable state.

Real-World Use Cases:

- **Banking System:** Protecting account balances from concurrent withdrawals
- **Inventory Management:** Ensuring stock counts are accurate
- **Cache Implementation:** Protecting cache data structure
- **File Writing:** Ensuring only one thread writes to a log file
- **Database Connection Pool:** Managing connection allocation

Advantages of Mutex Pattern

1. **Simple to Understand:** The concept of locking is intuitive.

2. **Prevents Race Conditions:** Ensures data consistency.

3. **Widely Supported:** Available in all programming languages.

4. **Language Integration:** Many languages have built-in support (synchronized in Java, lock in C#).

5. **Predictable:** Behavior is well-defined and understood.

Disadvantages of Mutex Pattern

1. **Performance Overhead:** Acquiring and releasing locks has cost.

2. **Deadlock Risk:** Improper use can cause deadlocks.

3. **Contention:** Threads waiting for locks reduce parallelism.

4. **No Reader Optimization:** Even multiple readers are serialized.

5. **Error-Prone:** Forgetting to release lock causes problems.

6. **Limited Scalability:** Coarse-grained locking limits concurrency.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Mutex	Exclusive access	Only one thread at a time
Semaphore	Multiple permits	Can allow N threads
Read-Write Lock	Readers/writer	Multiple readers allowed
Monitor	Mutex + condition	Adds wait/notify capability

Mutex vs Semaphore

- **Mutex:** Binary (locked/unlocked), ownership concept (only locking thread can unlock)
- **Semaphore:** Counter (can have multiple permits), no ownership

Mutex vs Read-Write Lock

- **Mutex:** Exclusive access for all operations
- **Read-Write Lock:** Multiple readers can access simultaneously

Common Pitfalls

1. **Forgetting to Unlock:** Use try/finally or language constructs (synchronized).

2. **Locking Too Much:** Reduces concurrency (coarse-grained).

3. **Locking Too Little:** Causes race conditions (fine-grained).

4. **Deadlock:** Acquiring locks in different orders.

5. **Lock Contention:** Too many threads waiting for same lock.

[Page 8]

Chapter 6: Signaling Pattern

Detailed Explanation

The Signaling pattern allows one thread to notify another thread that an event has occurred. It's a fundamental pattern for thread communication and coordination.

Understanding the Problem

Often in concurrent programs, one thread needs to wait for a condition to become true before proceeding. For example:

- A worker thread needs to wait until there's work in the queue
- A background thread needs to wait until it's time to run
- A consumer needs to wait until data is available

Without signaling, threads would have to busy-wait (constantly check), wasting CPU cycles.

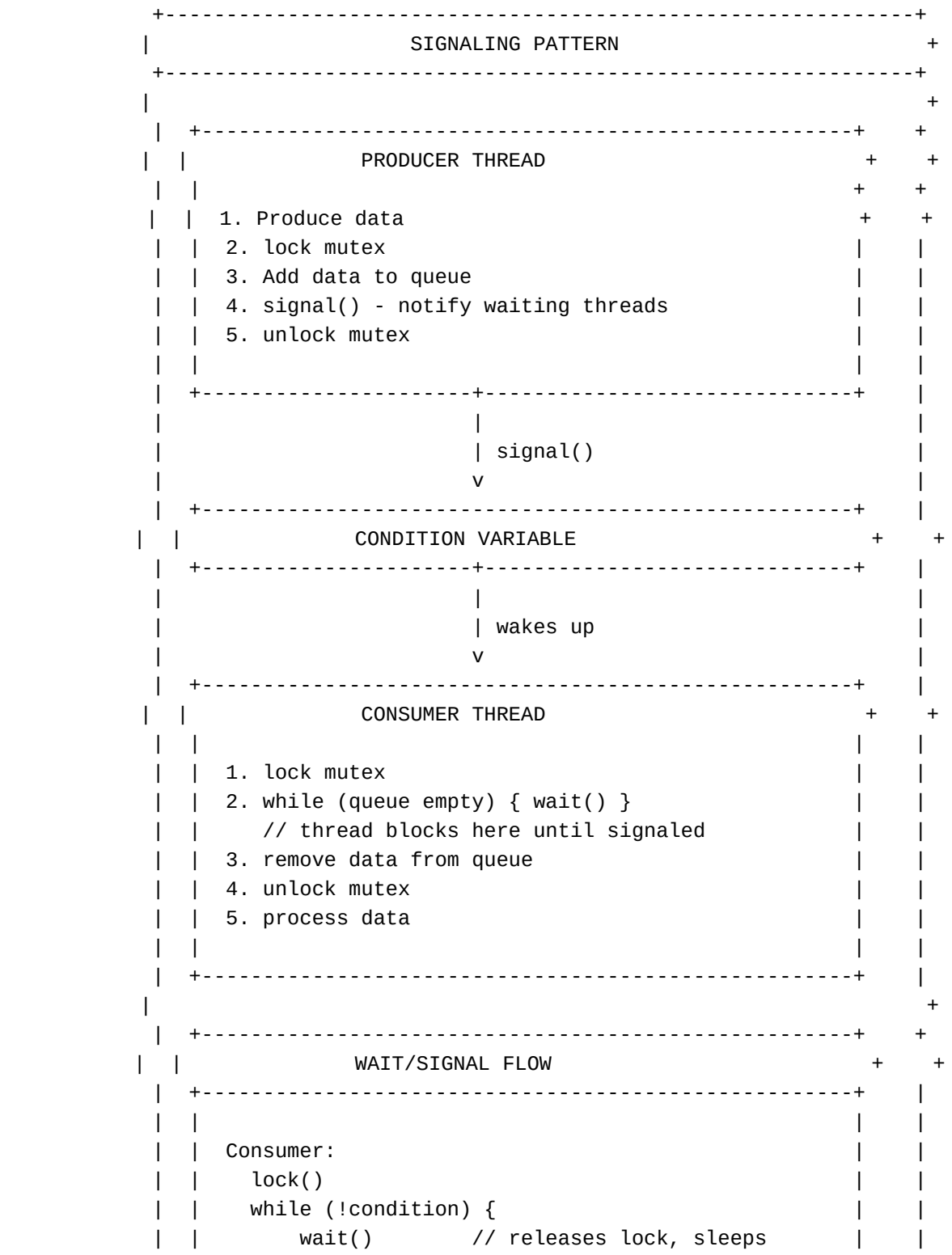
The Solution

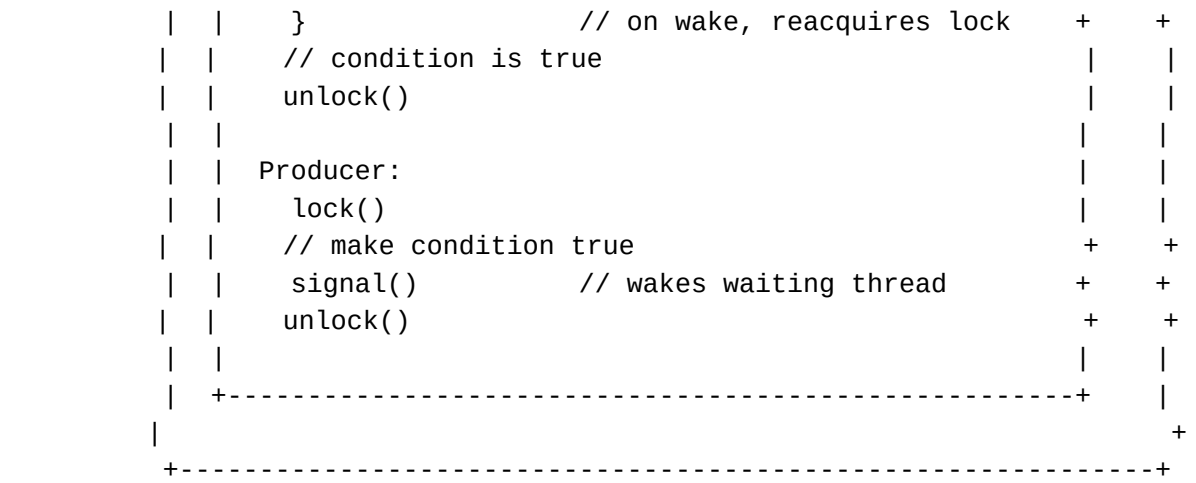
The Signaling pattern uses condition variables or semaphores to allow threads to wait efficiently and be notified when conditions change.

Key Concepts

- **Condition Variable:** An object that threads can wait on and be signaled
- **Wait:** Thread blocks until signaled, releasing associated mutex
- **Signal/Notify:** Wakes one waiting thread
- **Broadcast/NotifyAll:** Wakes all waiting threads
- **Spurious Wakeup:** Thread may wake without signal (must recheck condition)

Diagram: Signaling Pattern





Implementation Example

```
public class WorkQueue {
private final Queue<Runnable> queue = new LinkedList<>();
private final Object lock = new Object();

public void submit(Runnable task) {
synchronized(lock) {
queue.add(task);
lock.notify();  // signal waiting worker
}
}

public Runnable take() throws InterruptedException {
synchronized(lock) {
while (queue.isEmpty()) {
lock.wait();  // wait for work
}
return queue.poll();
}
}
}
```

When to Use Signaling Pattern

Suitable Situations:

1. When threads need to wait for conditions to become true.
2. When you want to avoid busy-waiting (polling).
3. In producer-consumer scenarios.
4. When implementing thread pools.
5. When coordinating dependent tasks.

Real-World Use Cases:

- **Work Queue:** Workers wait for tasks, producers signal new tasks
- **Connection Pool:** Threads wait for available connections
- **Event Handling:** Threads wait for events to occur
- **Background Jobs:** Scheduler signals when it's time to run
- **Resource Pool:** Threads wait for resources to become available

Advantages of Signaling Pattern

1. **Efficient:** Threads sleep, not busy-wait, saving CPU.

2. **Decoupled:** Producers don't need to know about consumers.

3. **Flexible:** Multiple threads can wait on same condition.

4. **Language Support:** Built into most threading libraries.

5. **Precise Control:** Can signal specific threads (notify) or all (notifyAll).

Disadvantages of Signaling Pattern

1. **Complexity:** Easy to get wrong (missed signals, lost wakeups).

2. **Spurious Wakeups:** Threads may wake without signal (must recheck).

3. **Deadlock Risk:** Improper lock ordering can cause deadlock.

4. **Starvation:** Some threads might never get signaled.

5. **Debugging Difficulty:** Race conditions in signaling are hard to reproduce.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
Signaling	Notify waiting threads	For condition waiting
Mutex	Exclusive access	No waiting on conditions
Semaphore	Counting permits	Can be used for signaling
Barrier	Multiple threads wait	All wait for each other

Signaling vs Semaphore for Signaling

- **Condition Variable:** Must be used with mutex, signals specific condition
- **Semaphore:** Can be used for signaling without mutex, counts events

Common Pitfalls

1. **Lost Wakeup:** Signaling before thread waits (thread misses signal).

2. **Spurious Wakeup:** Not checking condition in loop.

3. **Holding Lock While Waiting:** Must release lock during wait.

4. **Notifying Wrong Thread:** Using notify() when notifyAll() needed.

5. **Deadlock:** Acquiring locks in wrong order.

Best Practices

1. **Always Check Condition in Loop:** while (!condition) wait();

2. **Use notifyAll() Unless Sure:** Prevents missed signals.

3. **Minimize Work While Holding Lock:** Release quickly.

4. **Document Signaling Protocol:** Make clear what conditions mean.

5. **Test with Multiple Threads:** Signaling bugs often appear under load.

[Page 9]

Chapter 7: Rendezvous Pattern

Detailed Explanation

The Rendezvous pattern ensures that two threads meet at a specific point in their execution. Neither thread can proceed past that point until both have arrived.

Understanding the Problem

Sometimes in concurrent programs, you need two threads to synchronize at a specific point. For example:

- Two threads need to exchange data at the same time
- Thread A must reach a certain state before thread B can proceed
- Both threads must complete initialization before continuing

Without a rendezvous, one thread might race ahead, causing incorrect behavior or missing data.

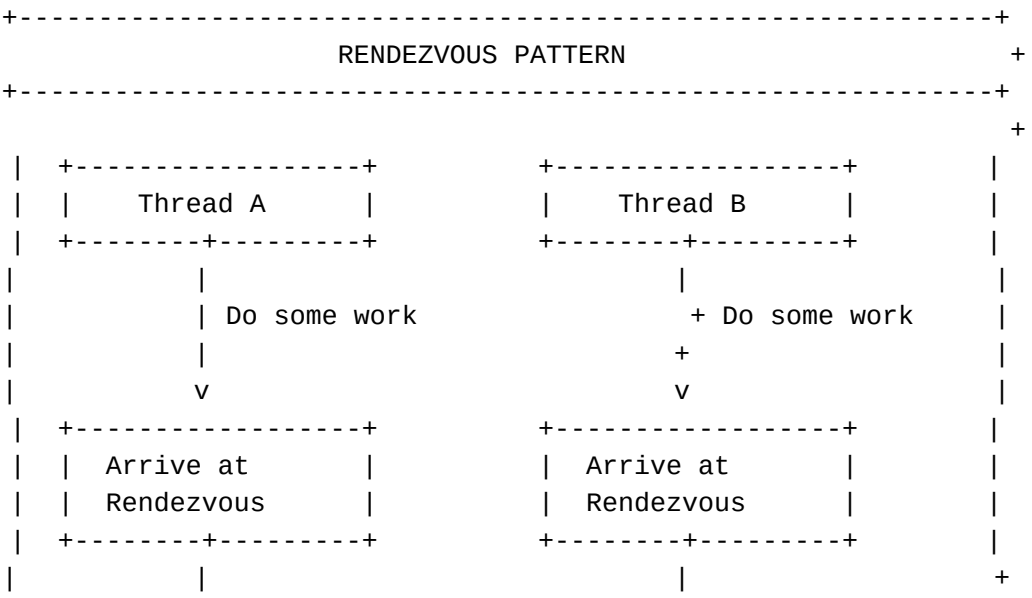
The Solution

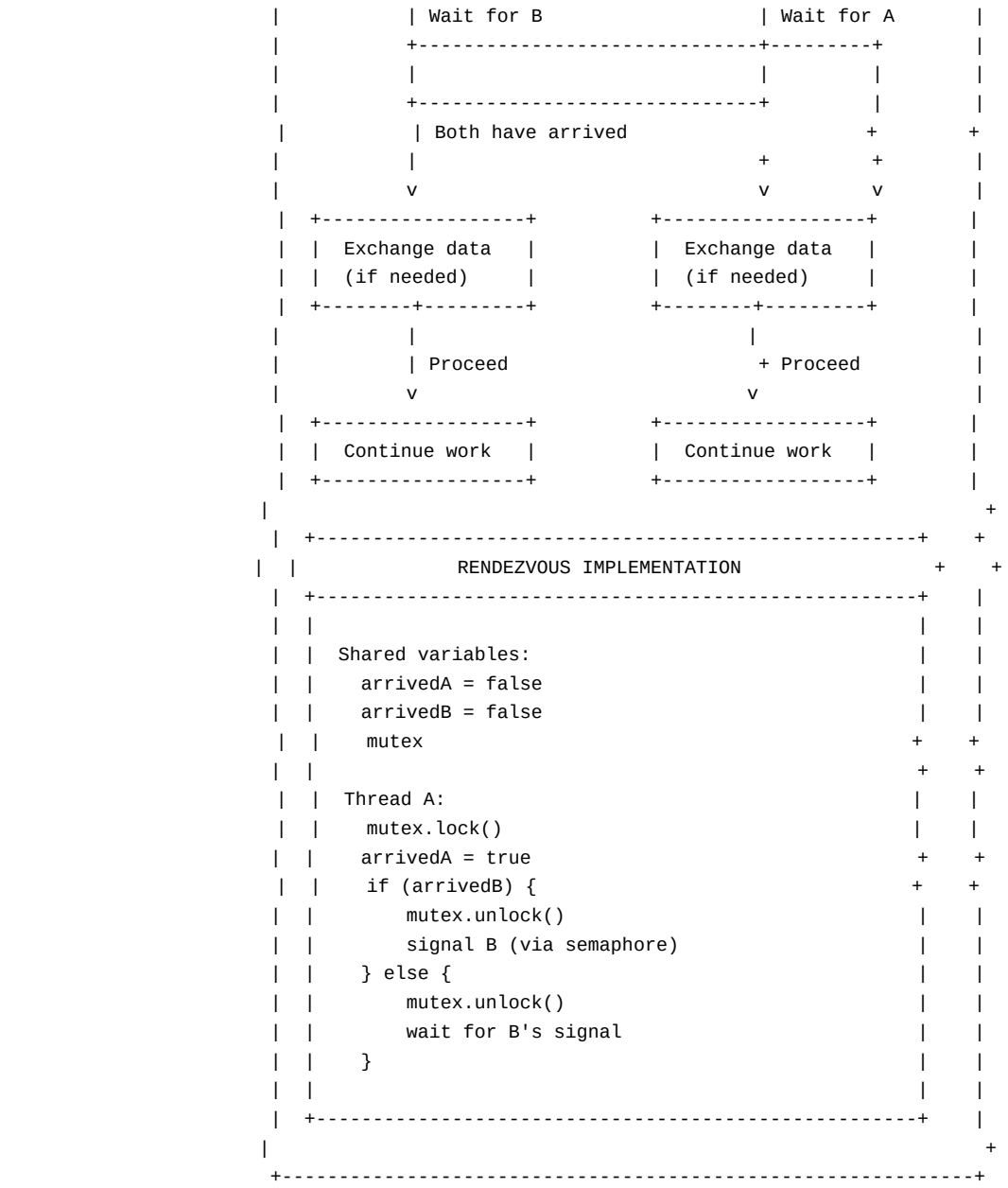
The Rendezvous pattern uses two barriers or semaphores to ensure both threads wait for each other. When both arrive, they both proceed.

Key Concepts

- **Arrival:** Thread reaches rendezvous point
- **Wait:** Thread waits for other thread to arrive
- **Exchange:** Threads can exchange data at rendezvous
- **Proceed:** Both threads continue after both have arrived

Diagram: Rendezvous Pattern





Implementation Example

```
public class Rendezvous {
private boolean arrived1 = false;
private boolean arrived2 = false;
private final Object lock = new Object();

public void meet(Thread other) throws InterruptedException {
synchronized(lock) {
if (Thread.currentThread().getId() == 1) {
arrived1 = true;
if (!arrived2) {
lock.wait();  // wait for thread 2
} else {
lock.notify(); // signal thread 2
}
} else {
arrived2 = true;
if (!arrived1) {
lock.wait();  // wait for thread 1
} else {
```

```
                    lock.notify(); // signal thread 1
                }
            }
        }
    }
}
```

Better Implementation Using Semaphores

```
public class Rendezvous {
    private final Semaphore sem1 = new Semaphore(0);
    private final Semaphore sem2 = new Semaphore(0);

    public void meetA() throws InterruptedException {
        // Thread A arrives
        sem2.release();  // signal B that A arrived
        sem1.acquire();  // wait for B
    }

    public void meetB() throws InterruptedException {
        // Thread B arrives
        sem1.release();  // signal A that B arrived
        sem2.acquire();  // wait for A
    }
}
```

When to Use Rendezvous Pattern

Suitable Situations:

1. When two threads need to exchange data at the same time.

2. When both threads must reach a certain state before continuing.

3. When implementing parallel algorithms with synchronization points.

4. In simulation systems where time steps must be synchronized.

5. When threads have mutual dependencies that must be resolved.

Real-World Use Cases:

- **Data Exchange:** Two threads swap data buffers
- **Parallel Algorithms:** Threads synchronize at iteration boundaries
- **Pipeline Stages:** Adjacent stages need to exchange data
- **Game Engines:** Physics and rendering threads synchronize
- **Simulation:** Time-step synchronization

Advantages of Rendezvous Pattern

1. **Simple Concept:** Easy to understand what it accomplishes.

2. **Deadlock Prevention:** Proper implementation avoids deadlock.

3. **Data Exchange:** Natural point for threads to swap data.

4. **Synchronization:** Ensures threads are in sync.

5. **No Busy-Waiting:** Threads sleep efficiently.

Disadvantages of Rendezvous Pattern

1. **Limited to Two Threads:** Basic pattern only for two.

2. **Complex Implementation:** Getting signaling right is tricky.

3. **Starvation Possible:** One thread might wait forever.

4. **Overhead:** Synchronization adds cost.

5. **Deadlock Risk:** Incorrect implementation can deadlock.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Rendezvous	Two threads meet	Both wait for each other
Barrier	N threads wait	All wait for all
Signaling	One thread notifies	One waits, one signals
Exchanger	Data exchange	Built-in Java class for this

Rendezvous vs Barrier

- **Rendezvous:** Exactly two threads, each waits for the other
- **Barrier:** N threads, all wait until all have arrived

Rendezvous vs Exchanger

Java provides Exchanger class specifically for this pattern:

```
        Exchanger<String> exchanger = new Exchanger<>();

        // Thread A
        String dataA = "data from A";
        String dataB = exchanger.exchange(dataA);  // waits for B

        // Thread B
        String dataB = "data from B";
        String dataA = exchanger.exchange(dataB);  // waits for A
```

Common Pitfalls

1. **Deadlock:** Both threads waiting but neither signals.

2. **Lost Signal:** Signaling before the other thread waits.

3. **Spurious Wakeup:** Not checking condition properly.

4. **Assuming Thread Order:** Can't assume which arrives first.

5. **Not Handling InterruptedException:** Threads may be interrupted.

[Page 10]

Chapter 8: Barrier Pattern

Detailed Explanation

The Barrier pattern allows multiple threads to wait for each other at a common point. No thread can proceed past the barrier until all threads have reached it.

Understanding the Problem

In parallel algorithms, you often have phases where all threads must complete one phase before any can start the next. For example:

- In matrix multiplication, all threads must finish computing their part of the result before moving to the next step

- In iterative algorithms, all threads must finish one iteration before starting the next
- In simulations, all entities must complete their updates for the current time step before time advances

Without a barrier, some threads would race ahead, processing data that isn't ready yet.

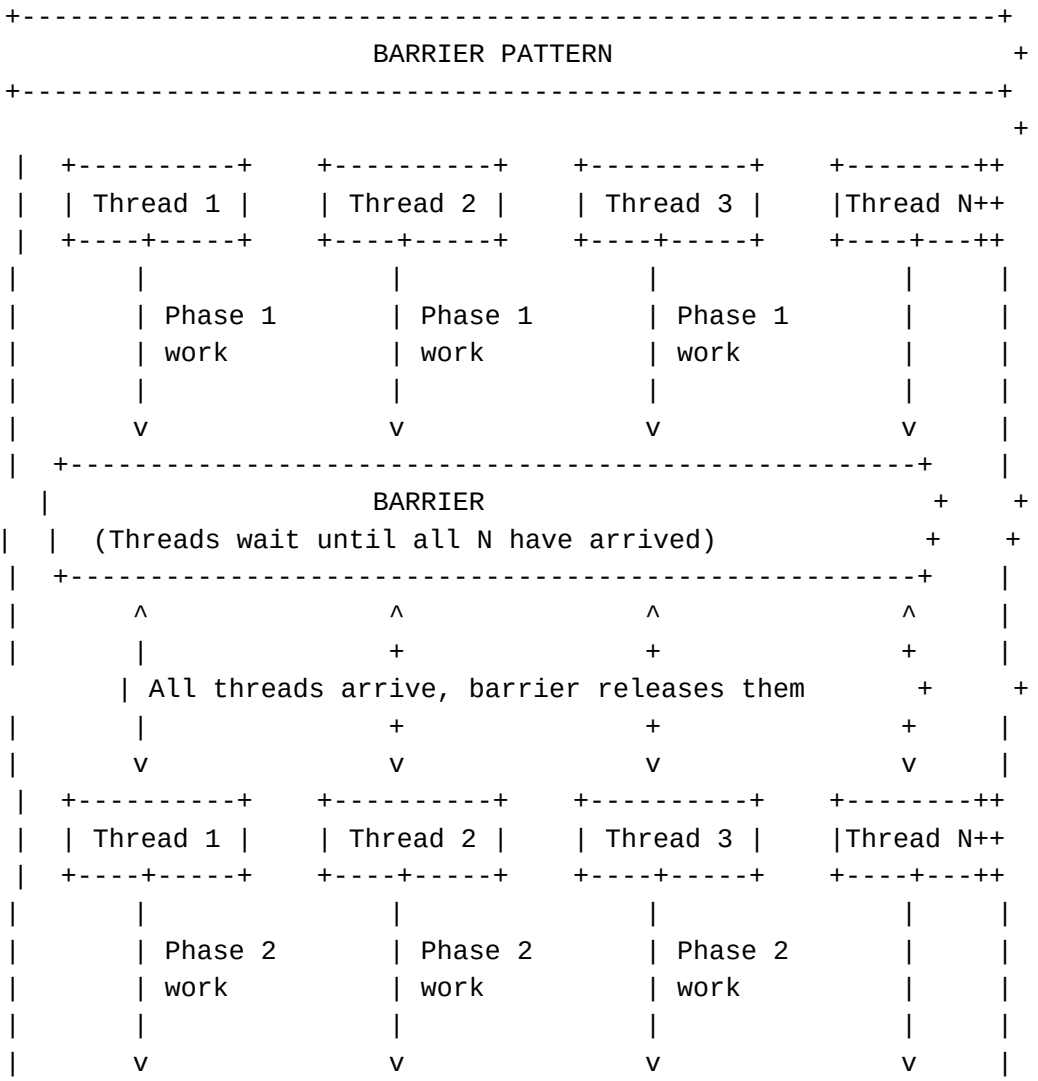
The Solution

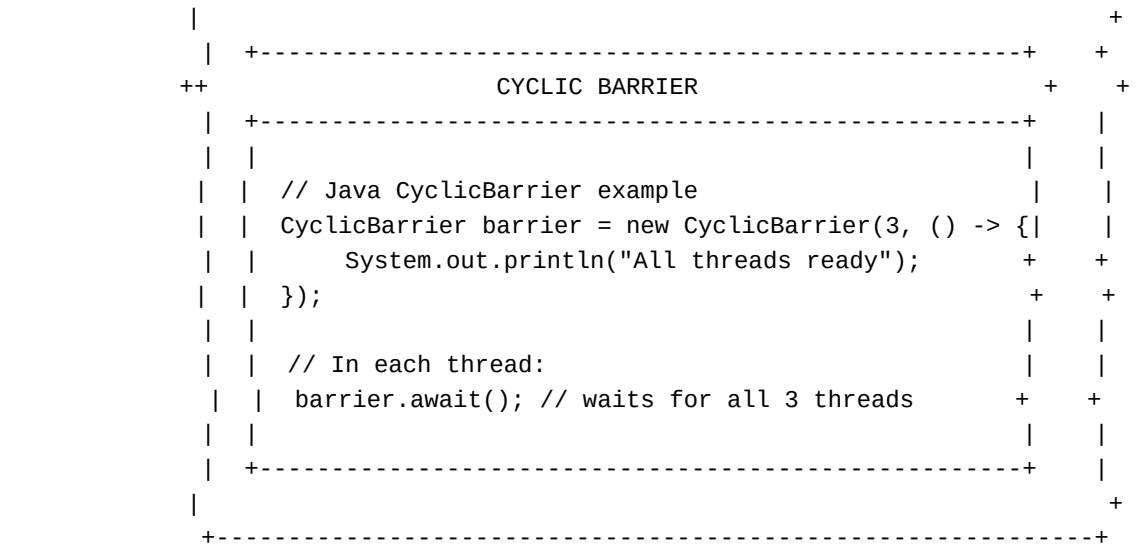
The Barrier pattern provides a synchronization point where threads wait. When the last thread arrives, all threads are released to continue.

Key Concepts

- **Parties:** Number of threads that must reach the barrier
- **Arrival:** Thread reaches barrier and waits
- **Generation:** A cycle of barrier usage (can be reused)
- **Broken Barrier:** When a thread times out or is interrupted
- **Barrier Action:** Optional action to run when barrier is tripped

Diagram: Barrier Pattern





Implementation Example

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;
public class MatrixMultiplication {
private static final int THREADS = 4;
private final CyclicBarrier barrier =
                                new CyclicBarrier(THREADS, () -> {
System.out.println("Phase complete, moving to next");
});

public void multiply() {
                                for (int i = 0; i < THREADS; i++) {
new Thread(new Worker(i)).start();
}
}

class Worker implements Runnable {
private int id;

Worker(int id) { this.id = id; }

public void run() {
                               for (int phase = 0; phase < 3; phase++) {
// Do work for this phase
                                    System.out.println("Thread " + id +
                                        " completing phase " + phase);

try {
// Wait for all threads
barrier.await();
                                      } catch (InterruptedException |
BrokenBarrierException e) {
return;
}
}
}
}
}
```

Custom Barrier Implementation

```
public class SimpleBarrier {
```

```
        private final int parties;
        private int count = 0;
        private final Object lock = new Object();

        public SimpleBarrier(int parties) {
            this.parties = parties;
        }

        public void await() throws InterruptedException {
            synchronized(lock) {
                count++;
                if (count == parties) {
                    // Last thread arrives - release all
                    lock.notifyAll();
                    count = 0; // reset for reuse
                } else {
                    // Wait for others
                    lock.wait();
                }
            }
        }
    }
```

When to Use Barrier Pattern

Suitable Situations:

1. When you have multi-phase parallel algorithms.

2. When threads need to synchronize at regular intervals.

3. When you need to combine partial results before continuing.

4. In iterative computations where each iteration depends on previous.

5. When implementing parallel simulations.

Real-World Use Cases:

- **Matrix Multiplication:** Threads compute portions, synchronize between steps
- **Image Processing:** Parallel filtering with multiple passes
- **Scientific Computing:** Iterative solvers (Jacobi, Gauss-Seidel)
- **Game Development:** Physics updates synchronized across entities
- **Data Processing:** MapReduce-like phases

Advantages of Barrier Pattern

1. **Simple Semantics:** Easy to understand the coordination.

2. **Reusable:** CyclicBarrier can be used multiple times.

3. **Efficient:** Threads sleep, not busy-wait.

4. **Barrier Action:** Can run code when barrier is tripped.

5. **Language Support:** Available in Java (CyclicBarrier), .NET (Barrier), etc.

Disadvantages of Barrier Pattern

1. **Performance Penalty:** All threads must wait for slowest.

2. **Deadlock Risk:** If a thread never arrives, others wait forever.

3. **Not Suitable for Uneven Work:** Slower threads hold up others.

4. **Broken Barrier:** Exceptions can leave barrier in broken state.

5. **Oversynchronization:** May force synchronization where not needed.

Comparison with Other Patterns

Pattern	Purpose	Difference
Barrier	All threads wait	N threads wait for each other
Rendezvous	Two threads meet	Special case for N=2
CountDownLatch	One-time event	Threads wait for count to reach zero
Phaser	Flexible barriers	More flexible, can add/remove parties

Barrier vs CountDownLatch

- **Barrier:** All threads wait for each other (can be reused)
- **CountDownLatch:** Threads wait for count to reach zero (one-time use)

Barrier vs Phaser

Java's Phaser provides more flexibility:

- Can dynamically add/remove parties

- Can be used in tree structures

- More complex but powerful

Common Pitfalls

1. **Not Enough Threads:** Barrier with 4 parties but only 3 threads arrive.

2. **BrokenBarrierException:** Handling when barrier is broken.

3. **Timeout:** Thread may time out waiting.

4. **Reset Issues:** Resetting barrier while threads wait.

5. **Assuming Order:** Don't assume threads arrive in any order.

[Page 11]

Chapter 9: Double-Checked Locking Pattern

Detailed Explanation

The Double-Checked Locking pattern optimizes lazy initialization in multi-threaded environments by checking a condition twice—once without locking, and once with locking—to avoid synchronization overhead after initialization is complete.

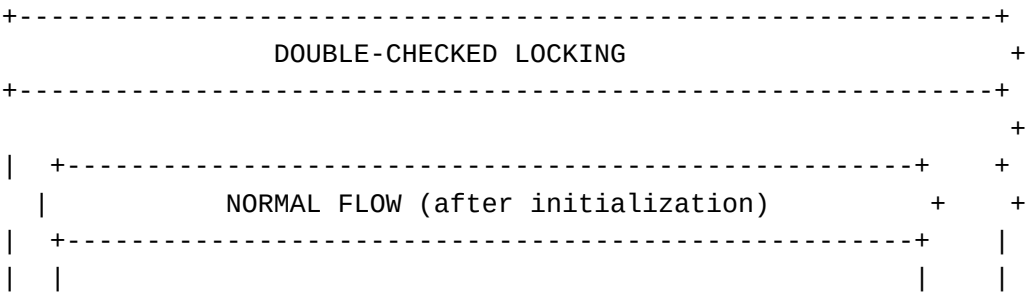
Understanding the Problem

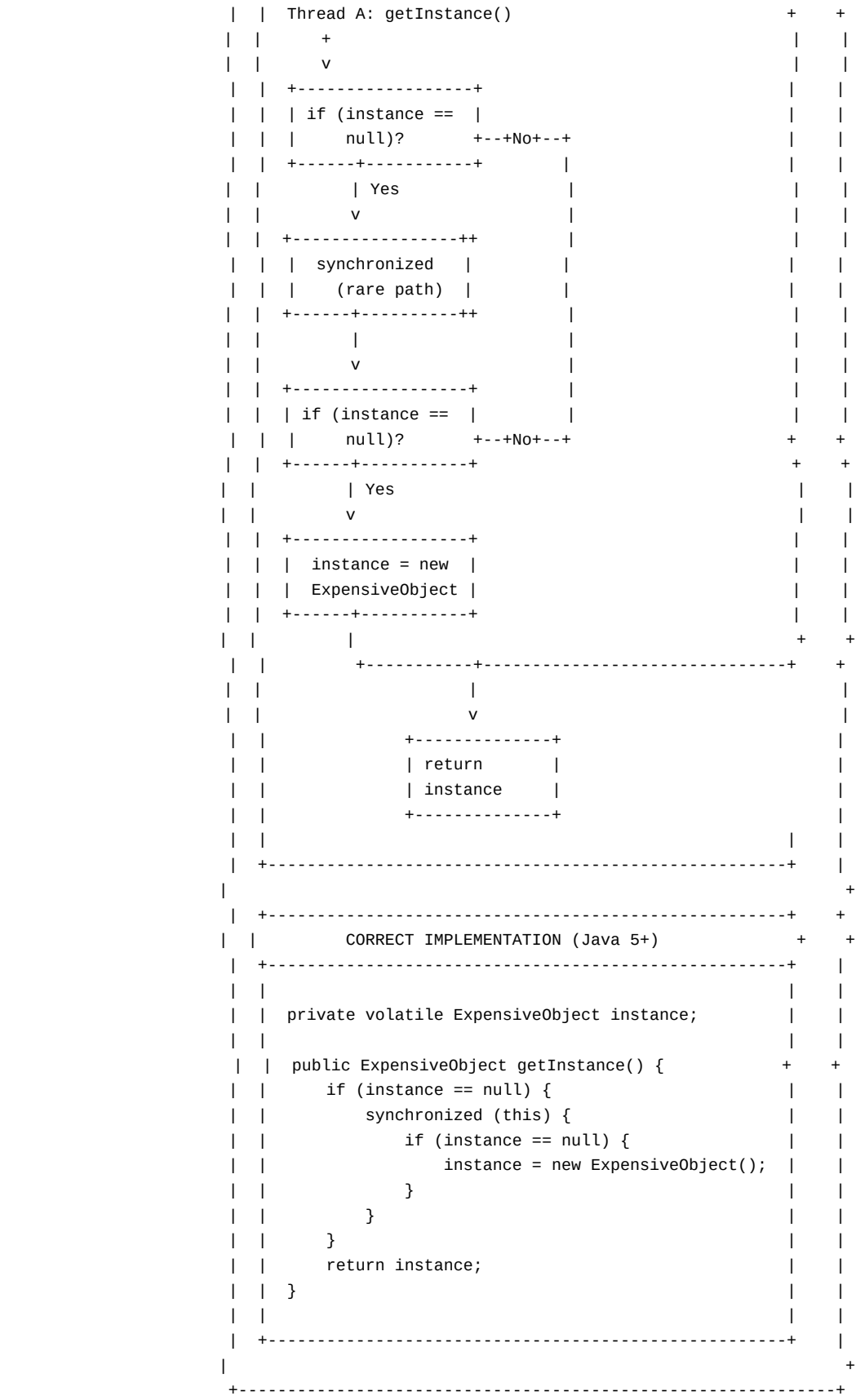
Lazy initialization is a common technique where you delay creating an expensive object until it's actually needed:

```
private ExpensiveObject instance;

public ExpensiveObject getInstance() {
    if (instance == null) {
        instance = new ExpensiveObject();
    }
    return instance;
}
```

This works fine in single-threaded code. But in multi-threaded code, two threads could both see instance == null and both create objects, violating the singleton property.
The naive fix is to synchronize the entire method:
public synchronized ExpensiveObject getInstance() {
if (instance == null) {
instance = new ExpensiveObject();
}
return instance;
}
This works but has a performance problem: after the instance is initialized, every call still acquires a lock, causing unnecessary synchronization overhead.
The Solution
Double-Checked Locking checks the condition twice:
1. First check without locking (fast path)
2. If instance is null, acquire lock
3. Check again inside lock (second check)
4. Create instance if still null
After initialization, the first check prevents any locking, eliminating synchronization overhead.
The Problem with Double-Checked Locking
In early Java versions (before Java 5), this pattern was broken due to the Java Memory Model. The problem was instruction reordering: the constructor could execute out of order, and another thread might see a partially constructed object.
Modern solutions use `volatile` (Java 5+) or atomic operations to fix this.
Diagram: Double-Checked Locking





Why Volatile is Needed

Without volatile, this sequence can happen:

1. Thread A enters synchronized block, sees instance == null
2. Thread A starts constructing instance (memory allocated)
3. Before constructor finishes, Thread A assigns instance to the memory
4. Thread B enters method, sees instance not null (first check)
5. Thread B returns partially constructed object (bad!)

Volatile ensures that the assignment happens after construction completes (happens-before relationship).

Alternative: Initialization-on-Demand Holder Idiom

A simpler, foolproof approach in Java:

```
public class Singleton {
    private Singleton() {}

    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

This uses class loading guarantees: the Holder class is loaded only when getInstance() is called, and class loading is thread-safe.

When to Use Double-Checked Locking

Suitable Situations:

1. When implementing lazy initialization in performance-critical code.

2. When you have an expensive object that may or may not be used.

3. When you need thread-safe singletons.

4. When you're using Java 5+ (with volatile support).

Situations to Avoid:

1. In Java versions before 5 (pattern is broken).

2. When simpler alternatives work (static initialization, holder idiom).

3. When object creation is cheap.

4. When you don't need lazy initialization.

Real-World Use Cases:

- **Singleton Pattern:** Thread-safe lazy singleton
- **Resource Pools:** Lazy creation of connection pools
- **Cache Initialization:** Lazy loading of cache data
- **Configuration Objects:** Load config only when first accessed
- **Plugin Systems:** Load plugins on demand

Advantages of Double-Checked Locking

1. **Performance:** No synchronization after initialization.

2. **Lazy Loading:** Object created only when needed.

3. **Thread Safety:** Correct implementation ensures safety.

4. **Flexibility:** Can be used for any lazy initialization, not just singletons.

Disadvantages of Double-Checked Locking

1. **Complexity:** Easy to get wrong (especially in older JVMs).

2. **Volatile Required:** Adds small overhead to each read.

3. **Not Needed in Many Cases:** Static initialization often sufficient.

4. **Memory Model Dependent:** Relies on language memory model guarantees.

5. **Testing Difficulty:** Race conditions hard to reproduce.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Double-Checked Locking	Optimized lazy init	Checks twice, locks once
Eager Initialization	Create at startup	Simpler, no lazy loading
Holder Idiom	Classloader-based	Simpler, no synchronization
AtomicReference	CAS-based init	Non-blocking alternative

Double-Checked Locking vs Holder Idiom

- **Double-Checked Locking:** More complex, works for any lazy init, not just singletons
- **Holder Idiom:** Simpler, but only works for static singletons

Double-Checked Locking vs AtomicReference

```
private AtomicReference<ExpensiveObject> ref = new AtomicReference<>();

public ExpensiveObject getInstance() {
    ExpensiveObject obj = ref.get();
    if (obj == null) {
        obj = new ExpensiveObject();
        if (!ref.compareAndSet(null, obj)) {
            obj = ref.get(); // someone else created it
        }
    }
    return obj;
}
```

This is non-blocking but may create multiple objects (though only one wins).

Common Pitfalls

1. **Missing Volatile:** Without volatile, pattern is broken.

2. **Using Double-Checked Locking with Non-Volatile:** Still broken.

3. **Assuming Works in All Languages:** Different memory models.

4. **Overusing:** Using when simpler solution exists.

5. **Not Handling Exceptions:** Constructor may throw.

Best Practices

1. **Use Java 5+ with volatile:** Guarantees correctness.

2. **Consider Holder Idiom:** Often simpler for singletons.

3. **Document Why Used:** Explain the performance reasoning.

4. **Test Under Load:** Race conditions may only appear under concurrency.

5. **Profile Before Optimizing:** Ensure this optimization is actually needed.

[Page 12]

Chapter 10: Read-Write Lock Pattern

Detailed Explanation

The Read-Write Lock pattern allows multiple threads to read a shared resource simultaneously while ensuring exclusive access for write operations. This optimizes concurrency when reads are more frequent than writes.

Understanding the Problem

Consider a data structure like a cache or a configuration store that is read frequently but updated rarely. Using a simple mutex would serialize all access, including read operations that could safely run in parallel.

For example:

- A configuration service is read thousands of times per second
- Configuration is updated only a few times per day
- With a mutex, all reads are serialized, creating unnecessary contention

The Solution

The Read-Write Lock pattern distinguishes between read access and write access:

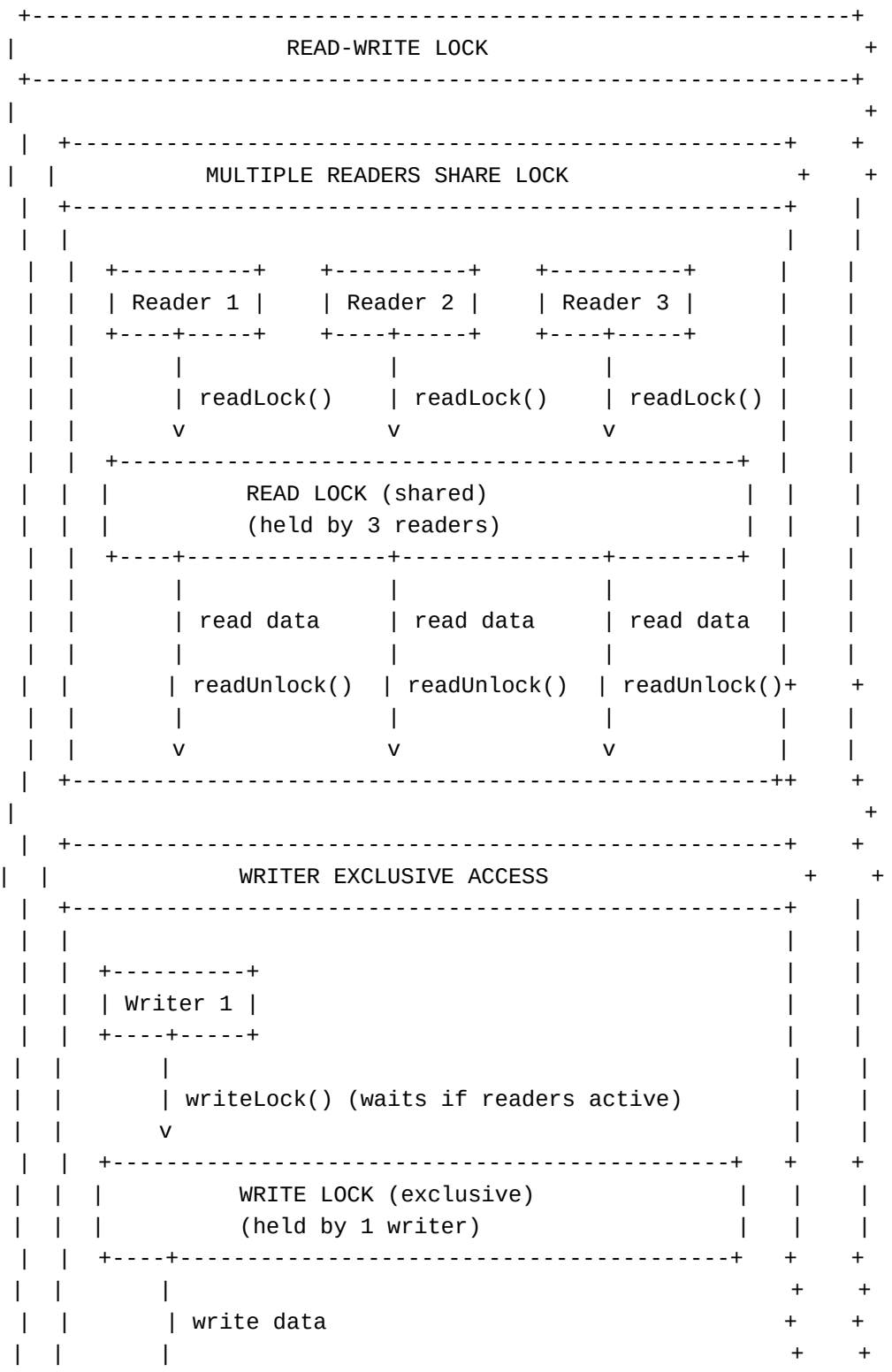
- **Read Lock:** Multiple threads can hold the read lock simultaneously (shared access)

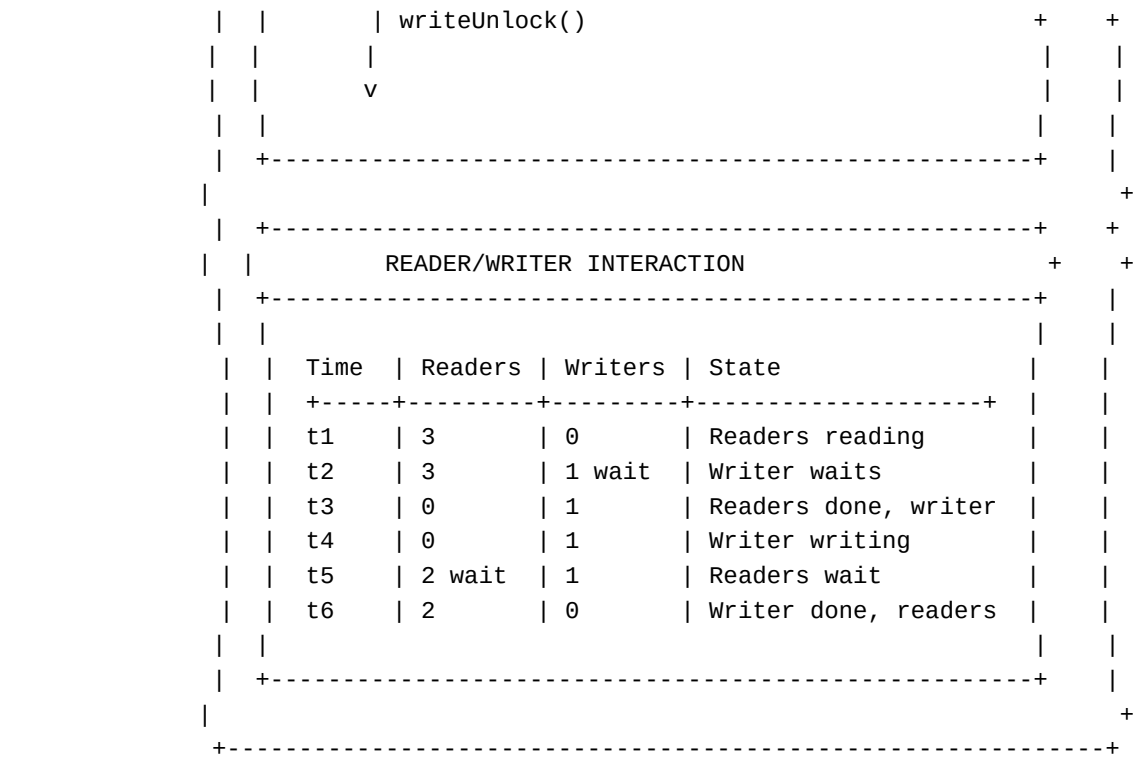
- **Write Lock:** Only one thread can hold the write lock, and no readers are allowed (exclusive access)

Locking Rules

1. Any number of threads can acquire the read lock simultaneously.
2. Only one thread can acquire the write lock.
3. No thread can acquire the write lock while any read lock is held.
4. No thread can acquire a read lock while the write lock is held.
5. Writers are usually given priority to prevent starvation.

Diagram: Read-Write Lock





Implementation Example

```
import java.util.concurrent.locks.ReentrantReadWriteLock;
public class ConfigurationStore {
private final ReentrantReadWriteLock rwLock =
new ReentrantReadWriteLock();
private final ReentrantReadWriteLock.ReadLock readLock =
rwLock.readLock();
private final ReentrantReadWriteLock.WriteLock writeLock =
rwLock.writeLock();

private String configuration;

public String getConfiguration() {
readLock.lock();
try {
return configuration;
} finally {
readLock.unlock();
}
}

public void updateConfiguration(String newConfig) {
writeLock.lock();
try {
configuration = newConfig;
} finally {
writeLock.unlock();
}
}
}
```

Custom Simple Implementation

```
public class SimpleReadWriteLock {
private int readers = 0;
private boolean writer = false;
```

```
    private int writeRequests = 0;

    public synchronized void lockRead() throws InterruptedException {
        while (writer || writeRequests > 0) {
            wait(); // wait for writer to finish or no write requests
        }
        readers++;
    }

    public synchronized void unlockRead() {
        readers--;
        notifyAll(); // wake up waiting writers
    }

    public synchronized void lockWrite() throws InterruptedException {
        writeRequests++;
        while (readers > 0 || writer) {
            wait(); // wait for readers and other writer
        }
        writeRequests--;
        writer = true;
    }

    public synchronized void unlockWrite() {
        writer = false;
        notifyAll(); // wake up waiting readers/writers
    }
}
```

When to Use Read-Write Lock Pattern

Suitable Situations:

1. When reads are much more frequent than writes.

2. When read operations are not trivial (so lock contention matters).

3. When you need to protect a data structure with clear read/write separation.

4. When you can afford the additional complexity over a simple mutex.

5. When writer starvation is not a concern or can be addressed.

Real-World Use Cases:

- **Cache Implementations:** Frequently read, occasionally updated
- **Configuration Services:** Read often, updated rarely
- **Database Connection Pools:** Connection status read often
- **In-Memory Data Grids:** Shared data with read-heavy access

- **Routing Tables:** Network routing tables, read often, updated occasionally

Advantages of Read-Write Lock Pattern

1. **Higher Concurrency:** Multiple readers can proceed in parallel.

2. **Performance:** Reduces contention in read-heavy workloads.

3. **Flexibility:** Can prioritize readers or writers.

4. **Reentrancy:** Many implementations allow same thread to re-acquire.

5. **Fairness Options:** Can configure fair vs. unfair locking.

Disadvantages of Read-Write Lock Pattern

1. **Complexity:** More complex than simple mutex.

2. **Writer Starvation:** If readers are continuous, writers may never get lock.

3. **Overhead:** Read-write locks have more overhead than simple locks.

4. **Potential for Deadlock:** Upgrading from read to write lock can deadlock.

5. **Not Always Faster:** For short operations, mutex may be faster.

Comparison with Other Patterns

Pattern	Purpose	Difference
Mutex	Exclusive access	All access serialized
Read-Write Lock	Shared reads, exclusive writes	Optimizes for read-heavy
Semaphore	Multiple permits	Can limit concurrent readers
Stamped Lock	Optimistic reads	Java 8+, even higher concurrency

Read-Write Lock vs Mutex

- **Mutex:** All operations serialized (simpler)
- **Read-Write Lock:** Reads parallel, writes serialized (more concurrent)

Read-Write Lock vs Stamped Lock

Java 8 introduced StampedLock with optimistic reading:

```
StampedLock lock = new StampedLock();

// Optimistic read - no locking!
long stamp = lock.tryOptimisticRead();
String data = configuration;
if (!lock.validate(stamp)) {
    // Lock was acquired - fall back to regular read lock
    stamp = lock.readLock();
    try {
        data = configuration;
    } finally {
        lock.unlockRead(stamp);
    }
}
```

This can be even faster when contention is low.

Common Pitfalls

1. **Lock Upgrade:** Trying to upgrade read lock to write lock (deadlock risk).

2. **Writer Starvation:** Continuous readers prevent writers.

3. **Not Releasing Locks:** Forgetting to unlock in finally blocks.

4. **Assuming Fairness:** Default may be unfair (writers may starve).

5. **Over-optimizing:** Using when mutex would be sufficient.

Best Practices

1. **Use try-finally:** Always release locks in finally blocks.

2. **Consider Lock Downgrade:** Can downgrade write to read (safe).

3. **Monitor Contention:** Measure if read-write lock is helping.

4. **Choose Fairness Appropriately:** Fair locks prevent starvation but are slower.

5. **Consider Alternatives:** For very short reads, mutex might be faster.

[Page 13]

Chapter 11: Monitor Object Pattern

Detailed Explanation

The Monitor Object pattern combines mutual exclusion (mutex) with condition synchronization. It ensures that only one thread can execute a method at a time, and provides mechanisms for threads to wait for specific conditions.

Understanding the Problem

In concurrent programming, you often need both:

- **Mutual Exclusion:** Only one thread at a time should execute certain methods
- **Condition Synchronization:** Threads need to wait until certain conditions are true

For example, in a bounded buffer:

- Only one thread should modify the buffer at a time (mutual exclusion)
- Producers must wait if the buffer is full (condition)
- Consumers must wait if the buffer is empty (condition)

Using separate mutex and condition variables works, but the pattern of acquiring mutex, checking condition, waiting, and releasing is common enough to warrant a unified abstraction.

The Solution

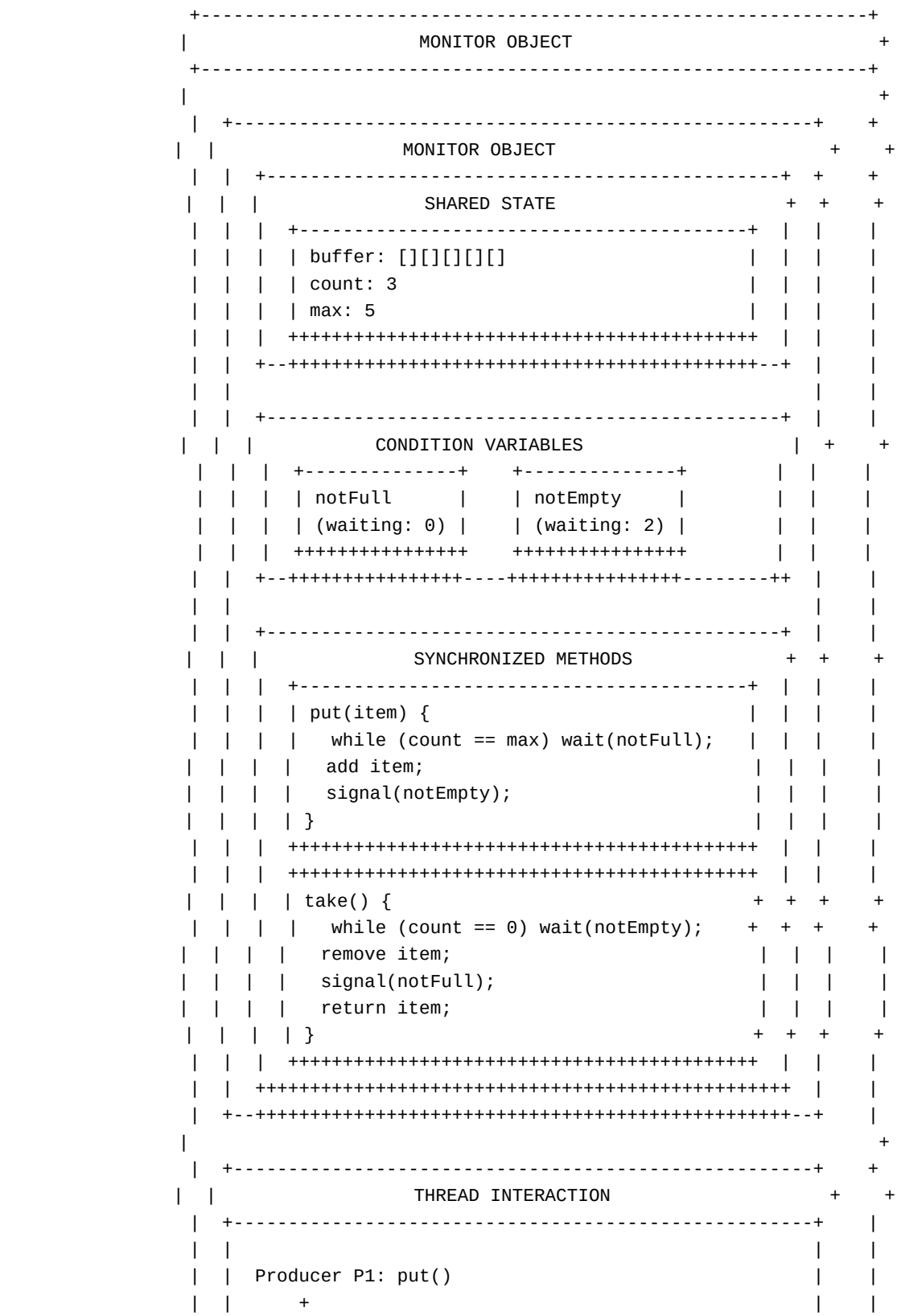
The Monitor Object pattern provides:

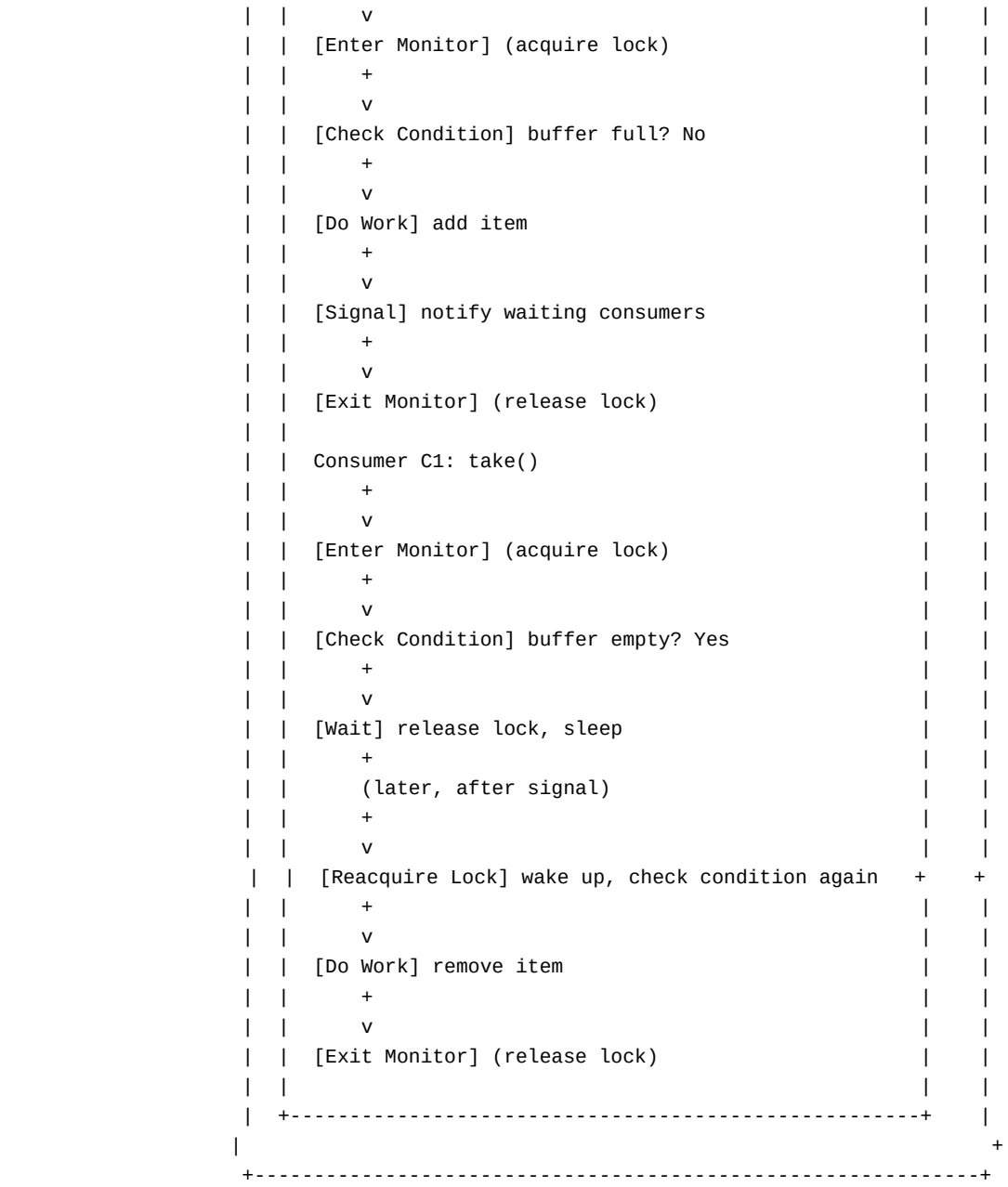
1. **Mutex:** All methods of the monitor are synchronized (only one thread can execute any method at a time)
2. **Condition Variables:** Threads can wait for specific conditions and be notified when conditions change
3. **Encapsulation:** The synchronization logic is encapsulated within the monitor object

Key Concepts

- **Monitor:** An object whose methods are executed with mutual exclusion
- **Condition Variable:** Allows threads to wait for a condition and be signaled
- **Wait:** Thread releases monitor lock and sleeps until signaled
- **Signal/Notify:** Wakes one waiting thread (which must reacquire lock)
- **Broadcast/NotifyAll:** Wakes all waiting threads

Diagram: Monitor Object





Implementation Example (Java)
In Java, every object can be a monitor using `synchronized` methods/blocks and `wait()/notify()`:

```
public class BoundedBuffer {
private final Object[] buffer;
private int count = 0;
private int putIndex = 0;
private int takeIndex = 0;

public BoundedBuffer(int capacity) {
buffer = new Object[capacity];
}

public synchronized void put(Object item)
throws InterruptedException {
while (count == buffer.length) {
wait(); // wait while buffer full
}
```

```
            buffer[putIndex] = item;
            putIndex = (putIndex + 1) % buffer.length;
            count++;

            notifyAll(); // notify waiting consumers
        }

        public synchronized Object take()
                throws InterruptedException {
            while (count == 0) {
                wait(); // wait while buffer empty
            }

            Object item = buffer[takeIndex];
            takeIndex = (takeIndex + 1) % buffer.length;
            count--;

            notifyAll(); // notify waiting producers

            return item;
        }
    }
```

Using explicit Lock and Condition (Java)

```
    import java.util.concurrent.locks.*;

    public class BoundedBuffer {
        private final Lock lock = new ReentrantLock();
        private final Condition notFull = lock.newCondition();
        private final Condition notEmpty = lock.newCondition();

        private final Object[] buffer;
        private int count = 0;
        private int putIndex = 0;
        private int takeIndex = 0;

        public BoundedBuffer(int capacity) {
            buffer = new Object[capacity];
        }

        public void put(Object item) throws InterruptedException {
            lock.lock();
            try {
                while (count == buffer.length) {
                    notFull.await();
                }

                buffer[putIndex] = item;
                putIndex = (putIndex + 1) % buffer.length;
                count++;

                notEmpty.signal();
            } finally {
                lock.unlock();
            }
        }

        public Object take() throws InterruptedException {
            lock.lock();
            try {
                while (count == 0) {
                    notEmpty.await();
```

```
                }

                Object item = buffer[takeIndex];
                takeIndex = (takeIndex + 1) % buffer.length;
                count--;

                notFull.signal();

                return item;
            } finally {
                lock.unlock();
            }
        }
    }
```

When to Use Monitor Object Pattern

Suitable Situations:

1. When you have shared data that needs mutual exclusion.

2. When threads need to wait for specific conditions.

3. When you want to encapsulate synchronization logic with the data.

4. In classic producer-consumer problems.

5. When building thread-safe data structures.

Real-World Use Cases:

- **Bounded Buffers:** Producer-consumer queues
- **Thread Pools:** Workers wait for tasks
- **Connection Pools:** Threads wait for available connections
- **Blocking Queues:** Java's BlockingQueue implementations
- **Resource Managers:** Managing limited resources

Advantages of Monitor Object Pattern

1. **Encapsulation:** Synchronization logic is inside the object.

2. **Simplicity:** Clean separation of concerns.

3. **Language Support:** Built into many languages (Java, C#).

4. **Familiar Pattern:** Well-understood by developers.

5. **Condition Management:** Explicit wait/notify for conditions.

Disadvantages of Monitor Object Pattern

1. **Single Lock:** All methods synchronized on same lock (can limit concurrency).

2. **Potential for Deadlock:** Can still deadlock if not careful.

3. **Nested Monitor Problem:** Can cause issues with nested monitors.

4. **Spurious Wakeups:** wait() can return without signal (must check in loop).

5. **Not Reentrant by Default:** Some implementations may not be reentrant.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
---	---	---
Monitor	Mutual exclusion + condition	Complete synchronization unit
Mutex	Mutual exclusion only	No condition waiting
Semaphore	Counting permits	Can be used for both
Guarded Suspension	Wait for condition	Often implemented with monitor

Monitor Object vs Active Object

- **Monitor:** Methods execute in caller's thread with mutual exclusion
- **Active Object:** Methods execute in different thread, returns futures

Monitor Object vs Read-Write Lock

- **Monitor:** Single lock for all methods
- **Read-Write Lock:** Different locks for read/write

Common Pitfalls

1. **Not Checking Condition in Loop:** Must use while, not if.

2. **Signaling While Not Holding Lock:** Must hold lock when signaling.

3. **Nested Monitor Deadlock:** Monitor inside monitor can cause deadlock.

4. **Lost Wakeup:** Signaling before thread waits.

5. **Not Handling InterruptedException:** Threads may be interrupted.

Best Practices

1. **Always Use While Loop:** while (!condition) wait();

2. **Signal After State Change:** Only signal when condition actually changed.

3. **Minimize Work in Synchronized Block:** Release lock quickly.

4. **Use notifyAll() Unless Sure:** Prevents missed signals.

5. **Document Waiting Conditions:** Make clear what threads wait for.

[Page 14]

Chapter 12: Guarded Suspension Pattern

Detailed Explanation

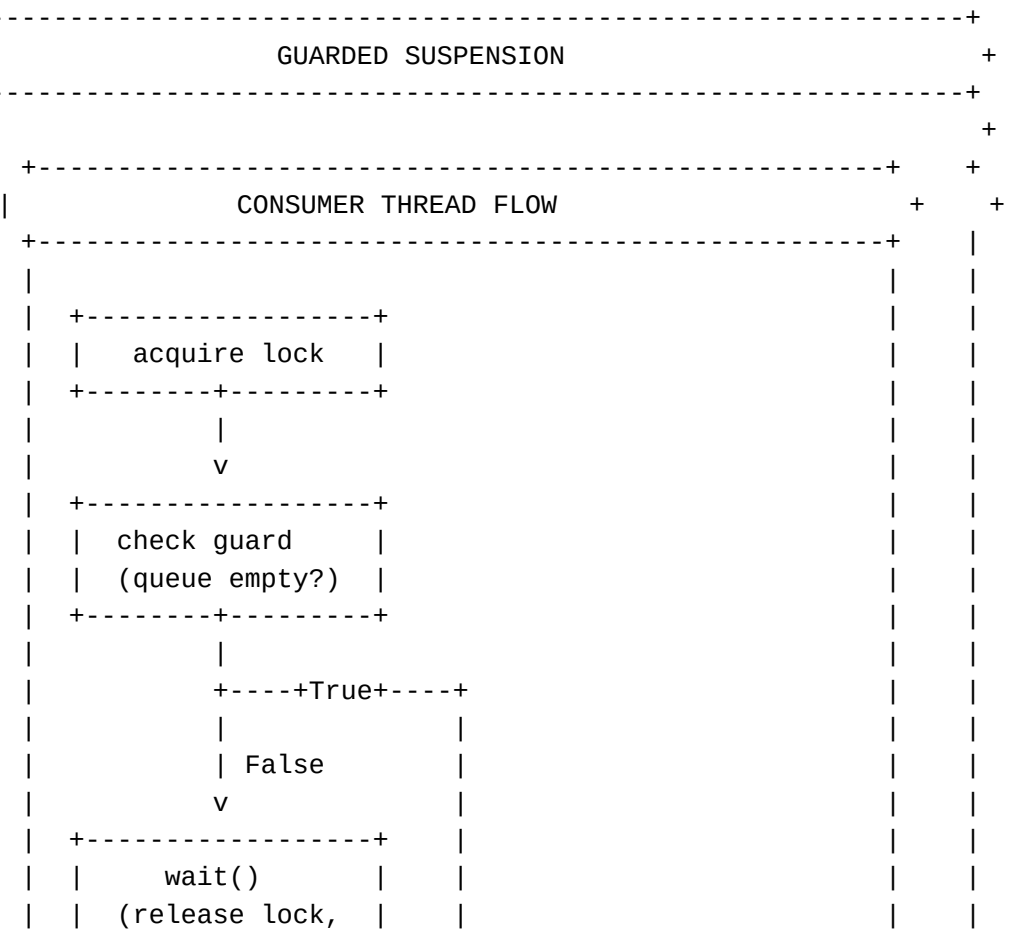
The Guarded Suspension pattern manages operations that must wait until a certain condition is true. Threads suspend execution (wait) until the condition becomes true, at which point they are notified and continue.

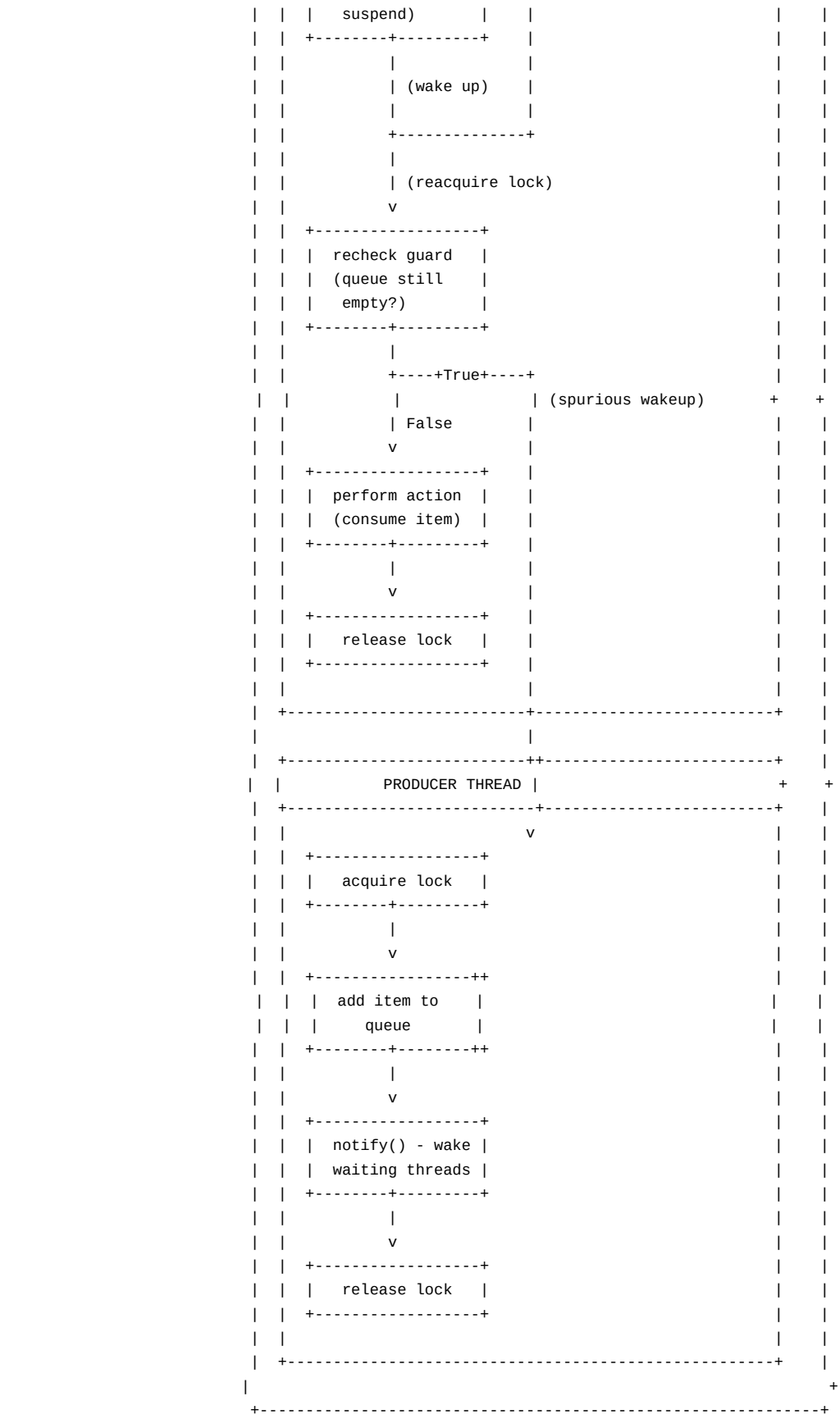
Understanding the Problem

In concurrent systems, operations often depend on conditions that may not be immediately true:

- A consumer cannot consume from an empty queue
- A client cannot get a connection from an empty pool

- A thread cannot proceed until some initialization is complete
The naive approach is busy-waiting (polling), which wastes CPU cycles. A better approach is to suspend the thread until the condition becomes true.
The Solution
The Guarded Suspension pattern provides a mechanism for threads to:
1. Check a condition (the guard)
2. If false, suspend (wait) until notified
3. When notified, recheck condition (in case of spurious wakeup)
4. When true, proceed with operation
Key Concepts
- **Guard:** The condition that must be true to proceed
- **Suspension:** Thread waits efficiently (doesn't consume CPU)
- **Notification:** Another thread signals that condition may be true
- **Spurious Wakeup:** Thread may wake without notification (must recheck)
Diagram: Guarded Suspension





Implementation Example

```
public class GuardedQueue<T> {
```

```
    private final Queue<T> queue = new LinkedList<>();
    private final Object lock = new Object();

    public T take() throws InterruptedException {
        synchronized(lock) {
            // Guarded suspension: wait while queue empty
            while (queue.isEmpty()) {
                lock.wait(); // suspend until notified
            }
            return queue.poll();
        }
    }

    public void put(T item) {
        synchronized(lock) {
            queue.add(item);
            lock.notify(); // wake one waiting thread
        }
    }
}
```

Generic Guarded Suspension Pattern

```
public abstract class GuardedBlock<T> {
    private final Object lock = new Object();
    private volatile boolean condition = false;
    private T result;

    public T execute() throws InterruptedException {
        synchronized(lock) {
            while (!conditionMet()) {
                lock.wait();
            }
            return getResult();
        }
    }

    public void signal(T result) {
        synchronized(lock) {
            this.result = result;
            this.condition = true;
            lock.notifyAll();
        }
    }

    protected abstract boolean conditionMet();
    protected T getResult() { return result; }
}
```

When to Use Guarded Suspension Pattern

Suitable Situations:

1. When threads need to wait for conditions to become true.

2. When busy-waiting (polling) is inefficient.

3. In producer-consumer scenarios.

4. When implementing resource pools.

5. When threads have dependencies on each other's state.

Real-World Use Cases:

- **Blocking Queue:** Consumer waits for items
- **Connection Pool:** Client waits for available connection
- **Initialization:** Thread waits for service to be ready
- **Lazy Loading:** Wait until data is loaded
- **Pipeline Stages:** Wait for previous stage to complete

Advantages of Guarded Suspension Pattern

1. **Efficient:** Threads sleep, not busy-wait.

2. **Simple:** Clean separation of condition checking.

3. **Language Support:** Built into many languages.

4. **Decoupled:** Producers and consumers don't need to know each other.

5. **Flexible:** Can wait on any condition.

Disadvantages of Guarded Suspension Pattern

1. **Spurious Wakeups:** Must check condition in loop.

2. **Lost Wakeups:** Can happen if signal before wait.

3. **Deadlock Risk:** Can deadlock if conditions never met.

4. **Debugging Difficulty:** Hard to trace wait/notify chains.

5. **Not Reusable:** Each condition needs its own guard.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Guarded Suspension	Wait for condition	General pattern
Monitor Object	Mutual exclusion + condition	Specific implementation
Balking	Abort if condition not met	Opposite (don't wait)
Semaphore	Counting permits	Can implement guarding

Guarded Suspension vs Balking

- **Guarded Suspension:** Wait until condition is true
- **Balking:** Return immediately if condition not true

Guarded Suspension vs Producer-Consumer

Guarded Suspension is a key part of the Producer-Consumer pattern. Producer-Consumer is a higher-level pattern that uses guarded suspension.

Common Pitfalls

1. **Using if Instead of while:** Must use while for condition check.

2. **Notifying Outside Lock:** Must hold lock when notifying.

3. **Lost Wakeup:** Signaling before thread starts waiting.

4. **Spurious Wakeups:** Not rechecking condition.

5. **Deadlock:** If condition never becomes true.

Best Practices

1. **Always Use While Loop:** while (!condition) wait();

2. **Hold Lock When Checking Condition:** Condition must be checked under lock.

3. **Hold Lock When Notifying:** Notify must be inside synchronized block.

4. **Use notifyAll() Unless Sure:** Prevents lost wakeups.

5. **Document Guard Conditions:** Make clear what threads are waiting for.

6. **Consider Timeouts:** Use timed wait to avoid waiting forever.

[Page 15]

PART THREE: TASK DISTRIBUTION PATTERNS

Chapter 13: Thread Pool Pattern

Detailed Explanation

The Thread Pool pattern maintains a pool of worker threads that can execute tasks concurrently. Instead of creating a new thread for each task, tasks are submitted to a queue, and idle worker threads pick them up and execute them.

Understanding the Problem

Creating a new thread for each task has several drawbacks:

1. **Overhead:** Thread creation and destruction are expensive operations.

2. **Resource Limits:** The OS has limits on the number of threads.

3. **Unbounded Growth:** Under load, you might create thousands of threads, overwhelming the system.

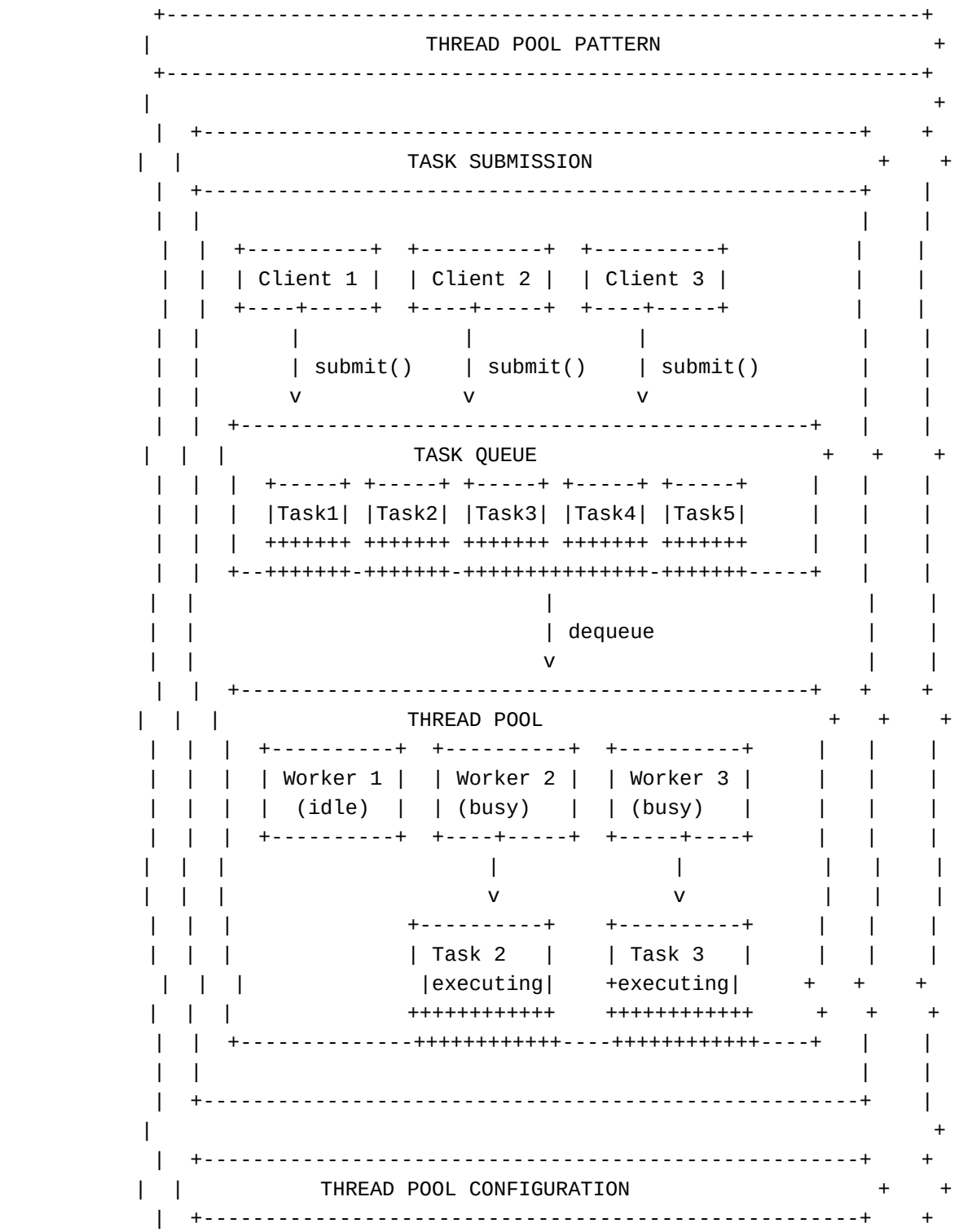
4. **Performance:** Too many threads cause context switching overhead.

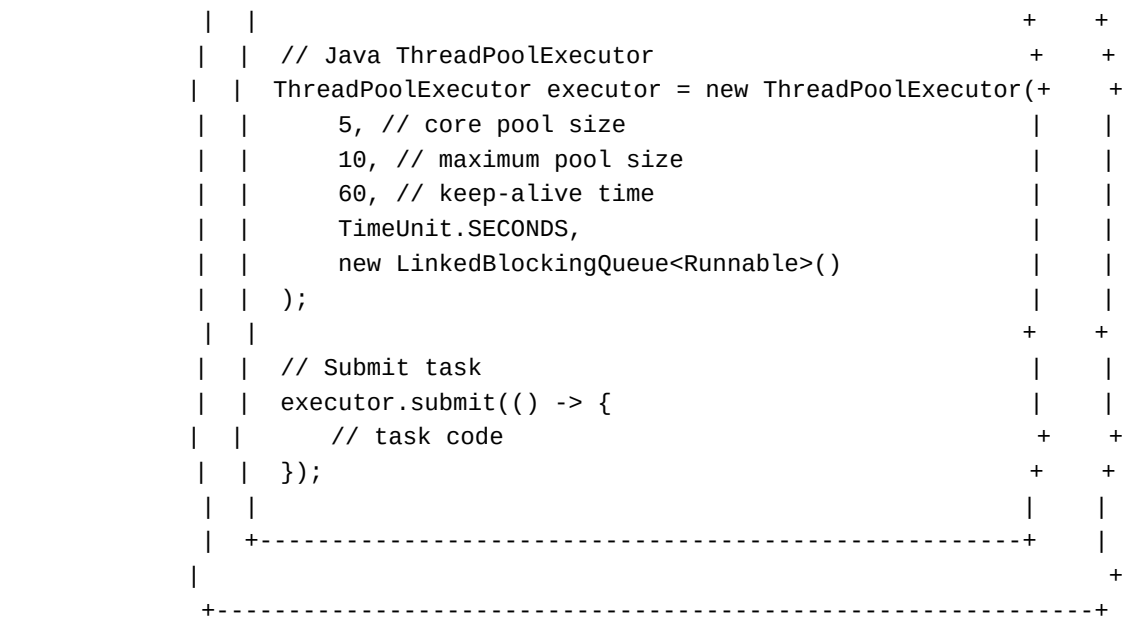
5. **Latency:** Creating a thread adds latency before task execution.

The Solution

The Thread Pool pattern pre-creates a fixed number of threads that wait for work. Tasks are submitted to a queue. Idle threads take tasks from the queue and execute them. After

completing a task, the thread returns to the pool to wait for the next task.
Key Concepts
- **Thread Pool:** Collection of worker threads
- **Task Queue:** Queue of pending tasks
- **Worker Thread:** Thread that takes tasks from queue and executes them
- **Core Pool Size:** Minimum number of threads to keep alive
- **Maximum Pool Size:** Maximum number of threads allowed
- **Keep-Alive Time:** How long idle threads survive
Diagram: Thread Pool





Implementation Example

```
import java.util.concurrent.*;
public class ThreadPoolExample {
private final ExecutorService threadPool;
private final BlockingQueue<Runnable> taskQueue;

public ThreadPoolExample(int poolSize, int queueSize) {
this.taskQueue = new LinkedBlockingQueue<>(queueSize);
this.threadPool = new ThreadPoolExecutor(
poolSize,                 // core pool size
poolSize,                 // max pool size
60L, TimeUnit.SECONDS,        // keep alive time
taskQueue,                  // work queue
new ThreadPoolExecutor.AbortPolicy() // rejection policy
);
}

public void submitTask(Runnable task) {
threadPool.submit(task);
}

public void shutdown() {
threadPool.shutdown();
try {
if (!threadPool.awaitTermination(60, TimeUnit.SECONDS)) {
threadPool.shutdownNow();
}
} catch (InterruptedException e) {
threadPool.shutdownNow();
}
}
}
```

Custom Simple Thread Pool

```
public class SimpleThreadPool {
private final BlockingQueue<Runnable> taskQueue;
private final List<Worker> workers;
private volatile boolean isRunning = true;
public SimpleThreadPool(int numThreads, int queueSize) {
```

```
            taskQueue = new LinkedBlockingQueue<>(queueSize);
            workers = new ArrayList<>();

            for (int i = 0; i < numThreads; i++) {
                Worker worker = new Worker();
                workers.add(worker);
                worker.start();
            }
        }

        public void submit(Runnable task) throws InterruptedException {
            if (!isRunning) {
                throw new RejectedExecutionException("Pool is shutdown");
            }
            taskQueue.put(task);
        }

        public void shutdown() {
            isRunning = false;
            for (Worker worker : workers) {
                worker.interrupt();
            }
        }

        private class Worker extends Thread {
            public void run() {
                while (isRunning) {
                    try {
                        Runnable task = taskQueue.take(); // blocks if empty
                        task.run();
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        break;
                    }
                }
            }
        }
    }
```

Thread Pool Sizing

The optimal pool size depends on the workload:

CPU-Bound Tasks:

- Pool size ≈ Number of CPU cores

- More threads cause context switching overhead

I/O-Bound Tasks:

- Pool size can be larger (threads spend time waiting)

- Formula: cores × (1 + wait time / service time)

Mixed Workloads:

- Consider separate pools for different task types

Rejection Policies

When the queue is full and all threads are busy, new tasks must be rejected:

1. **AbortPolicy:** Throws RejectedExecutionException
2. **CallerRunsPolicy:** Executes task in caller's thread
3. **DiscardPolicy:** Silently discards task
4. **DiscardOldestPolicy:** Discards oldest task in queue

When to Use Thread Pool Pattern

Suitable Situations:

1. When you have many short-lived tasks.

2. When you want to limit resource usage.

3. In server applications handling many requests.

4. When thread creation overhead is significant.

5. When you need to control concurrency level.

Real-World Use Cases:

- **Web Servers:** Handle HTTP requests
- **Database Connection Pools:** Manage connections
- **Background Job Processors:** Process queued jobs
- **Event Handlers:** Process events asynchronously
- **Parallel Processing:** Process data in parallel

Advantages of Thread Pool Pattern

1. **Reduced Overhead:** Threads are reused, not created per task.

2. **Resource Control:** Limits on threads prevent resource exhaustion.

3. **Improved Responsiveness:** No thread creation latency.

4. **Load Leveling:** Queue smooths out bursty loads.

5. **Thread Management:** Pool handles thread lifecycle.

6. **Monitoring:** Can track queue size, active threads.

Disadvantages of Thread Pool Pattern

1. **Complexity:** More complex than creating threads on demand.

2. **Tuning Difficulty:** Right pool size is hard to determine.

3. **Queue Bounds:** Need to decide queue size and rejection policy.

4. **Deadlock Risk:** If tasks depend on each other, pool can deadlock.

5. **Thread Leak:** If tasks don't complete, threads may leak.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
---	---	---
Thread Pool	Reuse threads	Fixed set of workers
Worker Pool	Similar	Often used interchangeably
Fork-Join	Divide and conquer	Specialized for recursive tasks
Active Object	Decouple invocation	Method execution in different thread

Thread Pool vs Fork-Join Pool

- **Thread Pool:** General-purpose, any tasks
- **Fork-Join Pool:** Specialized for divide-and-conquer, work stealing

Thread Pool vs Worker Pool

Often used interchangeably, but "worker pool" sometimes implies heterogeneous workers.

Common Pitfalls

1. **Pool Too Small:** Tasks queue up, response time suffers.

2. **Pool Too Large:** Too many threads cause thrashing.

3. **No Rejection Policy:** Tasks may be lost.

4. **Not Shutting Down:** Threads keep running, prevent JVM exit.

5. **Task Dependencies:** Tasks waiting for other tasks can deadlock pool.

Best Practices

1. **Size Pool Appropriately:** Consider task type (CPU vs I/O).

2. **Use Named Threads:** Helps with debugging.

3. **Handle Rejection:** Always have a rejection policy.

4. **Monitor Pool Metrics:** Queue size, active threads, completed tasks.

5. **Shutdown Gracefully:** Allow queued tasks to complete.

6. **Consider Work Stealing:** For recursive tasks, use ForkJoinPool.

7. **Don't Submit Tasks That Wait:** Can cause pool exhaustion.

[Page 16]

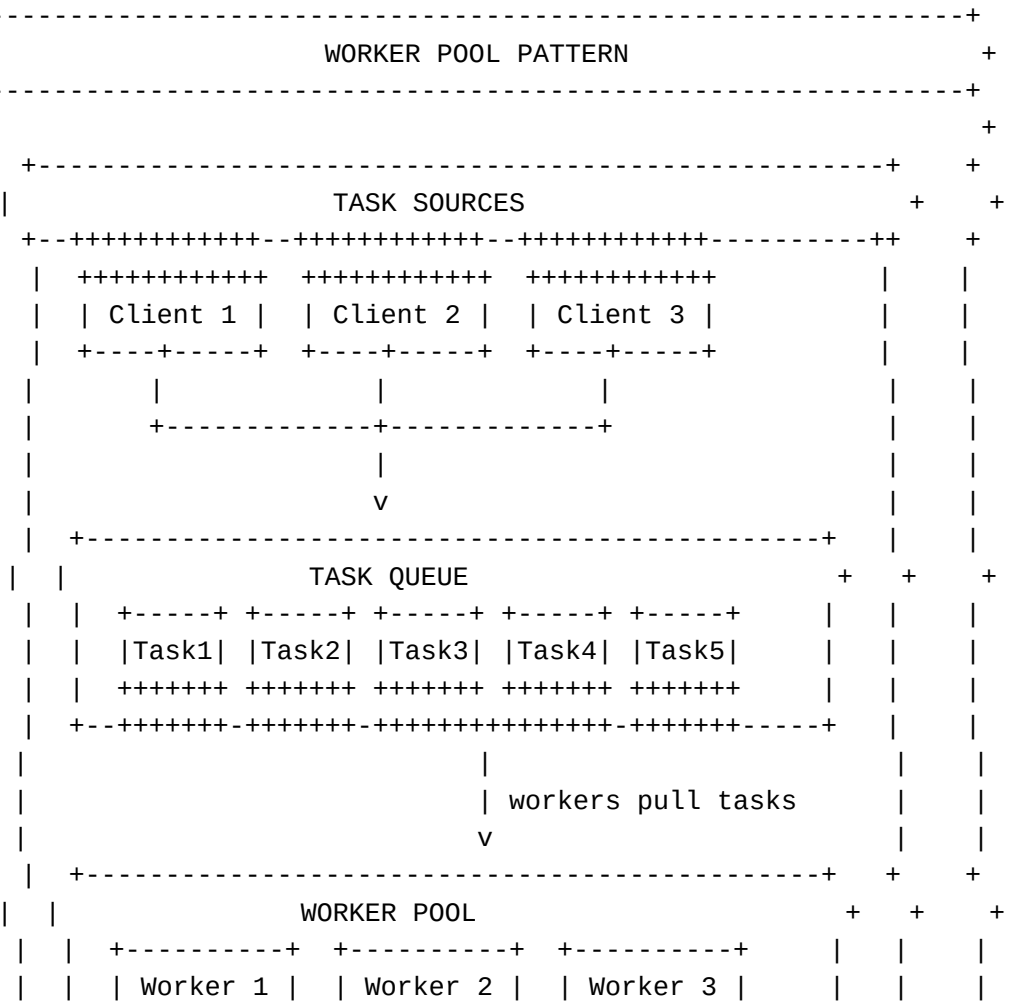
Chapter 14: Worker Pool Pattern

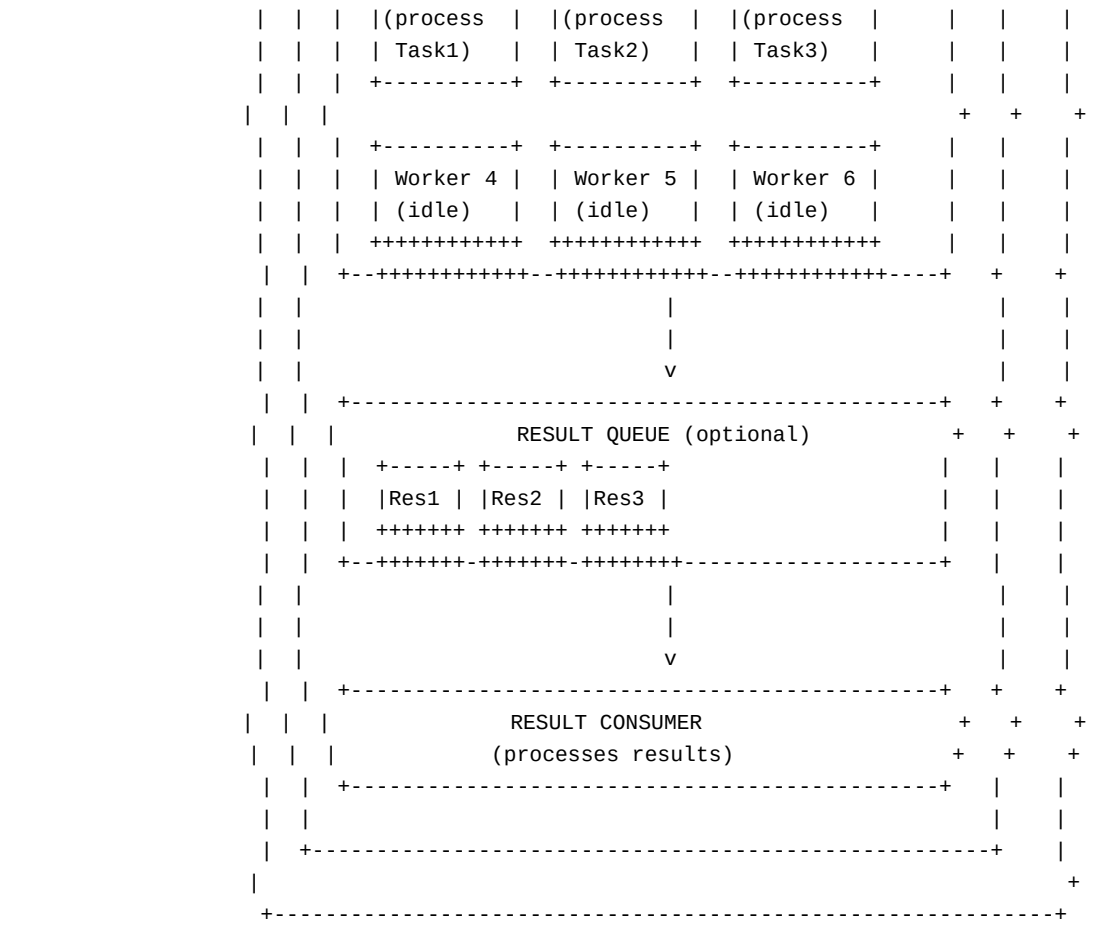
Detailed Explanation

The Worker Pool pattern is similar to Thread Pool but often implies a pool of identical workers that process tasks from a queue. Workers can be threads, processes, or even separate machines.

Understanding the Problem

In distributed and parallel systems, you often have a set of identical workers that can process tasks independently. You need a way to:
1. Distribute tasks among workers
2. Balance load across workers
3. Handle worker failures
4. Scale the number of workers dynamically
The Solution
The Worker Pool pattern creates multiple identical workers that pull tasks from a shared queue. Each worker processes one task at a time and returns to the pool when done.
Key Concepts
- **Task Queue:** Central queue of pending tasks
- **Workers:** Identical processing units
- **Dispatcher:** Optional component that assigns tasks (vs. workers pulling)
- **Result Queue:** Optional queue for collecting results
Diagram: Worker Pool





Implementation Example

```
public class WorkerPool<T, R> {
private final BlockingQueue<T> taskQueue;
private final BlockingQueue<R> resultQueue;
private final List<Worker> workers;
private volatile boolean running = true;

public WorkerPool(int numWorkers, int queueSize) {
this.taskQueue = new LinkedBlockingQueue<>(queueSize);
this.resultQueue = new LinkedBlockingQueue<>();
this.workers = new ArrayList<>();

                                    for (int i = 0; i < numWorkers; i++) {
Worker worker = new Worker();
workers.add(worker);
worker.start();
}
}

public void submitTask(T task) throws InterruptedException {
taskQueue.put(task);
}

public R takeResult() throws InterruptedException {
return resultQueue.take();
}

public void shutdown() {
running = false;
for (Worker worker : workers) {
worker.interrupt();
}
```

```
}

private class Worker extends Thread {
public void run() {
while (running) {
try {
T task = taskQueue.take();
R result = processTask(task);
resultQueue.put(result);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}

private R processTask(T task) {
// Override in subclass or provide processor
return null;
}
}
}
**Dynamic Worker Pool**
public class DynamicWorkerPool<T> {
private final BlockingQueue<T> taskQueue;
private final List<Worker> workers = new CopyOnWriteArrayList<>();
private final int minWorkers;
private final int maxWorkers;
private final int queueThreshold;

public DynamicWorkerPool(int min, int max, int threshold) {
this.minWorkers = min;
this.maxWorkers = max;
this.queueThreshold = threshold;
this.taskQueue = new LinkedBlockingQueue<>();

                                        for (int i = 0; i < min; i++) {
addWorker();
}

startMonitor();
}

private void addWorker() {
Worker w = new Worker();
workers.add(w);
w.start();
}

private void startMonitor() {
                                      Thread monitor = new Thread(() -> {
while (true) {
int queueSize = taskQueue.size();
int workerCount = workers.size();

if (queueSize > queueThreshold && workerCount < maxWorkers) {
addWorker();
} else if (queueSize == 0 && workerCount > minWorkers) {
removeWorker();
}
try {
```

```
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    break;
                }
            }
        });
        monitor.setDaemon(true);
        monitor.start();
    }

    private class Worker extends Thread {
        // Worker implementation
    }
}
```

When to Use Worker Pool Pattern

Suitable Situations:

1. When you have many independent tasks that can be processed in parallel.

2. When you need to scale processing horizontally.

3. In batch processing systems.

4. When tasks are CPU-intensive and benefit from parallelism.

5. When you need to process tasks from multiple sources.

Real-World Use Cases:

- **Image Processing:** Multiple workers process different images
- **Web Crawling:** Workers crawl different URLs
- **Data Transformation:** ETL pipelines
- **Report Generation:** Generate reports in parallel
- **Email Sending:** Send bulk emails

Advantages of Worker Pool Pattern

1. **Parallel Processing:** Multiple workers increase throughput.

2. **Scalability:** Can add more workers as load increases.

3. **Fault Tolerance:** Workers can fail independently.

4. **Load Balancing:** Natural load balancing as workers pull tasks.

5. **Backpressure:** Queue prevents overload.

Disadvantages of Worker Pool Pattern

1. **Complexity:** More complex than single-threaded processing.

2. **Resource Management:** Need to manage worker lifecycle.

3. **Result Collection:** Need to collect and combine results.

4. **Failure Handling:** Need to handle worker failures.

5. **Ordering:** Tasks may complete out of order.

Comparison with Other Patterns

Pattern	Purpose	Difference
Worker Pool	Parallel task processing	General pattern
Thread Pool	Reuse threads	Specific implementation
MapReduce	Large-scale data processing	Includes reduce phase
Fork-Join	Recursive decomposition	Work stealing

Worker Pool vs Thread Pool

Often used interchangeably, but "worker pool" can refer to processes or machines, not just threads.

Worker Pool vs MapReduce

- **Worker Pool:** General parallel processing
- **MapReduce:** Specialized for large datasets with map and reduce phases

Common Pitfalls

1. **Task Dependencies:** If tasks depend on each other, workers may deadlock.

2. **Resource Contention:** Workers competing for resources.

3. **Result Ordering:** Assuming results come back in order.

4. **Poison Tasks:** Tasks that never complete.

5. **Queue Overflow:** If producers outpace workers.

Best Practices

1. **Use Bounded Queues:** Prevent memory exhaustion.

2. **Implement Backpressure:** Slow down producers when queue is full.

3. **Handle Worker Failures:** Restart failed workers.

4. **Monitor Queue Size:** Alert if queue grows too large.

5. **Consider Task Priority:** Use priority queue if needed.

6. **Implement Graceful Shutdown:** Allow in-flight tasks to complete.

7. **Use Timeouts:** Prevent workers from hanging forever.

[Page 17]

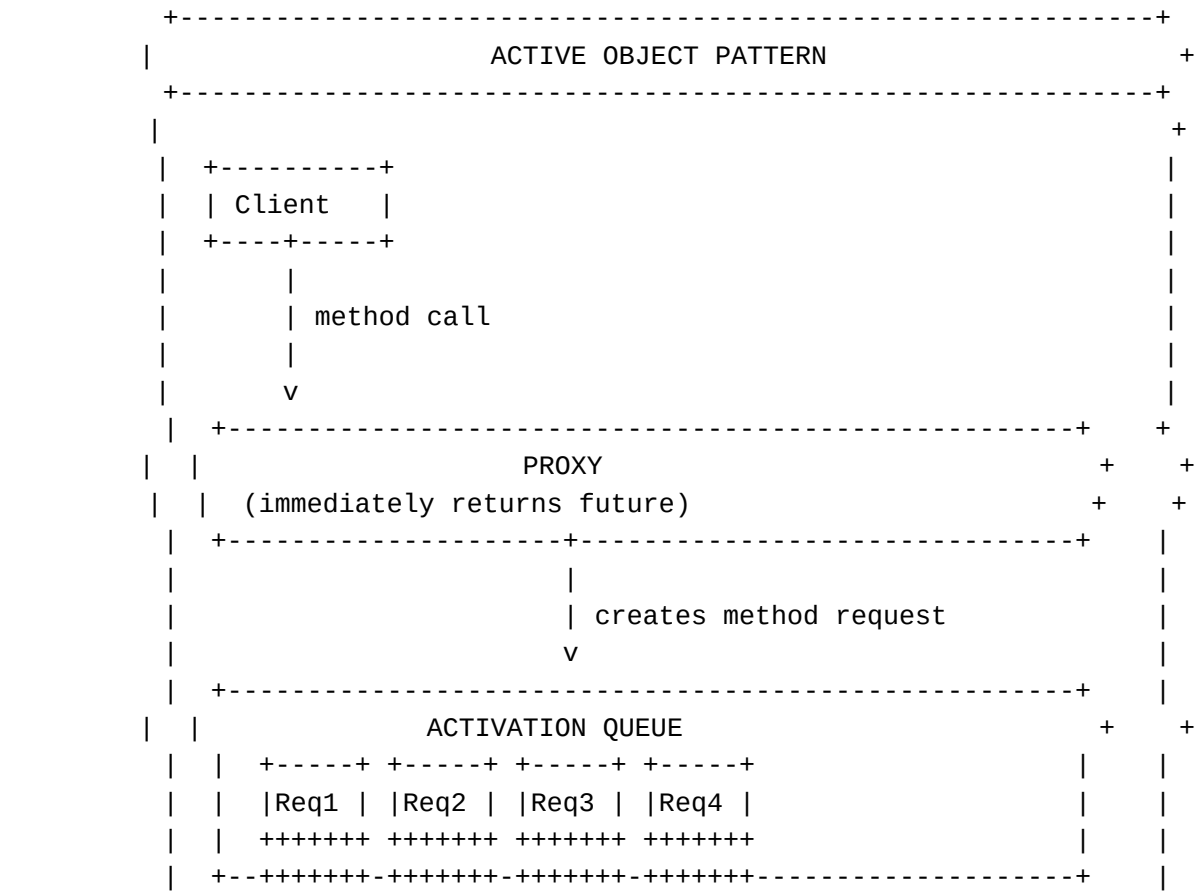
Chapter 15: Active Object Pattern

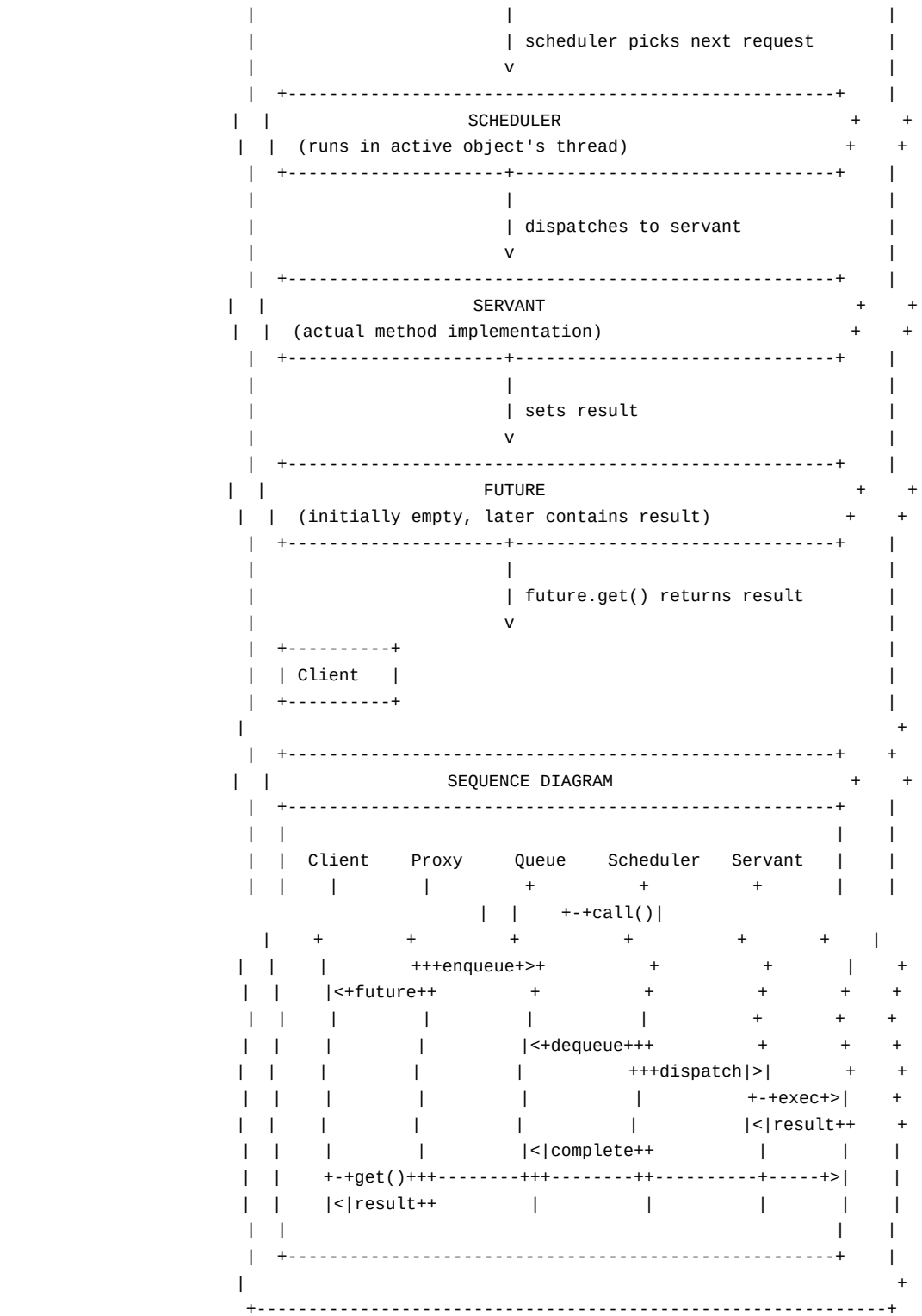
Detailed Explanation

The Active Object pattern decouples method invocation from method execution. Each object has its own thread of control and a queue of pending requests. Methods called on the active object return immediately (with a future), and execution happens asynchronously in the object's thread.

Understanding the Problem

In traditional object-oriented programming, when you call a method, the caller's thread executes the method. This has several limitations:
1. **Blocking:** The caller blocks until the method completes.
2. **Thread Safety:** You need synchronization if multiple threads call methods.
3. **Responsiveness:** Long-running methods freeze the caller.
4. **Concurrency Control:** Hard to manage concurrent access.
The Solution
The Active Object pattern creates objects with their own threads. Each active object has:
1. **Proxy:** The interface clients call (returns future immediately)
2. **Method Request:** Encapsulates a method call and its arguments
3. **Activation Queue:** Queue of pending method requests
4. **Scheduler:** Decides which request to execute next
5. **Servant:** The actual implementation of the methods
6. **Future:** Represents the pending result
Diagram: Active Object





Implementation Example

```
import java.util.concurrent.*;
import java.util.*;
// 1. The Future/Result
class ActiveFuture<T> {
private T result;
private boolean completed = false;
private final Object lock = new Object();

public T get() throws InterruptedException {
```

```
        synchronized(lock) {
            while (!completed) {
                lock.wait();
            }
            return result;
        }
    }

    public void set(T result) {
        synchronized(lock) {
            this.result = result;
            this.completed = true;
            lock.notifyAll();
        }
    }
}

// 2. Method Request
abstract class MethodRequest<T> {
    protected final ActiveFuture<T> future;

    public MethodRequest(ActiveFuture<T> future) {
        this.future = future;
    }

    public abstract void execute();
}

// 3. Concrete Method Request
class PrintRequest extends MethodRequest<String> {
    private final String message;
    private final Servant servant;

    public PrintRequest(String message, Servant servant, ActiveFuture<String> future) {
        super(future);
        this.message = message;
        this.servant = servant;
    }

    @Override
    public void execute() {
        String result = servant.print(message);
        future.set(result);
    }
}

class CalculateRequest extends MethodRequest<Integer> {
    private final int a;
    private final int b;
    private final Servant servant;

    public CalculateRequest(int a, int b, Servant servant, ActiveFuture<Integer> future) {
        super(future);
        this.a = a;
        this.b = b;
        this.servant = servant;
    }

    @Override
    public void execute() {
        int result = servant.calculate(a, b);
        future.set(result);
    }
}
// 4. Servant (actual implementation)
```

```
class Servant {
    public String print(String message) {
        try {
            Thread.sleep(1000); // Simulate work
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        return "Printed: " + message;
    }

    public int calculate(int a, int b) {
        try {
            Thread.sleep(500); // Simulate work
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        return a + b;
    }
}

// 5. Scheduler and Activation Queue
class Scheduler implements Runnable {
    private final BlockingQueue<MethodRequest<?>> queue;
    private final Servant servant;
    private volatile boolean running = true;

    public Scheduler(Servant servant) {
        this.queue = new LinkedBlockingQueue<>();
        this.servant = servant;
    }

    public <T> ActiveFuture<T> enqueue(MethodRequest<T> request) {
        queue.offer(request);
        return request.future;
    }

    @Override
    public void run() {
        while (running) {
            try {
                MethodRequest<?> request = queue.take();
                request.execute();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }
    }

    public void shutdown() {
        running = false;
    }
}

// 6. Proxy (Active Object)
class ActiveObject {
    private final Scheduler scheduler;
    private final Thread schedulerThread;

    public ActiveObject() {
        Servant servant = new Servant();
        this.scheduler = new Scheduler(servant);
        this.schedulerThread = new Thread(scheduler);
        this.schedulerThread.start();
    }
```

```
    public ActiveFuture<String> print(String message) {
        ActiveFuture<String> future = new ActiveFuture<>();
        PrintRequest request = new PrintRequest(message,
            new Servant(), future); // In real impl, share servant
        return scheduler.enqueue(request);
    }

    public ActiveFuture<Integer> calculate(int a, int b) {
        ActiveFuture<Integer> future = new ActiveFuture<>();
        CalculateRequest request = new CalculateRequest(a, b,
            new Servant(), future);
        return scheduler.enqueue(request);
    }

    public void shutdown() {
        scheduler.shutdown();
        schedulerThread.interrupt();
    }
}

// 7. Client Usage
public class ActiveObjectDemo {
    public static void main(String[] args) throws Exception {
        ActiveObject activeObject = new ActiveObject();

        // Non-blocking calls
        ActiveFuture<String> future1 = activeObject.print("Hello");
        ActiveFuture<Integer> future2 = activeObject.calculate(10, 20);

        // Do other work while tasks execute
        System.out.println("Doing other work...");

        // Get results (blocks if not ready)
        String result1 = future1.get();
        int result2 = future2.get();

        System.out.println("Results: " + result1 + ", " + result2);

        activeObject.shutdown();
    }
}
```

When to Use Active Object Pattern

Suitable Situations:

1. When you need asynchronous method execution.

2. When you want to simplify concurrency by encapsulating it in objects.

3. When you need to serialize access to an object (methods execute in single thread).

4. When you want to improve responsiveness (caller doesn't block).

5. When you need to schedule method execution (priority, delay).

Real-World Use Cases:

- **GUI Applications:** Background tasks without freezing UI
- **Game Engines:** AI updates, physics calculations
- **Network Servers:** Handle requests asynchronously
- **Robotics:** Control loops with timing constraints
- **Actors:** Actor model implementations

Advantages of Active Object Pattern

1. **Simplified Concurrency:** Each active object has its own thread.

2. **No Explicit Locking:** Methods execute sequentially in one thread.

3. **Responsiveness:** Caller doesn't block.

4. **Scheduling Flexibility:** Can prioritize requests.

5. **Encapsulation:** Concurrency control inside the object.

Disadvantages of Active Object Pattern

1. **Overhead:** Creating method request objects for each call.

2. **Complexity:** More complex than direct method calls.

3. **Debugging Difficulty:** Async flow harder to trace.

4. **Resource Usage:** Each active object needs a thread.

5. **Potential for Deadlock:** If active objects call each other.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
Active Object	Async method execution	Object with its own thread
Future/Promise	Represent async result	Often used with active object

Pattern	Purpose	Difference
Actor Model	Message-passing concurrency	Similar concept
Half-Sync/Half-Async	Separate sync/async layers	Different architecture

Active Object vs Actor Model

- **Active Object:** Object-oriented, methods become messages
- **Actor Model:** Pure message-passing, no shared state

Active Object vs Thread Pool

- **Active Object:** Each object has dedicated thread
- **Thread Pool:** Threads shared among many tasks

Common Pitfalls

1. **Too Many Threads:** Each active object needs a thread.

2. **Circular Dependencies:** Can cause deadlock.

3. **Queue Overflow:** Requests may be dropped.

4. **Shutdown Handling:** Need to drain queue properly.

5. **Future Timeouts:** Callers may wait forever.

Best Practices

1. **Limit Number of Active Objects:** Each uses a thread.

2. **Use Bounded Queues:** Prevent memory exhaustion.

3. **Implement Timeouts:** Don't wait forever for results.

4. **Handle InterruptedException:** Proper cleanup.

5. **Consider Thread Pool for Many Objects:** Hybrid approach.

6. **Monitor Queue Size:** Detect backpressure.

[Page 18]

Chapter 16: Future / Promise Pattern

Detailed Explanation

The Future/Promise pattern represents the result of an asynchronous computation. A Future is a read-only placeholder for a result that will be available later. A Promise is the writable counterpart that completes the Future.

Understanding the Problem

When you start an asynchronous operation, you need a way to:

1. Get the result when it's ready
2. Check if the operation is complete
3. Cancel the operation if needed
4. Handle errors that occurred during computation

Without a Future, you'd need callbacks, which can lead to "callback hell" with nested callbacks.

The Solution

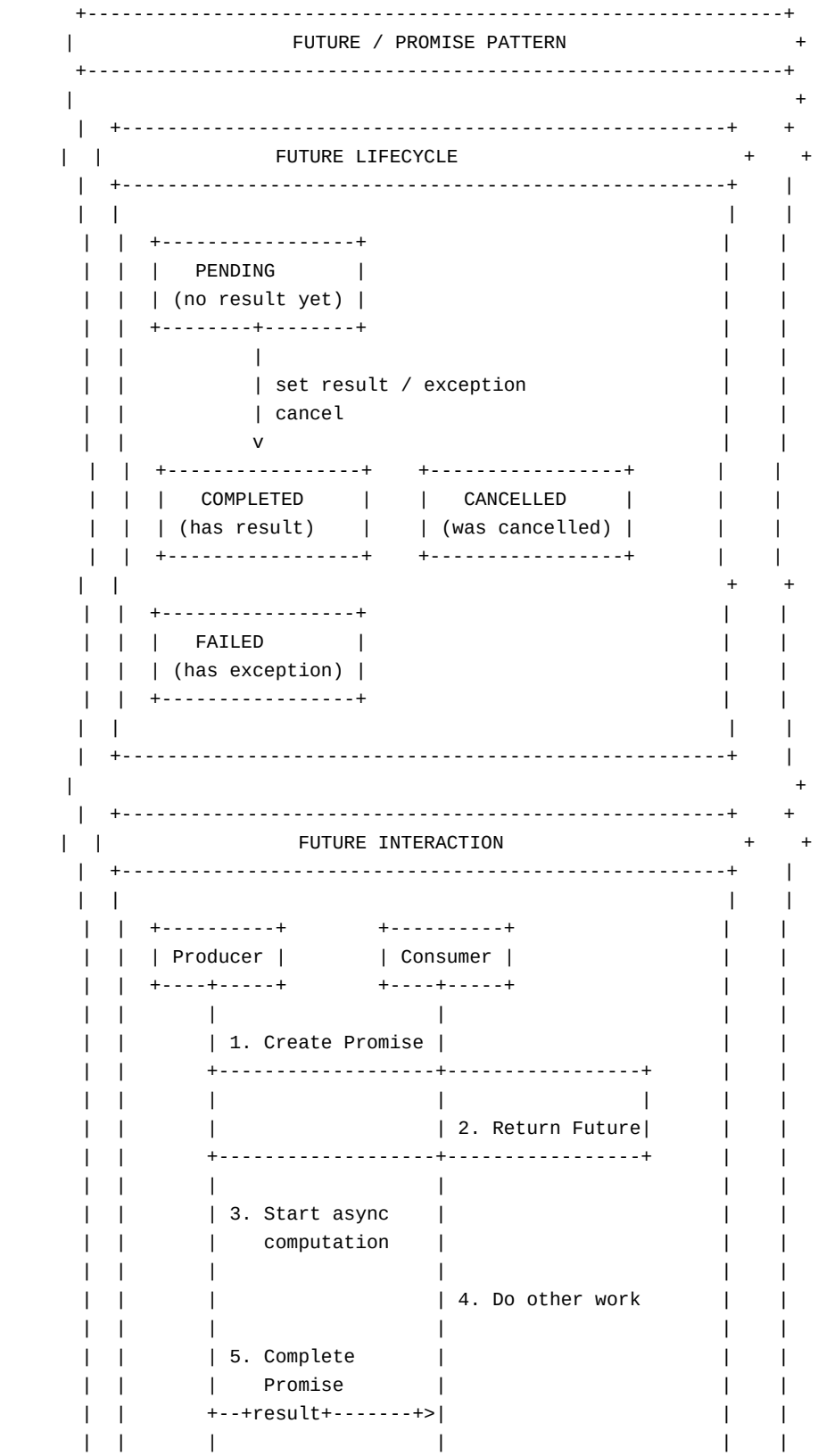
The Future pattern provides an object that acts as a proxy for a result not yet available. It offers methods to:

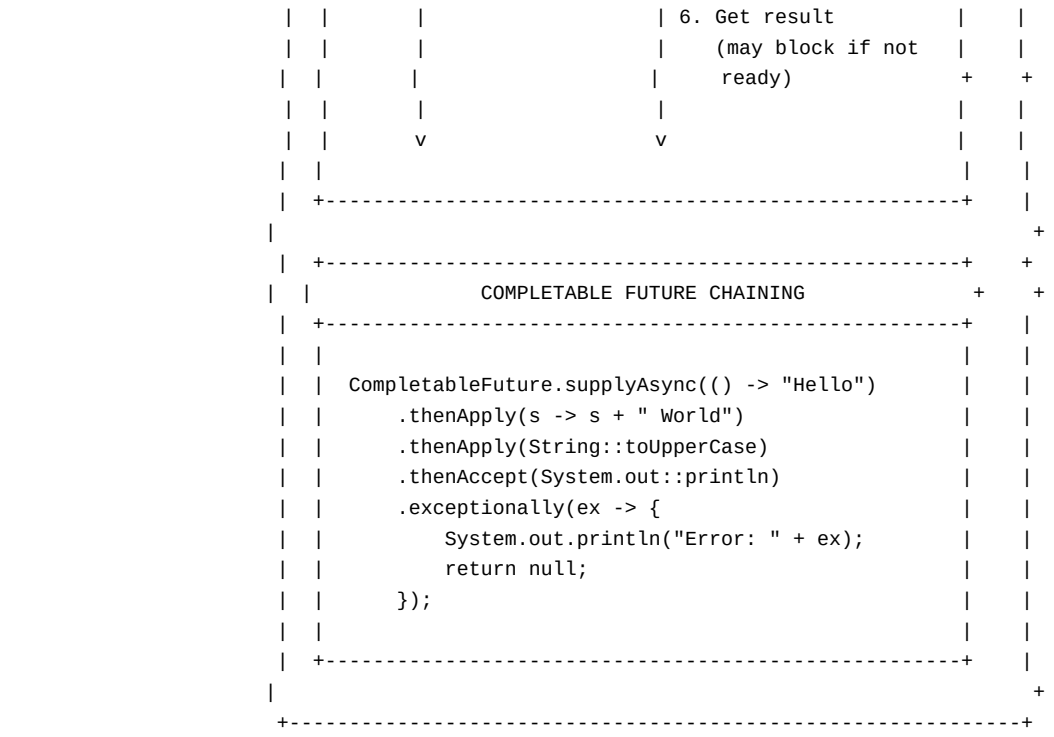
- Check if the computation is complete
- Wait for the result (blocking)
- Register callbacks for when result is ready
- Cancel the operation
- Handle exceptions

Key Concepts

- **Future:** Read-only view of pending result
- **Promise:** Completable future (can set result)
- **Callback:** Function to execute when result is ready
- **CompletableFuture:** Can chain operations
- **Cancellation:** Ability to cancel pending operation

Diagram: Future/Promise





Implementation Example (Simple Future)

```
public class SimpleFuture<T> {
private T result;
private boolean completed = false;
private Exception error;
private final Object lock = new Object();

public T get() throws InterruptedException, ExecutionException {
synchronized(lock) {
while (!completed) {
lock.wait();
}
if (error != null) {
throw new ExecutionException(error);
}
return result;
}
}

public T get(long timeout, TimeUnit unit)
throws InterruptedException, ExecutionException, TimeoutException {
synchronized(lock) {
                        long deadline = System.currentTimeMillis() + unit.toMillis(timeout);
while (!completed) {
                                long waitTime = deadline - System.currentTimeMillis();
if (waitTime <= 0) {
throw new TimeoutException();
}
lock.wait(waitTime);
}
if (error != null) {
throw new ExecutionException(error);
}
return result;
}
}
public boolean isDone() {
```

```
synchronized(lock) {
return completed;
}
}

// Promise methods (not in Future interface)
protected void complete(T result) {
synchronized(lock) {
this.result = result;
this.completed = true;
lock.notifyAll();
}
}

protected void completeExceptionally(Exception error) {
synchronized(lock) {
this.error = error;
this.completed = true;
lock.notifyAll();
}
}
}
```

Java's CompletableFuture Example

```
import java.util.concurrent.*;
public class FutureExample {
public static void main(String[] args) throws Exception {
// 1. Simple future with explicit completion
CompletableFuture<String> future = new CompletableFuture<>();

// In another thread
                                            new Thread(() -> {
try {
Thread.sleep(1000);
future.complete("Hello from async task");
} catch (InterruptedException e) {
future.completeExceptionally(e);
}
}).start();

// Do other work
System.out.println("Doing other work...");

// Get result (blocks)
String result = future.get();
                                    System.out.println("Result: " + result);

// 2. Chained completable futures
                                      CompletableFuture.supplyAsync(() -> {
// First async task
return "Hello";
                                              }).thenApply(s -> {
// Transform result (in same thread or pool)
                                                return s + " World";
                               }).thenApply(String::toUpperCase).thenAccept(s -> {
// Consume final result
                                      System.out.println("Final result: " + s);
                                            }).exceptionally(ex -> {
// Handle any exception in chain
                                         System.err.println("Error: " + ex);
return null;
```

});

// 3. Combine multiple futures
```
CompletableFuture<String> f1 = CompletableFuture.supplyAsync(() -> "Hello");
CompletableFuture<String> f2 = CompletableFuture.supplyAsync(() -> "World");

CompletableFuture<String> combined = f1.thenCombine(f2, (s1, s2) -> s1 + " " + s2);
System.out.println("Combined: " + combined.get());
```

// 4. Wait for any of multiple futures
```
CompletableFuture<Object> any = CompletableFuture.anyOf(f1, f2);
System.out.println("First completed: " + any.get());
```

// 5. Wait for all futures
```
CompletableFuture<Void> all = CompletableFuture.allOf(f1, f2);
all.get(); // wait for both
System.out.println("Both completed");
```

// 6. Async with timeout
```
CompletableFuture<String> withTimeout = CompletableFuture
.supplyAsync(() -> {
try { Thread.sleep(2000); } catch (Exception e) {}
return "Done";
})
.orTimeout(1, TimeUnit.SECONDS)
.exceptionally(ex -> "Timeout occurred");

System.out.println("With timeout: " + withTimeout.get());
}
}
```

When to Use Future/Promise Pattern

Suitable Situations:

1. When performing asynchronous operations.
2. When you need to avoid callback hell.
3. When you want to compose multiple async operations.
4. When you need to handle timeouts and cancellations.
5. When you want to parallelize independent tasks.

Real-World Use Cases:

- **Web Clients:** Async HTTP requests
- **Database Queries:** Non-blocking database access
- **File I/O:** Async file operations

- **Microservices:** Parallel service calls
- **UI Applications:** Background tasks without freezing

Advantages of Future/Promise Pattern

1. **Abstraction:** Clean representation of async result.

2. **Composability:** Can chain and combine futures.

3. **Non-blocking:** Caller can do other work.

4. **Error Handling:** Unified exception handling.

5. **Cancellation:** Can cancel pending operations.

6. **Timeouts:** Built-in timeout support.

Disadvantages of Future/Promise Pattern

1. **Complexity:** More complex than synchronous code.

2. **Memory Leaks:** If futures are never completed.

3. **Debugging Difficulty:** Async stack traces are harder.

4. **Blocking Calls:** get() blocks if not careful.

5. **Thread Usage:** May consume threads.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
---	---	---
Future	Async result placeholder	Read-only
Promise	Completable future	Can set result
Callback	Notification on completion	Inversion of control
Reactive Streams	Async stream of data	Multiple values

Future vs Callback

- **Future:** You actively ask for result
- **Callback:** System calls you when ready

Future vs Reactive Streams

- **Future:** Single result
- **Reactive Streams:** Stream of results (Observable/Flow)

Common Pitfalls

1. **Blocking on Future:** Using get() defeats async purpose.

2. **Not Handling Exceptions:** Uncaught exceptions are lost.

3. **Memory Leaks:** Futures never completed.

4. **Thread Starvation:** Too many async tasks.

5. **Callback Hell:** Nested thenApply chains.

Best Practices

1. **Prefer Async Methods:** Avoid get() when possible.

2. **Always Handle Exceptions:** Use exceptionally() or handle().

3. **Set Timeouts:** Prevent waiting forever.

4. **Use ExecutorService:** Control thread usage.

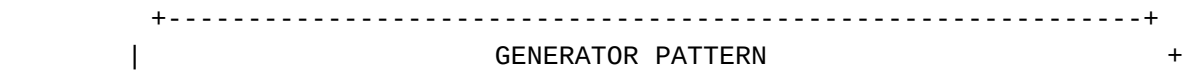
5. **Compose, Don't Nest:** Use thenCompose for async operations.

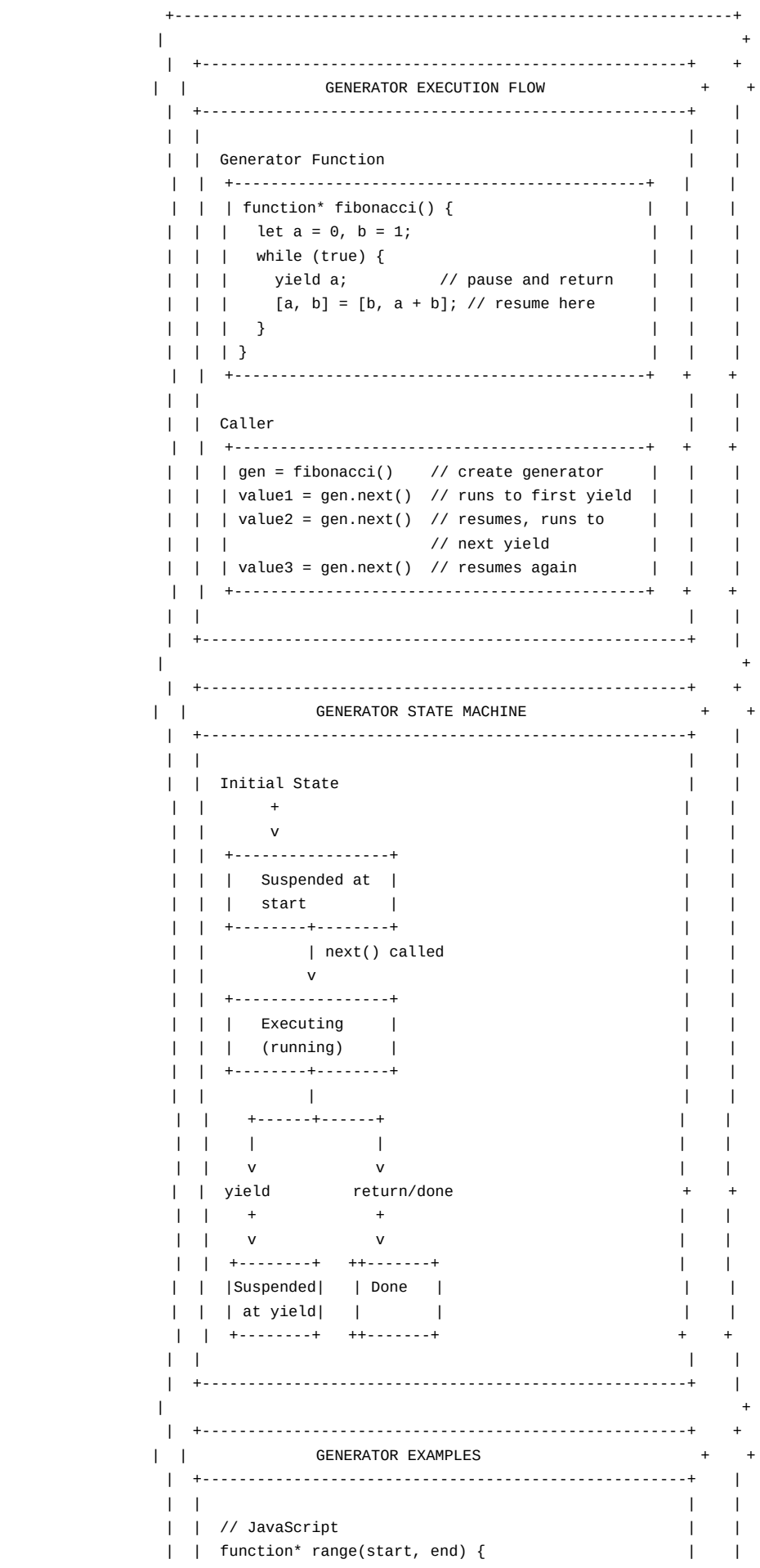
6. **Consider CompletableFuture:** Rich API for composition.

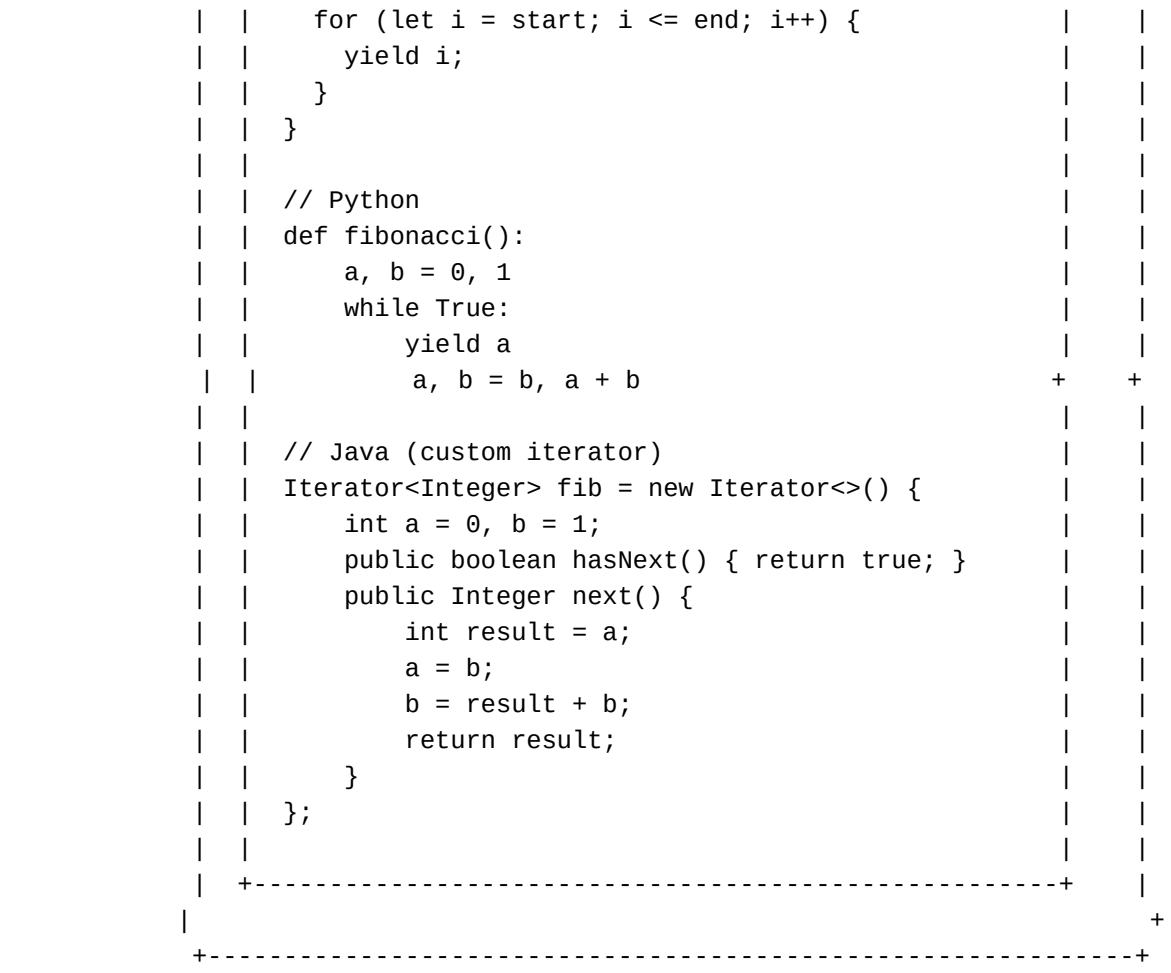
[Page 19]

Chapter 17: Generator Pattern

Detailed Explanation
The Generator pattern produces a sequence of values lazily, one at a time, without computing all values at once. Generators maintain their own state between calls and can yield multiple values over time.
Understanding the Problem
Sometimes you need to generate a sequence of values, but:
1. The sequence is too large to store in memory (infinite or huge)
2. You want to generate values lazily (on-demand)
3. The generation algorithm is stateful and needs to remember previous state
Examples:
- Fibonacci sequence (infinite)
- Reading a file line by line
- Generating prime numbers
- Paginated API results
The Solution
The Generator pattern provides a function that returns an iterator-like object. Each time you ask for the next value, the generator resumes execution from where it left off, maintaining its internal state.
Key Concepts
- **Generator Function:** Function that can pause and resume
- **Yield:** Statement that returns a value and pauses execution
- **Iterator:** Object that provides next() method
- **State:** Generator's internal state preserved between calls
- **Lazy Evaluation:** Values computed only when needed
Diagram: Generator Pattern




GENERATOR EXECUTION FLOW
Generator Function
function* fibonacci() {
let a = 0, b = 1;
while (true) {
yield a; // pause and return
[a, b] = [b, a + b]; // resume here
}
}
Caller
gen = fibonacci() // create generator
value1 = gen.next() // runs to first yield
value2 = gen.next() // resumes, runs to
// next yield
value3 = gen.next() // resumes again
GENERATOR STATE MACHINE
Initial State
Suspended at start
next() called
Executing (running)
yield
return/done
Suspended at yield
Done
GENERATOR EXAMPLES
// JavaScript
function* range(start, end) {



Implementation Examples

JavaScript Generator:

```
// Generator function
function* fibonacci() {
let a = 0, b = 1;
while (true) {
yield a;
                                    [a, b] = [b, a + b];
}
}
// Usage
const fib = fibonacci();
console.log(fib.next().value); // 0
console.log(fib.next().value); // 1
console.log(fib.next().value); // 1
console.log(fib.next().value); // 2
console.log(fib.next().value); // 3
// Take first 10 fibonacci numbers
const first10 = Array.from({ length: 10 }, () => fib.next().value);
```

Python Generator:

```
def fibonacci():
a, b = 0, 1
while True:
yield a
                                      a, b = b, a + b
# Usage
```

```
fib = fibonacci()
for i in range(10):
    print(next(fib))

# Generator expression
squares = (x*x for x in range(10))
```

Java Custom Iterator (simulating generator):

```
public class RangeIterator implements Iterator<Integer> {
    private final int end;
    private int current;

    public RangeIterator(int start, int end) {
        this.current = start;
        this.end = end;
    }

    @Override
    public boolean hasNext() {
        return current <= end;
    }

    @Override
    public Integer next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        return current++; // return and increment state
    }
}

// Usage
Iterator<Integer> range = new RangeIterator(1, 10);
while (range.hasNext()) {
    System.out.println(range.next());
}
```

Java with Continuations (simulated):

```
// Not built-in, but can be simulated with state machine
class FibonacciGenerator {
    private int a = 0;
    private int b = 1;
    private int state = 0;

    public Integer next() {
        switch (state) {
            case 0:
                state = 1;
                return 0;
            case 1:
                state = 2;
                return 1;
            default:
                int result = a + b;
                a = b;
                b = result;
                return result;
        }
    }
```

```
        }
```

When to Use Generator Pattern

Suitable Situations:

1. When generating infinite or large sequences.

2. When you want lazy evaluation (compute only when needed).

3. When the generation algorithm is stateful.

4. When processing streams of data.

5. When you want to simplify stateful iteration logic.

Real-World Use Cases:

- **Data Streaming:** Reading large files line by line
- **Infinite Sequences:** Fibonacci, prime numbers
- **Paginated APIs:** Fetching next page only when needed
- **Tree Traversal:** In-order, pre-order traversal
- **Event Streams:** Processing events as they arrive

Advantages of Generator Pattern

1. **Memory Efficient:** No need to store entire sequence.

2. **Lazy Evaluation:** Compute only what you need.

3. **Infinite Sequences:** Can represent infinite data.

4. **Clean State Management:** State preserved automatically.

5. **Simplifies Code:** Complex iteration logic becomes simple.

Disadvantages of Generator Pattern

1. **Language Support:** Not all languages have generators.

2. **One-Time Use:** Generators are typically exhaustible.

3. **Debugging Difficulty:** Execution pauses and resumes.

4. **Overhead:** Some overhead for state preservation.

5. **Not Thread-Safe:** Generators usually single-threaded.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Generator	Lazy sequence generation	Yields multiple values
Iterator	Sequential access	Similar, but manual state
Observer	Push notifications	Push vs pull
Future	Single async result	Single value

Generator vs Iterator

- **Generator:** Automatically maintains state, uses yield
- **Iterator:** Manual state management, next()/hasNext()

Generator vs Stream (Java 8+)

- **Generator:** Pull-based, lazy
- **Stream:** Can be lazy or eager, functional operations

Common Pitfalls

1. **Exhausting Generator:** Can't reuse after completion.

2. **Infinite Loops:** Forgetting to break infinite generators.

3. **State Leakage:** Shared state between generators.

4. **Resource Leaks:** Not closing generator resources.

5. **Thread Safety:** Generators not thread-safe by default.

Best Practices

1. **Document Termination:** Clearly indicate if generator is infinite.

2. **Use try-with-resources:** For generators that need cleanup.

3. **Prefer Lazy Evaluation:** Only generate what you need.

4. **Compose Generators:** Combine multiple generators.

5. **Consider Memory:** Generators don't eliminate memory usage entirely.

6. **Test Edge Cases:** Empty sequences, end conditions.

[Page 20]

Chapter 18: Fork-Join Pattern

Detailed Explanation

The Fork-Join pattern is a divide-and-conquer approach to parallel processing. A large task is recursively split (forked) into smaller subtasks until they're small enough to solve directly. Results are then combined (joined) to produce the final result.

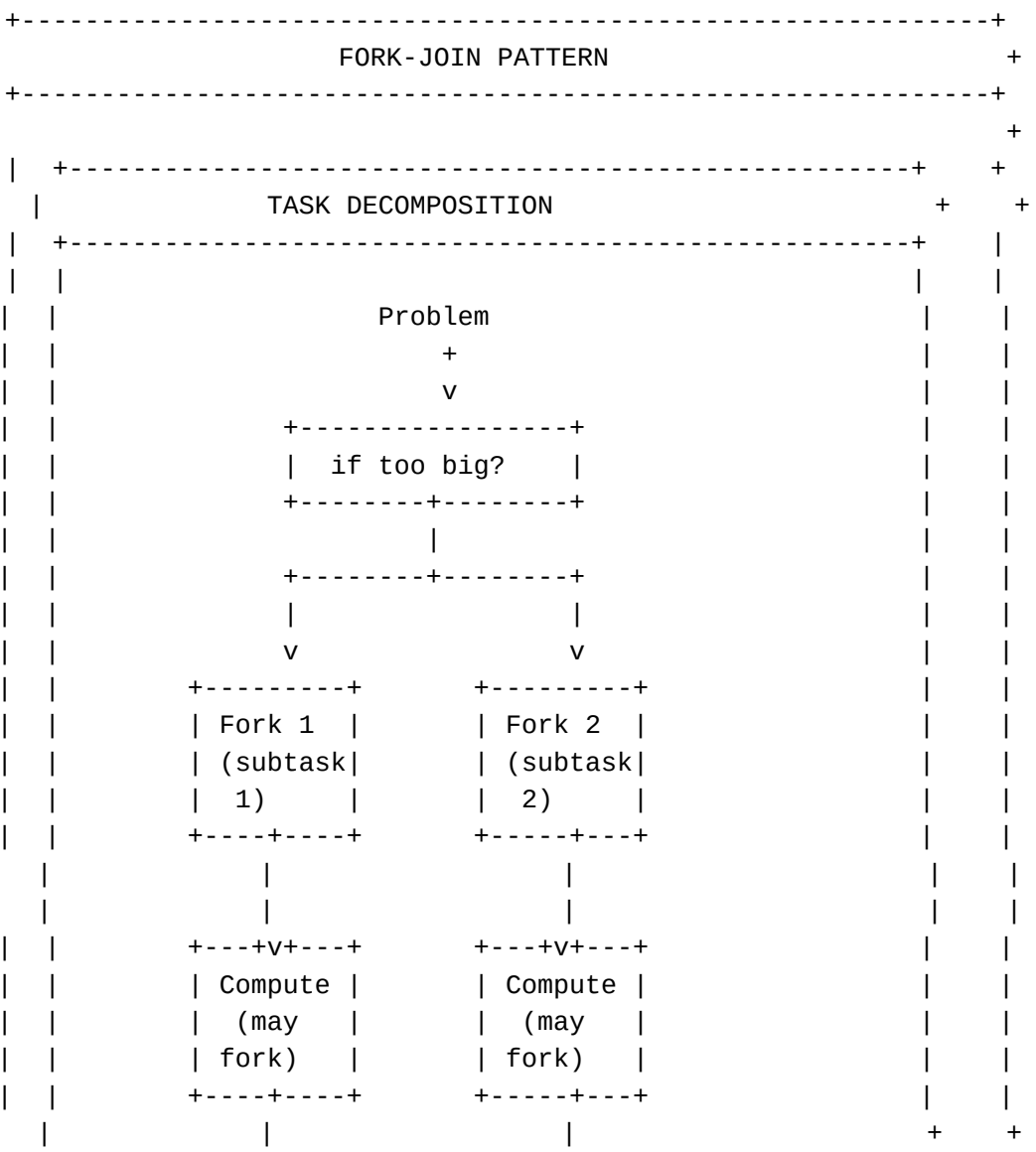
Understanding the Problem

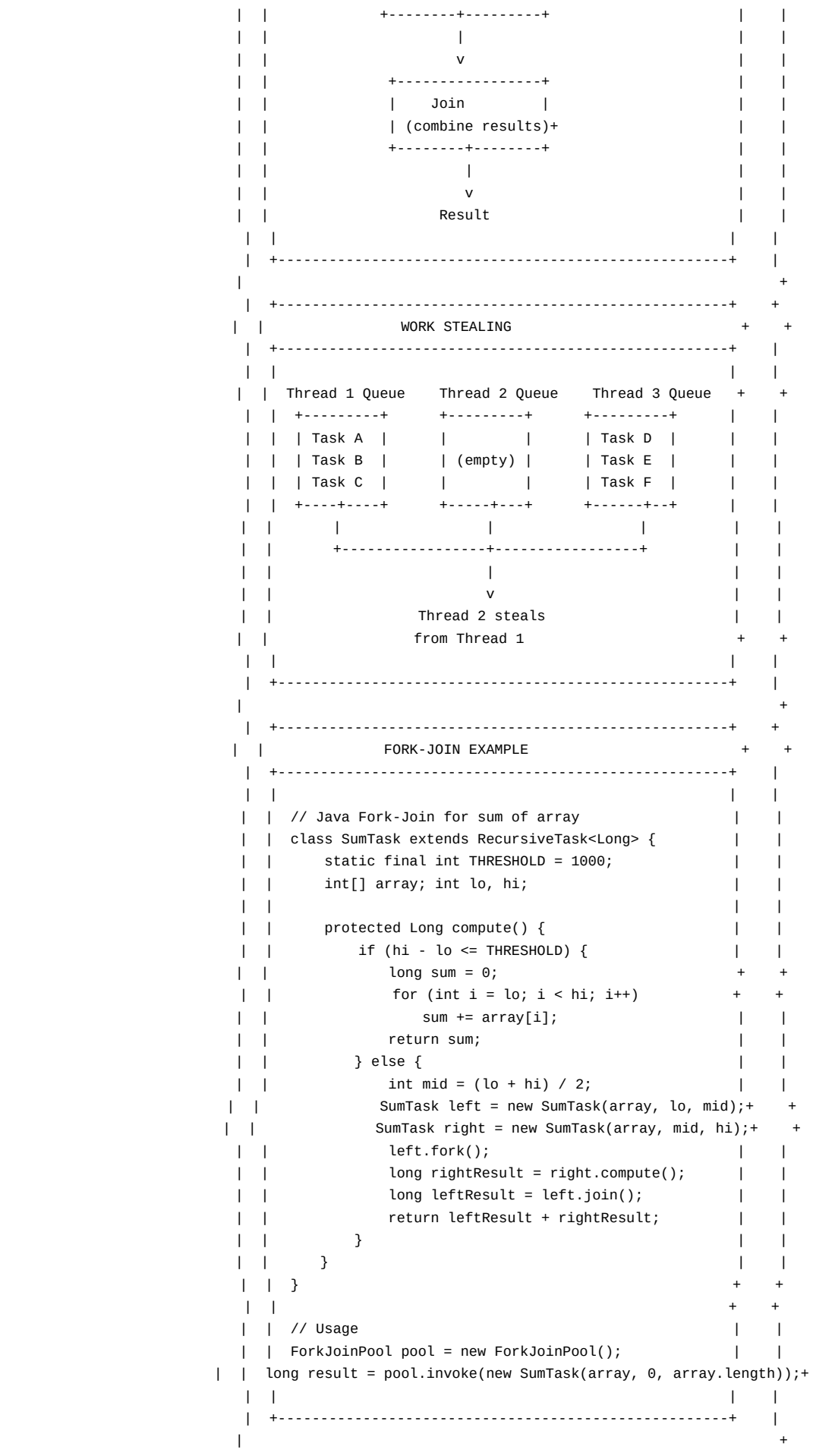
Many problems can be solved by dividing them into smaller, independent subproblems:

- Sorting large arrays (merge sort, quicksort)
- Searching in large datasets
- Matrix multiplication
- Image processing
- Monte Carlo simulations

Doing this sequentially doesn't utilize multiple cores. The challenge is to parallelize the divide-and-conquer algorithm efficiently.

The Solution
The Fork-Join pattern provides:
1. **Fork:** Split a task into smaller subtasks that can run in parallel
2. **Join:** Wait for subtasks to complete and combine their results
3. **Work Stealing:** Idle threads steal work from busy threads' queues for better load balancing
Key Concepts
- **ForkJoinPool:** Special thread pool for fork-join tasks
- **ForkJoinTask:** Base class for tasks (RecursiveTask for results, RecursiveAction for no result)
- **Work Stealing:** Threads with empty queues steal from others
- **Threshold:** Size at which task is solved directly (not forked)
Diagram: Fork-Join Pattern




Join
(combine results)
Result
WORK STEALING
Thread 1 Queue
Thread 2 Queue
Thread 3 Queue
Task A
Task B
Task C
(empty)
Task D
Task E
Task F
Thread 2 steals
from Thread 1
FORK-JOIN EXAMPLE
// Java Fork-Join for sum of array
class SumTask extends RecursiveTask<Long> {
static final int THRESHOLD = 1000;
int[] array; int lo, hi;
protected Long compute() {
if (hi - lo <= THRESHOLD) {
long sum = 0;
for (int i = lo; i < hi; i++)
sum += array[i];
return sum;
} else {
int mid = (lo + hi) / 2;
SumTask left = new SumTask(array, lo, mid);
SumTask right = new SumTask(array, mid, hi);
left.fork();
long rightResult = right.compute();
long leftResult = left.join();
return leftResult + rightResult;
}
}
}
// Usage
ForkJoinPool pool = new ForkJoinPool();
long result = pool.invoke(new SumTask(array, 0, array.length));

```
            +-------------------------------------------------------------+
```

Implementation Example

```
import java.util.concurrent.*;
// RecursiveTask for tasks that return a result
class SumTask extends RecursiveTask<Long> {
                          private static final int THRESHOLD = 10_000;
private final int[] array;
private final int start;
private final int end;

public SumTask(int[] array, int start, int end) {
this.array = array;
this.start = start;
this.end = end;
}

@Override
protected Long compute() {
                                    int length = end - start;

// If task is small enough, compute directly
if (length <= THRESHOLD) {
long sum = 0;
                                  for (int i = start; i < end; i++) {
                                             sum += array[i];
}
return sum;
}

// Otherwise, split into two subtasks
                                   int mid = start + length / 2;
SumTask leftTask = new SumTask(array, start, mid);
SumTask rightTask = new SumTask(array, mid, end);

// Fork left task (async)
leftTask.fork();

// Compute right task directly
long rightResult = rightTask.compute();

// Wait for left task result
long leftResult = leftTask.join();

                                 return leftResult + rightResult;
}
}
// RecursiveAction for tasks that don't return a result
class QuickSortTask extends RecursiveAction {
private static final int THRESHOLD = 1000;
private final int[] array;
private final int left;
private final int right;

public QuickSortTask(int[] array, int left, int right) {
this.array = array;
this.left = left;
this.right = right;
}
@Override
```

```
    protected void compute() {
        if (left < right) {
            if (right - left < THRESHOLD) {
                // Sequential sort for small arrays
                Arrays.sort(array, left, right + 1);
            } else {
                int pivot = partition(array, left, right);

                QuickSortTask leftTask = new QuickSortTask(array, left, pivot - 1);
                QuickSortTask rightTask = new QuickSortTask(array, pivot + 1, right);

                // Fork both in parallel
                invokeAll(leftTask, rightTask);
            }
        }
    }

    private int partition(int[] array, int left, int right) {
        // Standard quicksort partition
        int pivot = array[right];
        int i = left - 1;
        for (int j = left; j < right; j++) {
            if (array[j] <= pivot) {
                i++;
                swap(array, i, j);
            }
        }
        swap(array, i + 1, right);
        return i + 1;
    }

    private void swap(int[] array, int i, int j) {
        int temp = array[i];
        array[i] = array[j];
        array[j] = temp;
    }
}

// Usage
public class ForkJoinExample {
    public static void main(String[] args) {
        // Create large array
        int[] array = new int[1_000_000];
        Random random = new Random();
        for (int i = 0; i < array.length; i++) {
            array[i] = random.nextInt(100);
        }

        // Create fork-join pool (uses number of processors by default)
        ForkJoinPool pool = new ForkJoinPool();

        // Compute sum
        SumTask sumTask = new SumTask(array, 0, array.length);
        long sum = pool.invoke(sumTask);
        System.out.println("Sum: " + sum);

        // Sort array
        QuickSortTask sortTask = new QuickSortTask(array, 0, array.length - 1);
        pool.invoke(sortTask);

        // Check first few elements
        System.out.println("First 10 sorted: " +
            Arrays.toString(Arrays.copyOf(array, 10)));

        // Pool statistics
```

```
            System.out.println("Pool size: " + pool.getPoolSize());
            System.out.println("Steal count: " + pool.getStealCount());

            pool.shutdown();
        }
    }
```

ForkJoinPool Configuration

```
    // Create with custom parallelism
    ForkJoinPool customPool = new ForkJoinPool(
        4, // parallelism level
        ForkJoinPool.defaultForkJoinWorkerThreadFactory,
        null, // exception handler
        true  // async mode
    );

    // Common pool (shared across application)
    ForkJoinPool commonPool = ForkJoinPool.commonPool();
    System.out.println("Common pool parallelism: " +
        ForkJoinPool.getCommonPoolParallelism());
```

When to Use Fork-Join Pattern

Suitable Situations:

1. When problems can be solved with divide-and-conquer.

2. When subtasks are independent (no synchronization needed).

3. When you have multiple cores available.

4. When tasks are CPU-bound (not I/O-bound).

5. When task sizes are irregular (work stealing helps).

Real-World Use Cases:

- **Sorting:** Parallel merge sort, quicksort
- **Searching:** Parallel search in large datasets
- **Matrix Operations:** Multiplication, inversion
- **Image Processing:** Parallel filtering, transformations
- **Data Analysis:** Aggregations, statistics

Advantages of Fork-Join Pattern

1. **Automatic Parallelism:** Recursive decomposition maps naturally to parallelism.

2. **Work Stealing:** Excellent load balancing.

3. **Scalability:** Scales with number of cores.

4. **Efficient:** Minimizes overhead of thread creation.

5. **Familiar Model:** Divide-and-conquer is intuitive.

Disadvantages of Fork-Join Pattern

1. **Overhead:** Forking and joining has cost.

2. **Threshold Tuning:** Need to choose right threshold.

3. **Not for I/O:** Threads block on I/O, reducing benefits.

4. **Complexity:** More complex than sequential.

5. **Debugging Difficulty:** Parallel execution hard to debug.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
---	---	---
Fork-Join	Divide and conquer	Recursive decomposition
Thread Pool	General task execution	No built-in work stealing
MapReduce	Large-scale data	Map and reduce phases
Pipeline	Sequential stages	Linear flow, not tree

Fork-Join vs Thread Pool

- **Fork-Join:** Work stealing, optimized for fork-join tasks
- **Thread Pool:** General purpose, no work stealing

Fork-Join vs MapReduce

- **Fork-Join:** In-memory, fine-grained tasks

- **MapReduce:** Distributed, large-scale data processing

Common Pitfalls

1. **Threshold Too Small:** Too much forking overhead.

2. **Threshold Too Large:** Not enough parallelism.

3. **Blocking Operations:** Threads block, reducing work stealing benefits.

4. **Shared Mutable State:** Requires synchronization.

5. **Not Using join() Properly:** Forgetting to join.

Best Practices

1. **Choose Threshold Carefully:** Profile to find optimal size.

2. **Use invokeAll() for Multiple Forks:** More efficient than manual forking.

3. **Avoid Synchronization:** Tasks should be independent.

4. **Use RecursiveTask for Results:** RecursiveAction for side effects.

5. **Consider getSurplusQueuedTaskCount():** For adaptive thresholds.

6. **Monitor Steal Count:** High steal count indicates good load balancing.

7. **Don't Block in Tasks:** Use for CPU-bound work only.

[Page 21]

Chapter 19: Leader-Followers Pattern

Detailed Explanation

The Leader-Followers pattern provides an efficient thread pool model where one thread (the leader) waits for events, while other threads (followers) wait their turn to become leader. When an event arrives, the leader handles it and promotes a follower to become the new leader.

Understanding the Problem

Traditional thread pool approaches for event handling have drawbacks:

1. **Multiple threads waiting:** All threads block on the same event source, causing contention.
2. **Thread-per-connection:** Creates many threads, not scalable.
3. **Reactor pattern:** Dispatcher thread hands off to worker threads, causing context switches.

The Solution

The Leader-Followers pattern maintains a pool of threads where:

- **Leader:** One thread waits for events on the event source
- **Followers:** Other threads wait in a pool to become leader
- When an event arrives, the leader:

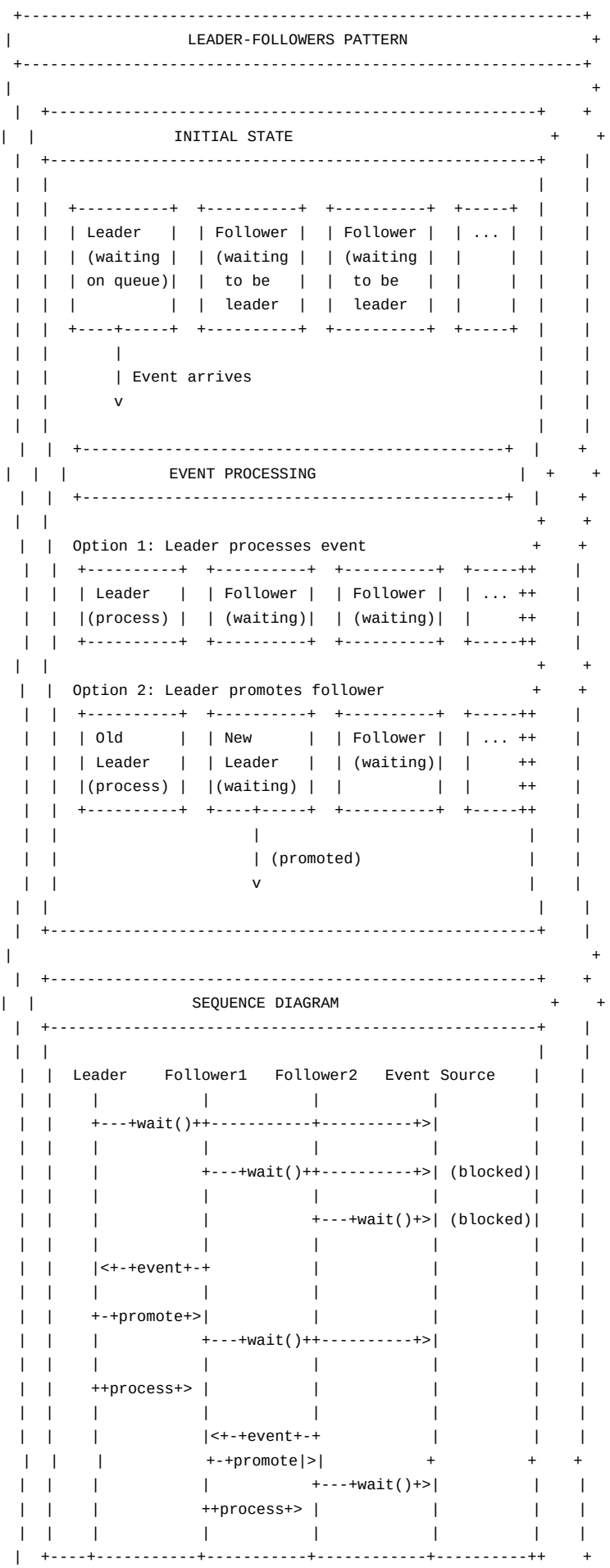
1. Processes the event, OR
2. Promotes a follower to become the new leader, then processes the event

- Promoted follower becomes the new leader and waits for events

Key Concepts

- **Event Source:** Source of events (network socket, queue)
- **Leader:** Thread currently waiting for events
- **Follower Pool:** Threads waiting to become leader
- **Promotion:** Making a follower the new leader
- **Demotion:** Leader becoming follower after processing

Diagram: Leader-Followers


LEADER-FOLLOWERS PATTERN
INITIAL STATE
Leader (waiting on queue)
Follower (waiting to be leader
Follower (waiting to be leader
...
Event arrives
EVENT PROCESSING
Option 1: Leader processes event
Leader (process)
Follower (waiting)
Follower (waiting)
...
Option 2: Leader promotes follower
Old Leader (process)
New Leader (waiting)
Follower (waiting)
...
(promoted)
SEQUENCE DIAGRAM
Leader
Follower1
Follower2
Event Source
wait()
(blocked)
(blocked)
event
promote
process

```
                |                                                                 +
                 +-----------------------------------------------------------+
```

Implementation Example

```
import java.util.concurrent.*;
import java.util.concurrent.locks.*;
public class LeaderFollowers {
private final int threadCount;
private final BlockingQueue<Runnable> eventQueue;
private final Lock lock = new ReentrantLock();
private final Condition leaderAvailable = lock.newCondition();
private Thread leader = null;
private final Set<Thread> followers = ConcurrentHashMap.newKeySet();
private volatile boolean running = true;

public LeaderFollowers(int threadCount, BlockingQueue<Runnable> eventQueue) {
this.threadCount = threadCount;
this.eventQueue = eventQueue;

// Create all follower threads
                                    for (int i = 0; i < threadCount; i++) {
FollowerThread follower = new FollowerThread();
followers.add(follower);
follower.start();
}

// Initially, make one thread the leader
promoteNewLeader();
}

private void promoteNewLeader() {
lock.lock();
try {
if (!followers.isEmpty()) {
// Take a follower and make it leader
Iterator<Thread> it = followers.iterator();
leader = it.next();
followers.remove(leader);
leaderAvailable.signal(); // Wake up the new leader
}
} finally {
lock.unlock();
}
}

private void demoteLeader() {
lock.lock();
try {
if (leader != null) {
followers.add(leader);
leader = null;
}
} finally {
lock.unlock();
}
}

private class FollowerThread extends Thread {
@Override
public void run() {
while (running) {
```

```
// Wait to become leader
lock.lock();
try {
while (leader != this && running) {
leaderAvailable.await();
}
if (!running) break;
} catch (InterruptedException e) {
break;
} finally {
lock.unlock();
}

                                        // Now we are the leader - wait for event
try {
Runnable event = eventQueue.take();

// Before processing, promote a new leader
promoteNewLeader();

// Process the event
event.run();

// After processing, we become a follower again
demoteLeader();

} catch (InterruptedException e) {
// Interrupted, exit
break;
}
}
}
}

public void shutdown() {
running = false;
lock.lock();
try {
leaderAvailable.signalAll();
} finally {
lock.unlock();
}
}

// Example usage
public static void main(String[] args) {
BlockingQueue<Runnable> queue = new LinkedBlockingQueue<>();
LeaderFollowers lf = new LeaderFollowers(4, queue);

// Submit some events
                                          for (int i = 0; i < 10; i++) {
final int taskId = i;
                                                  queue.add(() -> {
                                    System.out.println("Processing task " + taskId +
                                         " by " + Thread.currentThread().getName());
try {
Thread.sleep(100); // Simulate work
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
});
}

// Let it run for a while
try {
```

```
Thread.sleep(5000);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}

lf.shutdown();
}
}
```

Simpler Implementation Using Executor

```
import java.util.concurrent.*;
public class SimpleLeaderFollowers {
private final BlockingQueue<Runnable> eventQueue;
private final ExecutorService workerPool;
private volatile Thread leader;

public SimpleLeaderFollowers(int threadCount, BlockingQueue<Runnable> queue) {
this.eventQueue = queue;
                    this.workerPool = Executors.newFixedThreadPool(threadCount - 1);

// Main thread becomes leader
this.leader = Thread.currentThread();

// Start follower threads
                               for (int i = 0; i < threadCount - 1; i++) {
workerPool.submit(this::followerTask);
}
}

public void start() {
while (!Thread.interrupted()) {
try {
// Leader waits for event
Runnable event = eventQueue.take();

// Promote follower (submit to pool)
                                                  workerPool.submit(() -> {
// This follower will become temporary leader
// In real implementation, more sophisticated
event.run();
});

// Leader continues as leader (in this simple version)
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}

private void followerTask() {
// Followers just wait to be used
try {
                                           Thread.sleep(Long.MAX_VALUE);
} catch (InterruptedException e) {
// Expected when promoted
}
}

public void shutdown() {
workerPool.shutdownNow();
```

```
            }
        }
```

When to Use Leader-Followers Pattern

Suitable Situations:

1. When handling many events from a single source.

2. When you want to minimize context switching.

3. In high-performance network servers.

4. When event processing is relatively quick.

5. When you need predictable latency.

Real-World Use Cases:

- **Network Servers:** Handling client connections
- **Event Processing Systems:** Processing events from a queue
- **Real-time Systems:** Where latency is critical
- **Database Listeners:** Waiting for notifications
- **Message Queue Consumers:** Processing messages

Advantages of Leader-Followers Pattern

1. **Reduced Context Switching:** Only one thread waits for events.

2. **Cache Efficiency:** Leader's cache warm for event handling.

3. **No Handoff:** Event handled directly by leader (option 1).

4. **Scalability:** Can add more followers.

5. **Simplicity:** Clear leader/follower roles.

Disadvantages of Leader-Followers Pattern

1. **Complex Implementation:** Promotion/demotion logic.

2. **Single Point of Contention:** Leader promotion requires synchronization.

3. **Not Suitable for Slow Processing:** If event processing is slow, other events wait.

4. **Priority Inversion:** Low-priority event might be handled first.

5. **Debugging Difficulty:** Dynamic roles hard to trace.

Comparison with Other Patterns

Pattern	Purpose	Difference
Leader-Followers	Event handling with one waiter	Single leader waits
Reactor	Demultiplex events	Dispatcher thread
Thread Pool	General task execution	All threads wait
Half-Sync/Half-Async	Separate sync/async layers	Different architecture

Leader-Followers vs Reactor

- **Leader-Followers:** Leader thread waits, promotes followers
- **Reactor:** Dispatcher thread waits, hands off to workers

Leader-Followers vs Half-Sync/Half-Async

- **Leader-Followers:** All threads in same layer
- **Half-Sync/Half-Async:** Separates sync and async layers

Common Pitfalls

1. **Race Conditions:** In promotion logic.

2. **Thread Starvation:** If promotion is slow.

3. **Deadlock:** If leader waits for follower that's waiting for leader.

4. **Priority Issues:** No guarantee of event priority.

5. **Complex Shutdown:** Need to cleanly terminate all threads.

Best Practices

1. **Use Non-blocking Promotion:** Avoid holding locks during promotion.

2. **Consider Timeout:** Leader should timeout if no events.

3. **Monitor Queue Size:** Detect backpressure.

4. **Implement Graceful Shutdown:** Allow in-progress events to complete.

5. **Profile Contention:** Measure leader promotion overhead.

6. **Consider Hybrid Approaches:** Combine with thread pool for slow operations.

[Page 22]

PART FOUR: DATA FLOW PATTERNS

Chapter 20: Producer-Consumer Pattern

Detailed Explanation

The Producer-Consumer pattern separates the production of data from its consumption using a shared queue. Producers generate data and put it into the queue; consumers take data from the queue and process it.

Understanding the Problem

In many systems, data production and consumption happen at different rates:

- Producers may generate data faster than consumers can process
- Consumers may be idle when no data is available
- Multiple producers and consumers need to coordinate

Without proper coordination:
- Producers might overload consumers
- Consumers might waste CPU polling for data
- Shared data structures might be corrupted

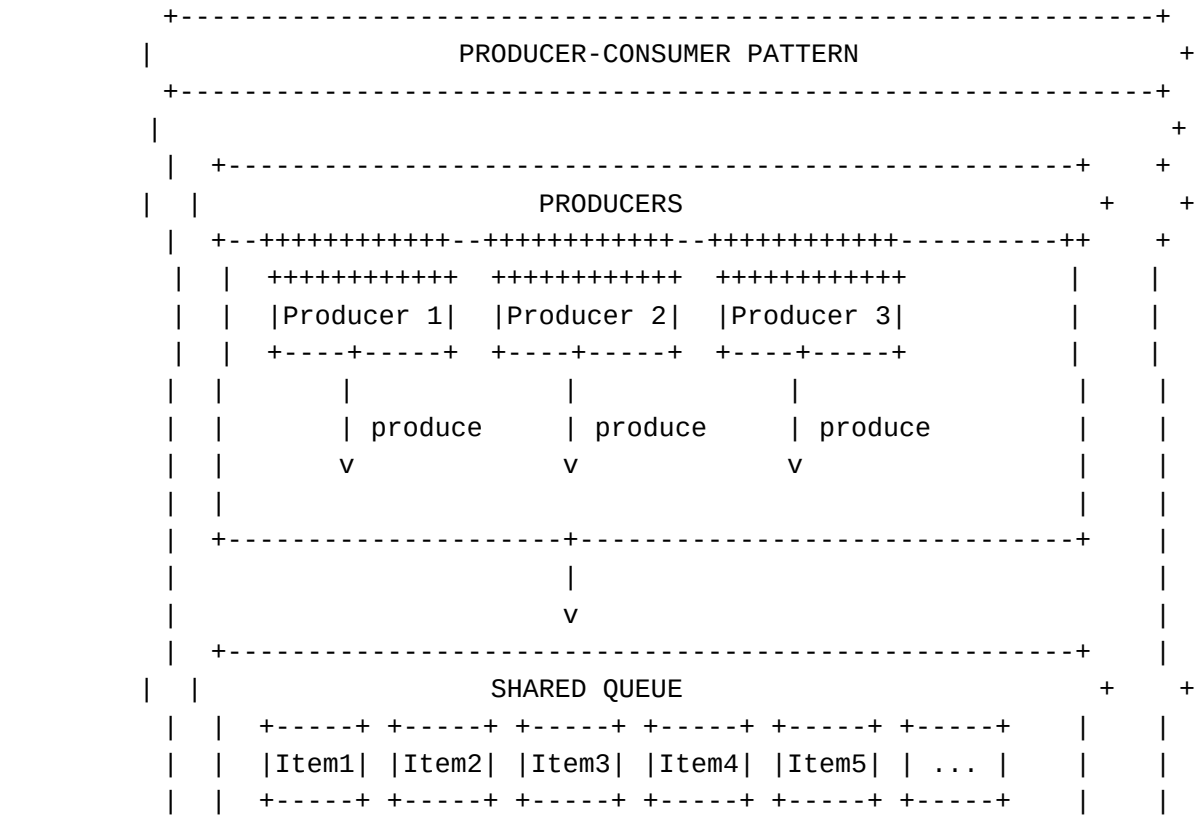
The Solution

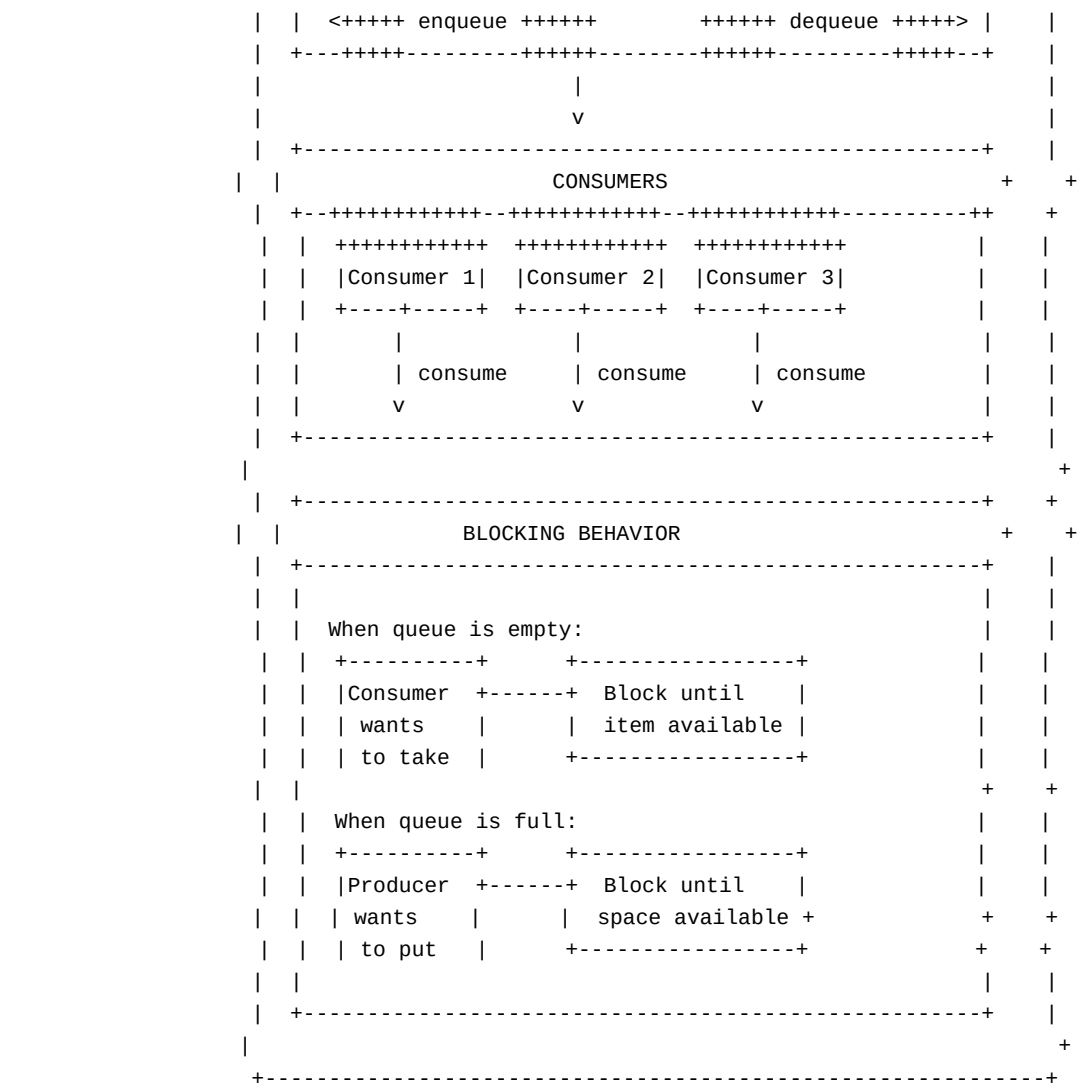
The Producer-Consumer pattern uses a thread-safe queue as a buffer between producers and consumers:
- **Producers:** Add items to the queue (block if queue is full)
- **Consumers:** Remove items from the queue (block if queue is empty)
- **Queue:** Thread-safe, bounded or unbounded

Key Concepts

- **Producer:** Thread(s) that create data
- **Consumer:** Thread(s) that process data
- **Queue:** Buffer between producers and consumers
- **Bounded Queue:** Limits number of items (provides backpressure)
- **Blocking:** Threads wait when queue is full/empty

Diagram: Producer-Consumer





Implementation Example

```
import java.util.concurrent.*;
import java.util.*;
// 1. Simple implementation with wait/notify
class SimpleBoundedBuffer<T> {
private final T[] buffer;
private int count = 0;
private int putIndex = 0;
private int takeIndex = 0;
private final Object lock = new Object();

@SuppressWarnings("unchecked")
public SimpleBoundedBuffer(int capacity) {
buffer = (T[]) new Object[capacity];
}

public void put(T item) throws InterruptedException {
synchronized(lock) {
while (count == buffer.length) {
lock.wait(); // buffer full, wait
}

buffer[putIndex] = item;
                                    putIndex = (putIndex + 1) % buffer.length;
                                                count++;
lock.notifyAll(); // notify waiting consumers
```

```
        }
    }

    public T take() throws InterruptedException {
        synchronized(lock) {
            while (count == 0) {
                lock.wait(); // buffer empty, wait
            }

            T item = buffer[takeIndex];
            takeIndex = (takeIndex + 1) % buffer.length;
            count--;

            lock.notifyAll(); // notify waiting producers

            return item;
        }
    }
}

// 2. Using Java's BlockingQueue
class ProducerConsumerExample {
    private static final int CAPACITY = 10;
    private static final int PRODUCER_COUNT = 2;
    private static final int CONSUMER_COUNT = 3;

    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(CAPACITY);

        // Start producers
        for (int i = 0; i < PRODUCER_COUNT; i++) {
            new Thread(new Producer(queue, "P" + i)).start();
        }

        // Start consumers
        for (int i = 0; i < CONSUMER_COUNT; i++) {
            new Thread(new Consumer(queue, "C" + i)).start();
        }
    }

    static class Producer implements Runnable {
        private final BlockingQueue<Integer> queue;
        private final String name;
        private static int counter = 0;

        Producer(BlockingQueue<Integer> queue, String name) {
            this.queue = queue;
            this.name = name;
        }

        public void run() {
            try {
                while (true) {
                    int item = counter++;
                    queue.put(item); // blocks if queue full
                    System.out.println(name + " produced: " + item);
                    Thread.sleep(100); // simulate work
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    static class Consumer implements Runnable {
```

```
private final BlockingQueue<Integer> queue;
private final String name;

Consumer(BlockingQueue<Integer> queue, String name) {
this.queue = queue;
this.name = name;
}

public void run() {
try {
while (true) {
Integer item = queue.take(); // blocks if queue empty
                                        System.out.println(name + " consumed: " + item);
Thread.sleep(200); // simulate work (slower than producer)
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}
}
}
// 3. Multiple producers/consumers with poison pill
class PoisonPillExample {
                                    private static final int POISON_PILL = -1;

public static void main(String[] args) {
BlockingQueue<Integer> queue = new LinkedBlockingQueue<>();

// Start producers
                                          for (int i = 0; i < 3; i++) {
                                                 new Thread(() -> {
                                             for (int j = 0; j < 10; j++) {
try {
queue.put(j);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}
// Signal completion
try {
                                                       queue.put(POISON_PILL);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}).start();
}

// Consumer that stops on poison pill
                                                  new Thread(() -> {
try {
int poisonCount = 0;
while (poisonCount < 3) { // wait for all producers
int item = queue.take();
                                                    if (item == POISON_PILL) {
                                                            poisonCount++;
} else {
                                                System.out.println("Processing: " + item);
}
}
System.out.println("All done");
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}).start();
```

```
            }
        }
```

When to Use Producer-Consumer Pattern

Suitable Situations:

1. When data production and consumption rates differ.

2. When you have multiple producers and/or consumers.

3. When you want to decouple production from processing.

4. When you need to handle bursty workloads.

5. In pipeline and workflow systems.

Real-World Use Cases:

- **Logging Systems:** Application produces logs, consumers write to disk
- **Message Queues:** RabbitMQ, Kafka, ActiveMQ
- **Web Servers:** Request queue between accept and worker threads
- **Image Processing:** Upload images, process asynchronously
- **Print Spoolers:** Applications produce print jobs, printer consumes

Advantages of Producer-Consumer Pattern

1. **Decoupling:** Producers and consumers don't know each other.

2. **Load Leveling:** Queue smooths out bursty production.

3. **Flexible Scaling:** Can add more producers/consumers independently.

4. **Backpressure:** Bounded queue prevents overload.

5. **Thread Safety:** Queue handles synchronization.

Disadvantages of Producer-Consumer Pattern

1. **Complexity:** More complex than direct calls.

2. **Queue Overhead:** Memory and synchronization overhead.

3. **Potential for Stale Data:** Items may wait in queue.

4. **Deadlock Risk:** If producers and consumers block each other.

5. **Queue Size Dilemma:** Too small causes blocking, too large uses memory.

Comparison with Other Patterns

Pattern	Purpose	Difference
Producer-Consumer	Buffer between prod/cons	Core pattern
Pipeline	Sequential stages	Multiple queues in series
Publish-Subscribe	Multiple subscribers	Broadcast, not point-to-point
Active Object	Async method execution	Each object has queue

Producer-Consumer vs Pipeline

- **Producer-Consumer:** Single queue between producers and consumers
- **Pipeline:** Multiple stages, each with its own queue

Producer-Consumer vs Publish-Subscribe

- **Producer-Consumer:** Each item consumed by one consumer
- **Publish-Subscribe:** Each item goes to all subscribers

Common Pitfalls

1. **Unbounded Queues:** Can cause memory exhaustion.

2. **Lost Wakeups:** Signaling before consumer waits.

3. **Poison Pill Issues:** Not handling multiple producers.

4. **Thread Leakage:** Not handling interrupted exceptions.

5. **Priority Inversion:** Low-priority items processed first.

Best Practices

1. **Use Bounded Queues:** Prevent memory exhaustion.

2. **Implement Backpressure:** Slow producers when queue is full.

3. **Use Poison Pills:** For graceful shutdown.

4. **Monitor Queue Size:** Alert on queue buildup.

5. **Consider Priorities:** Use PriorityQueue if needed.

6. **Handle InterruptedException:** Proper cleanup.

7. **Test Under Load:** Verify behavior at limits.

[Page 23]

Chapter 21: Pipeline Pattern

Detailed Explanation

The Pipeline pattern chains together multiple processing stages, where each stage performs a specific operation on data and passes the result to the next stage. Data flows through the pipeline like an assembly line.

Understanding the Problem

Complex data processing often requires multiple steps:

- Read data → Parse → Validate → Transform → Enrich → Store
- Image: Load → Resize → Filter → Compress → Save
- Video: Decode → Process frames → Encode

Doing all steps sequentially uses only one thread. But different steps can often be parallelized if we pipeline them.

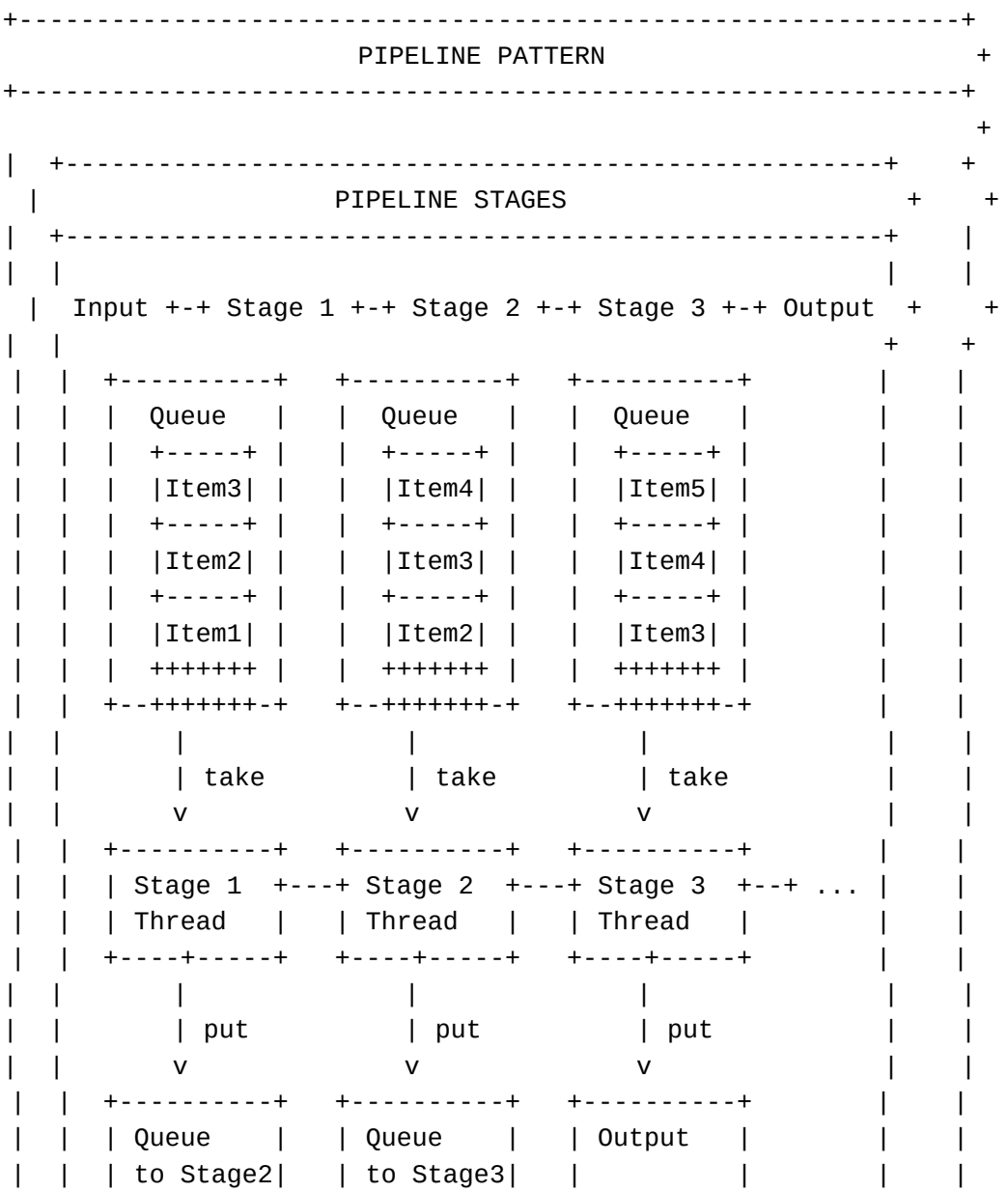
The Solution

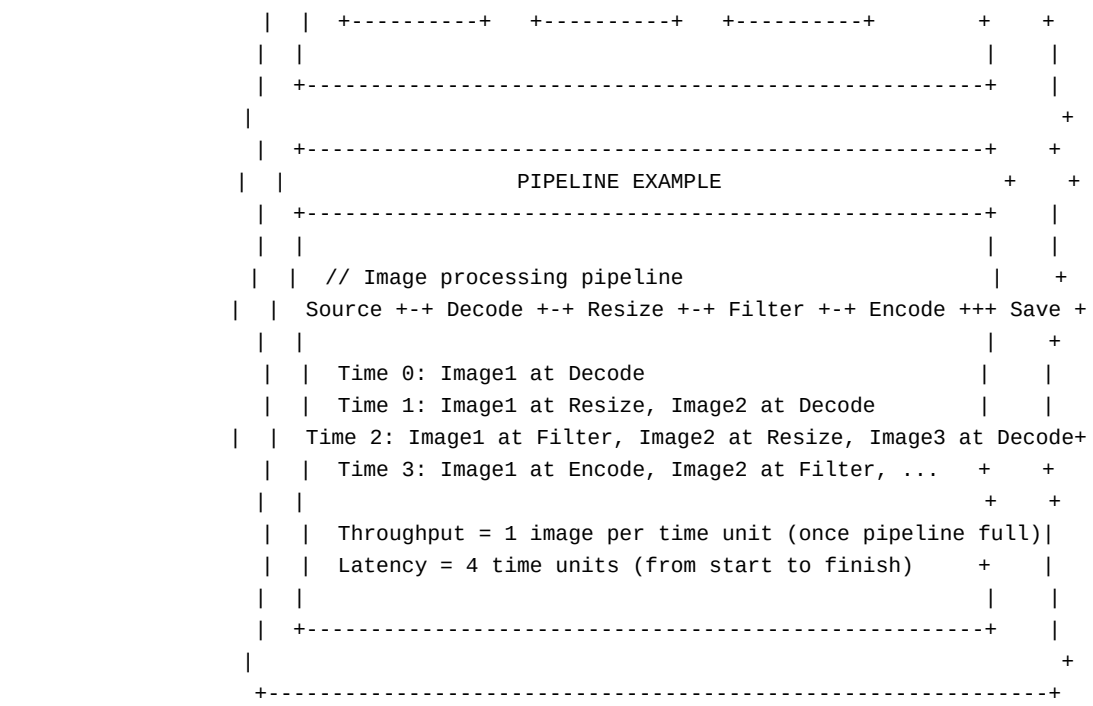
The Pipeline pattern breaks processing into stages connected by queues. Each stage runs in its own thread (or thread pool), processing items from its input queue and passing results to the next stage's queue.

Key Concepts

- **Stage:** Processing unit with input and output queues
- **Pipeline:** Sequence of stages
- **Data Item:** Unit of work flowing through pipeline
- **Parallelism:** Multiple items can be in different stages simultaneously
- **Backpressure:** Slow stages cause upstream queues to fill

Diagram: Pipeline Pattern





Implementation Example

```
import java.util.concurrent.*;
import java.util.function.Function;
// 1. Generic Pipeline Stage
class PipelineStage<I, O> {
private final BlockingQueue<I> inputQueue;
private final BlockingQueue<O> outputQueue;
private final Function<I, O> processor;
private final Thread thread;
private volatile boolean running = true;

public PipelineStage(Function<I, O> processor, int queueCapacity) {
this.inputQueue = new LinkedBlockingQueue<>(queueCapacity);
this.outputQueue = null; // for last stage
this.processor = processor;
this.thread = new Thread(this::run);
}

public PipelineStage(Function<I, O> processor,
BlockingQueue<O> outputQueue,
int queueCapacity) {
this.inputQueue = new LinkedBlockingQueue<>(queueCapacity);
this.outputQueue = outputQueue;
this.processor = processor;
this.thread = new Thread(this::run);
}

public void start() {
thread.start();
}

public void stop() {
running = false;
thread.interrupt();
}

public void submit(I item) throws InterruptedException {
inputQueue.put(item);
```

```
}

private void run() {
while (running) {
try {
I item = inputQueue.take();
O result = processor.apply(item);

if (outputQueue != null) {
outputQueue.put(result);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}
}
// 2. Pipeline Builder
class PipelineBuilder<T> {
private final BlockingQueue<T> sourceQueue;
private PipelineStage<?, ?> lastStage;

public PipelineBuilder(BlockingQueue<T> sourceQueue) {
this.sourceQueue = sourceQueue;
}

public <R> PipelineBuilder<R> addStage(
Function<T, R> processor, int queueCapacity) {

BlockingQueue<R> queue = new LinkedBlockingQueue<>(queueCapacity);
PipelineStage<T, R> stage = new PipelineStage<>(
processor, queue, queueCapacity);

// Connect previous stage to this one
if (lastStage == null) {
                                        // First stage - connect to source
                                                new Thread(() -> {
try {
while (true) {
T item = sourceQueue.take();
stage.submit(item);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}).start();
}

stage.start();
lastStage = stage;

return (PipelineBuilder<R>) this;
}

public void submit(T item) throws InterruptedException {
sourceQueue.put(item);
}
}
// 3. Complete Pipeline Example
public class PipelineExample {
public static void main(String[] args) throws Exception {
// Create pipeline stages
```

```
BlockingQueue<String> input = new LinkedBlockingQueue<>();
BlockingQueue<String> output = new LinkedBlockingQueue<>();

// Stage 1: Read and clean
PipelineStage<String, String> stage1 = new PipelineStage<>(
                                                    s -> {
                                            System.out.println("Stage1 cleaning: " + s);
return s.trim().toLowerCase();
},
new LinkedBlockingQueue<>(10),
10
);

// Stage 2: Parse
PipelineStage<String, String[]> stage2 = new PipelineStage<>(
                                                    s -> {
                                             System.out.println("Stage2 parsing: " + s);
                                                      return s.split("\\s+");
},
new LinkedBlockingQueue<>(10),
10
);

// Stage 3: Count words
PipelineStage<String[], Integer> stage3 = new PipelineStage<>(
                                                  words -> {
                                    System.out.println("Stage3 counting: " + Arrays.toString(words));
return words.length;
},
output,
10
);

// Connect stages
                                        // input -> stage1 -> stage2 -> stage3 -> output

// Connect input to stage1
                                                 new Thread(() -> {
try {
while (true) {
String item = input.take();
stage1.submit(item);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}).start();

// Connect stage1 to stage2
// (In a real implementation, stages would be connected directly)
                                                 new Thread(() -> {
try {
while (true) {
// This is simplified; real implementation would have
// stage1's output queue connected to stage2's input
}
} catch (Exception e) {
Thread.currentThread().interrupt();
}
}).start();

// Start stages
stage1.start();
stage2.start();
stage3.start();
```

```
// Submit work
input.put("Hello World");
input.put("This is a test");
input.put("Pipeline pattern example");

// Collect results
                                        for (int i = 0; i < 3; i++) {
Integer result = output.take();
                                    System.out.println("Word count: " + result);
}

// Cleanup
stage1.stop();
stage2.stop();
stage3.stop();
}
}
```

Better Implementation Using ExecutorService

```
import java.util.concurrent.*;
import java.util.function.Function;
class PipelineStage2<I, O> {
private final BlockingQueue<I> inputQueue;
private final BlockingQueue<O> outputQueue;
private final Function<I, O> processor;
private final ExecutorService executor;
private final String name;

public PipelineStage2(String name, Function<I, O> processor,
int queueCapacity, ExecutorService executor) {
this.name = name;
this.inputQueue = new LinkedBlockingQueue<>(queueCapacity);
this.outputQueue = new LinkedBlockingQueue<>();
this.processor = processor;
this.executor = executor;

start();
}

private void start() {
                                            executor.submit(() -> {
try {
while (!Thread.interrupted()) {
I item = inputQueue.take();
O result = processor.apply(item);
outputQueue.put(result);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
});
}

public void submit(I item) throws InterruptedException {
inputQueue.put(item);
}

public O take() throws InterruptedException {
return outputQueue.take();
}
public BlockingQueue<O> getOutputQueue() {
```

```
return outputQueue;
}
}
class PipelineBuilder2 {
private final ExecutorService executor;
private BlockingQueue<?> lastOutputQueue;

public PipelineBuilder2(ExecutorService executor) {
this.executor = executor;
}

public <I, O> PipelineBuilder2 addStage(String name,
Function<I, O> processor,
int queueCapacity) {
PipelineStage2<I, O> stage = new PipelineStage2<>(
name, processor, queueCapacity, executor);

if (lastOutputQueue != null) {
// Connect previous stage to this one
BlockingQueue<I> prevQueue = (BlockingQueue<I>) lastOutputQueue;
connectQueues(prevQueue, stage);
}

lastOutputQueue = stage.getOutputQueue();
return this;
}

private <I> void connectQueues(BlockingQueue<I> source,
PipelineStage2<I, ?> target) {
                                        executor.submit(() -> {
try {
while (!Thread.interrupted()) {
I item = source.take();
target.submit(item);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
});
}

public BlockingQueue<?> getOutputQueue() {
return lastOutputQueue;
}
}
```

When to Use Pipeline Pattern

Suitable Situations:

1. When processing has multiple distinct stages.
2. When stages can operate in parallel.
3. When you want to increase throughput through parallelism.

4. When each stage can be developed independently.

5. When you need to process streams of data.

Real-World Use Cases:

- **ETL Pipelines:** Extract, Transform, Load
- **Video Processing:** Decode, process, encode
- **Compiler Design:** Lexical analysis, parsing, code generation
- **Image Processing:** Load, resize, filter, save
- **Data Analysis Pipelines:** Filter, map, reduce

Advantages of Pipeline Pattern

1. **Parallelism:** Multiple items processed simultaneously.

2. **Modularity:** Stages are independent and reusable.

3. **Scalability:** Can scale individual stages.

4. **Throughput:** Increases overall throughput.

5. **Decoupling:** Stages don't need to know about each other.

Disadvantages of Pipeline Pattern

1. **Latency:** First item takes time to go through pipeline.

2. **Complexity:** More complex than sequential processing.

3. **Load Balancing:** Fast stages may wait for slow ones.

4. **Backpressure:** Slow stages cause upstream queues to fill.

5. **Debugging Difficulty:** Hard to trace items through pipeline.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Pipeline	Sequential stages	Linear flow
Producer-Consumer	Single queue	Building block for pipeline
Fork-Join	Tree decomposition	Not linear
MapReduce	Map then reduce	Two stages only

Pipeline vs Producer-Consumer

- **Pipeline:** Multiple stages in series
- **Producer-Consumer:** Single stage with queue

Pipeline vs Fork-Join

- **Pipeline:** Linear flow, items flow through
- **Fork-Join:** Tree decomposition, combine results

Common Pitfalls

1. **Unbounded Queues:** Memory exhaustion.

2. **Uneven Stage Speeds:** Fast stages blocked by slow ones.

3. **Deadlocks:** Circular dependencies between stages.

4. **Thread Leaks:** Not shutting down stages properly.

5. **Error Handling:** Errors in one stage affect others.

Best Practices

1. **Use Bounded Queues:** Provide backpressure.

2. **Monitor Queue Sizes:** Detect bottlenecks.

3. **Implement Timeouts:** Prevent indefinite blocking.

4. **Handle Errors Gracefully:** Poison pills or error queues.

5. **Balance Stage Speeds:** Consider parallelizing slow stages.

6. **Test with Various Loads:** Verify pipeline behavior.

7. **Consider Circuit Breakers:** Prevent cascade failures.

[Page 24]

Chapter 22: Fan-Out / Fan-In Pattern

Detailed Explanation

The Fan-Out/Fan-In pattern distributes work to multiple concurrent workers (fan-out) and then combines their results (fan-in). It's useful for parallelizing independent subtasks.

Understanding the Problem

Many tasks can be broken into independent subtasks:

- Processing multiple files simultaneously
- Querying multiple databases or services
- Performing independent calculations
- Fetching multiple web pages

Doing these sequentially is slow. But if we do them in parallel, we need to wait for all results and combine them.

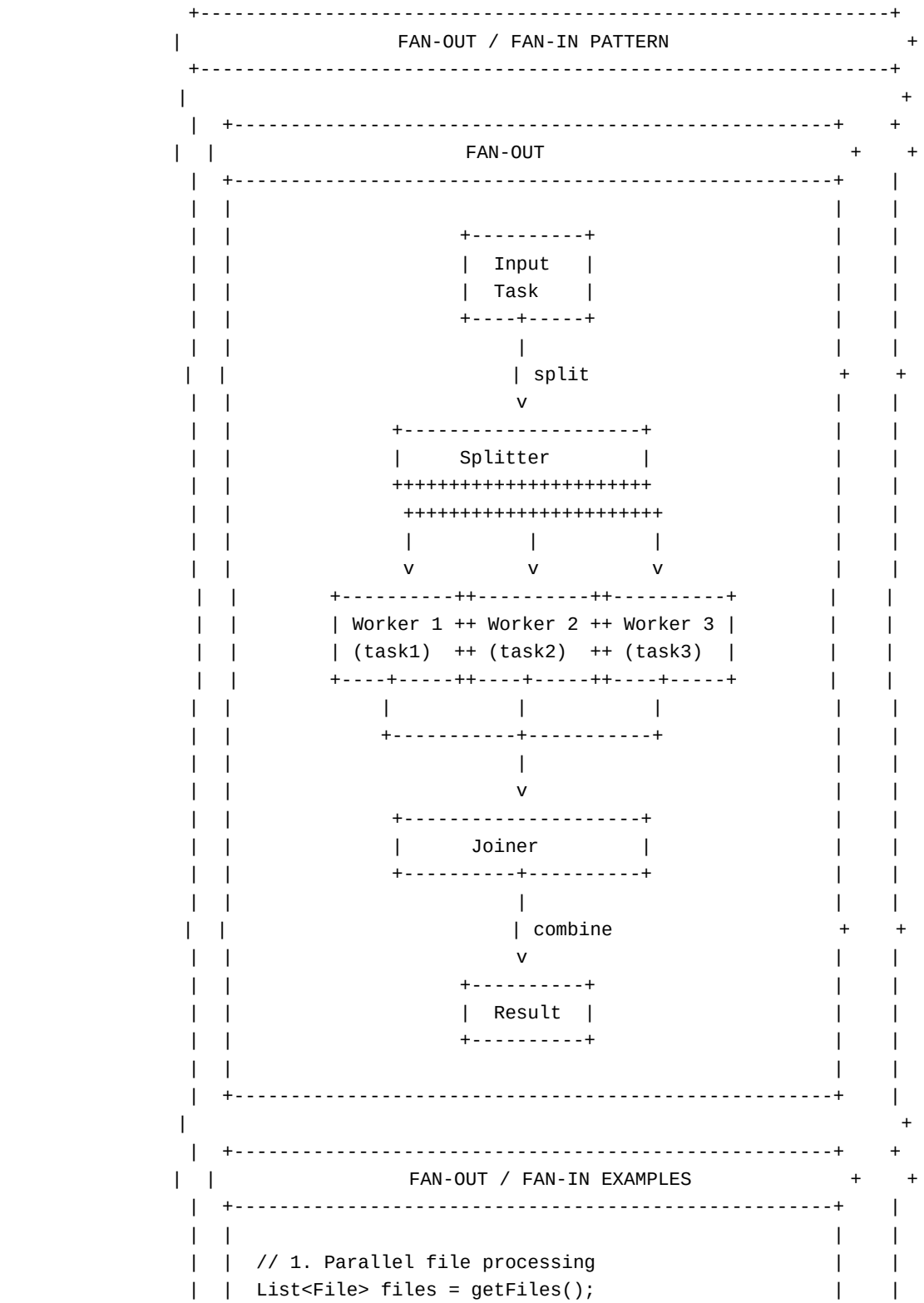
The Solution

The Fan-Out/Fan-In pattern provides:

1. **Fan-Out:** Distribute work to multiple workers (threads, processes, services)
2. **Parallel Processing:** Workers execute independently
3. **Fan-In:** Collect and combine results from all workers
4. **Synchronization:** Wait for all workers to complete

Key Concepts
- **Splitter:** Distributes work items to workers
- **Workers:** Process individual work items
- **Joiner:** Collects and combines results
- **Aggregator:** Combines partial results
- **Barrier:** Waits for all workers to complete

Diagram: Fan-Out / Fan-In





Implementation Example

```
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;
// 1. Simple Fan-Out/Fan-In with CompletableFuture
class FanOutFanInExample {
private final ExecutorService executor =
Executors.newFixedThreadPool(10);

public List<Result> processFiles(List<File> files) {
                                        // Fan-out: submit all tasks
List<CompletableFuture<Result>> futures = files.stream()
                                    .map(file -> CompletableFuture.supplyAsync(
                                          () -> processFile(file), executor))
.collect(Collectors.toList());

                                            // Fan-in: wait for all
CompletableFuture<Void> allDone = CompletableFuture
.allOf(futures.toArray(new CompletableFuture[0]));

// This blocks until all complete
allDone.join();

// Collect results
return futures.stream()
.map(CompletableFuture::join)
.collect(Collectors.toList());
}

private Result processFile(File file) {
// Simulate processing
return new Result(file.getName(), file.length());
}
```

```
}
// 2. Custom Fan-Out/Fan-In Implementation
class FanOutFanIn<T, R> {
private final int workerCount;
private final Function<T, R> worker;
private final ExecutorService executor;

public FanOutFanIn(int workerCount, Function<T, R> worker) {
this.workerCount = workerCount;
this.worker = worker;
this.executor = Executors.newFixedThreadPool(workerCount);
}

public List<R> process(List<T> items) throws InterruptedException {
// Create a queue for results
BlockingQueue<R> results = new LinkedBlockingQueue<>();

                                        // Fan-out: submit all tasks
CountDownLatch latch = new CountDownLatch(items.size());

for (T item : items) {
                                             executor.submit(() -> {
try {
R result = worker.apply(item);
results.put(result);
} catch (Exception e) {
// Handle error
} finally {
latch.countDown();
}
});
}

                                      // Fan-in: wait for all to complete
latch.await();

// Collect all results
List<R> resultList = new ArrayList<>();
results.drainTo(resultList);
return resultList;
}

public void shutdown() {
executor.shutdown();
}
}
// 3. Parallel Service Aggregator
class ServiceAggregator {
private final ExecutorService executor =
Executors.newFixedThreadPool(3);

public UserProfile getUserProfile(String userId) {
                                     // Fan-out: call services in parallel
CompletableFuture<UserInfo> userInfoFuture =
CompletableFuture.supplyAsync(
                                        () -> getUserInfo(userId), executor);

CompletableFuture<List<Order>> ordersFuture =
CompletableFuture.supplyAsync(
                                         () -> getOrders(userId), executor);

CompletableFuture<List<Review>> reviewsFuture =
CompletableFuture.supplyAsync(
```

```
                                        () -> getReviews(userId), executor);

                                    // Fan-in: wait for all and combine
return CompletableFuture
.allOf(userInfoFuture, ordersFuture, reviewsFuture)
                                                .thenApply(v -> {
UserInfo info = userInfoFuture.join();
List<Order> orders = ordersFuture.join();
List<Review> reviews = reviewsFuture.join();
return new UserProfile(info, orders, reviews);
})
.join(); // Block for demo
}

private UserInfo getUserInfo(String userId) {
// Simulate API call
return new UserInfo(userId, "John Doe");
}

private List<Order> getOrders(String userId) {
// Simulate database query
return Arrays.asList(new Order("1", 100), new Order("2", 200));
}

private List<Review> getReviews(String userId) {
// Simulate API call
return Arrays.asList(new Review("Great!", 5));
}
}
// 4. Scatter-Gather Implementation
class ScatterGather<T, R> {
private final List<Function<T, R>> workers;
private final ExecutorService executor;

public ScatterGather(List<Function<T, R>> workers, int threadPoolSize) {
this.workers = workers;
this.executor = Executors.newFixedThreadPool(threadPoolSize);
}

public List<R> gather(T input, long timeout, TimeUnit unit)
throws InterruptedException, TimeoutException {

List<CompletableFuture<R>> futures = workers.stream()
                                    .map(worker -> CompletableFuture.supplyAsync(
                                          () -> worker.apply(input), executor))
.collect(Collectors.toList());

CompletableFuture<Void> all = CompletableFuture
.allOf(futures.toArray(new CompletableFuture[0]));

try {
all.get(timeout, unit);
} catch (ExecutionException e) {
// Handle
}

// Return completed results (some may be incomplete on timeout)
return futures.stream()
                                .filter(f -> f.isDone() && !f.isCompletedExceptionally())
.map(CompletableFuture::join)
.collect(Collectors.toList());
}
}
```

When to Use Fan-Out / Fan-In Pattern

Suitable Situations:

1. When tasks can be broken into independent subtasks.

2. When you need to call multiple services in parallel.

3. When processing multiple files or data items.

4. When you need to aggregate results from multiple sources.

5. In map-reduce style computations.

Real-World Use Cases:

- **Web Scraping:** Fetch multiple pages in parallel
- **Microservices:** Parallel service calls
- **Data Processing:** Process multiple files
- **Search Engines:** Query multiple shards
- **Batch Processing:** Parallel job execution

Advantages of Fan-Out / Fan-In Pattern

1. **Performance:** Significant speedup for independent tasks.

2. **Scalability:** Can add more workers.

3. **Resilience:** Failure of one worker doesn't affect others.

4. **Simplicity:** Clear pattern for parallel processing.

5. **Resource Efficiency:** Can limit concurrent workers.

Disadvantages of Fan-Out / Fan-In Pattern

1. **Resource Usage:** Many threads consume resources.

2. **Overhead:** Creating many tasks has overhead.

3. **Error Handling:** Complex to handle partial failures.

4. **Load Imbalance:** Some workers may finish early.

5. **Result Aggregation:** Combining results can be complex.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Fan-Out/Fan-In	Parallel independent tasks	Split and combine
Fork-Join	Recursive decomposition	Tree structure
MapReduce	Map then reduce	Two-phase, distributed
Pipeline	Sequential stages	Linear flow

Fan-Out/Fan-In vs Fork-Join

- **Fan-Out/Fan-In:** Independent tasks, no dependencies
- **Fork-Join:** Recursive, tasks may fork further

Fan-Out/Fan-In vs MapReduce

- **Fan-Out/Fan-In:** General parallel pattern
- **MapReduce:** Specialized for large datasets with map and reduce

Common Pitfalls

1. **Too Many Threads:** Overwhelming the system.

2. **Not Handling Errors:** One failure shouldn't stop all.

3. **Memory Leaks:** Futures not cleaned up.

4. **Deadlocks:** If tasks depend on each other.

5. **Timeout Issues:** Not setting timeouts for external calls.

Best Practices

1. **Limit Concurrency:** Use bounded thread pools.

2. **Handle Partial Results:** Decide what to do if some tasks fail.

3. **Set Timeouts:** Prevent waiting forever.

4. **Use CompletableFuture:** For clean composition.

5. **Monitor Queue Sizes:** Detect bottlenecks.

6. **Consider Cancellation:** Cancel remaining tasks if one fails.

7. **Test with Various Loads:** Verify behavior at limits.

[Page 25]

Chapter 23: MapReduce Pattern

Detailed Explanation

The MapReduce pattern processes large datasets by dividing work into map and reduce phases. It's designed for distributed processing across many machines.

Understanding the Problem

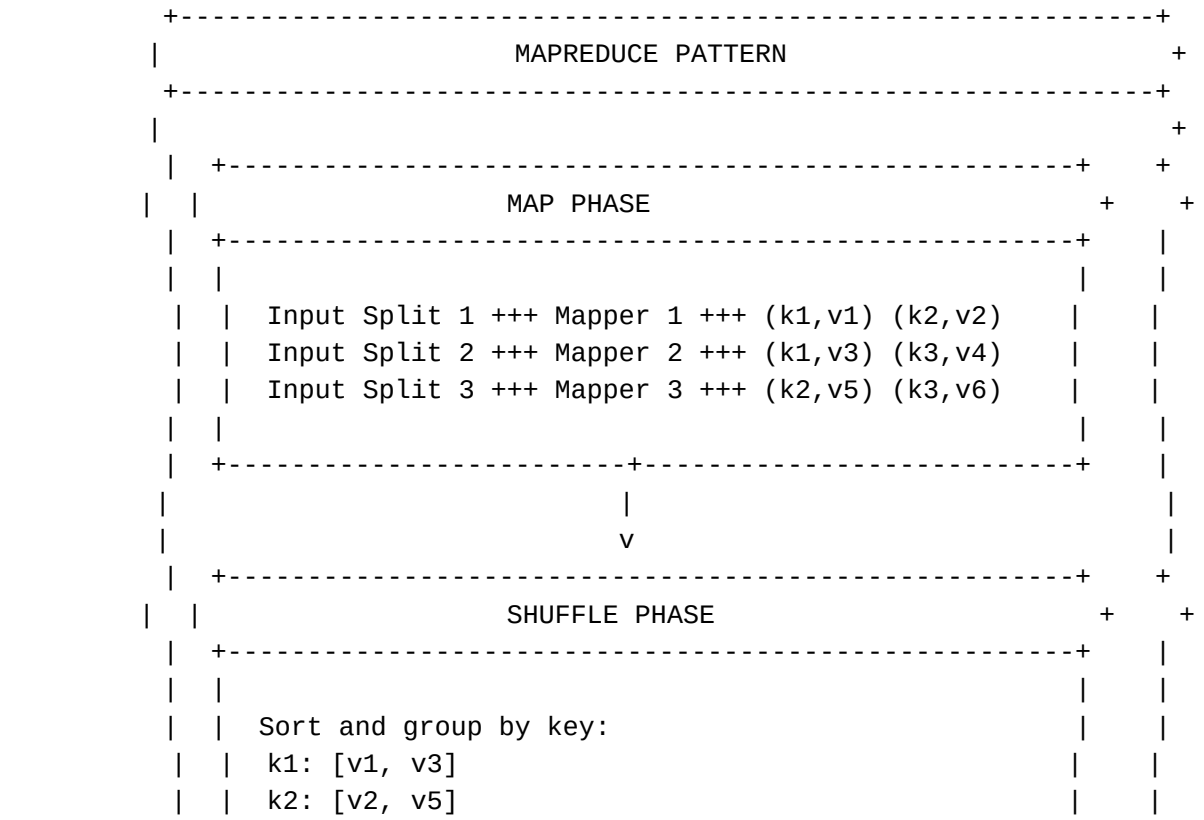
Processing extremely large datasets (terabytes or petabytes) on a single machine is impossible due to:

1. **Storage Limits:** Data too large for one machine
2. **Processing Limits:** Would take too long
3. **Fault Tolerance:** Single machine failure stops everything

The Solution

MapReduce divides processing into two phases:

1. **Map Phase:**
- Split input into chunks
- Process chunks in parallel across many machines
- Each map task produces key-value pairs
2. **Shuffle Phase:**
- Sort and group by key
- Transfer data from mappers to reducers
3. **Reduce Phase:**
- Process each group of values for a key
- Produce final results
Key Concepts
- **Mapper:** Processes input chunk, emits key-value pairs
- **Reducer:** Aggregates values for each key
- **Shuffle:** Sorts and groups by key between phases
- **Partitioner:** Distributes keys to reducers
- **Combiner:** Local aggregation on mapper (mini-reducer)
Diagram: MapReduce





Implementation Example (Simplified)

```
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;
// 1. Simple In-Memory MapReduce Framework
class MapReduce<MapKey, MapValue, ReduceKey, ReduceValue> {
private final int numMappers;
private final int numReducers;
private final ExecutorService executor;

public MapReduce(int numMappers, int numReducers) {
this.numMappers = numMappers;
this.numReducers = numReducers;
                    this.executor = Executors.newFixedThreadPool(numMappers + numReducers);
}

public List<Pair<ReduceKey, ReduceValue>> execute(
```

```
List<?> input,
Mapper<MapKey, MapValue> mapper,
Reducer<ReduceKey, MapKey, MapValue, ReduceValue> reducer)
throws InterruptedException, ExecutionException {

// 1. Split input into chunks
List<List<?>> splits = splitInput(input, numMappers);

// 2. Map phase
List<Future<List<Pair<MapKey, MapValue>>>> mapFutures = new ArrayList<>();
for (List<?> split : splits) {
                                mapFutures.add(executor.submit(() -> mapper.map(split)));
}

// Collect all map outputs
List<Pair<MapKey, MapValue>> mapOutputs = new ArrayList<>();
for (Future<List<Pair<MapKey, MapValue>>> future : mapFutures) {
mapOutputs.addAll(future.get());
}

// 3. Shuffle phase (group by key)
Map<ReduceKey, List<MapValue>> grouped = groupByKey(mapOutputs, reducer);

// 4. Reduce phase
List<Future<Pair<ReduceKey, ReduceValue>>> reduceFutures = new ArrayList<>();
for (Map.Entry<ReduceKey, List<MapValue>> entry : grouped.entrySet()) {
                                        reduceFutures.add(executor.submit(() ->
reducer.reduce(entry.getKey(), entry.getValue())));
}

// Collect results
List<Pair<ReduceKey, ReduceValue>> results = new ArrayList<>();
for (Future<Pair<ReduceKey, ReduceValue>> future : reduceFutures) {
results.add(future.get());
}

return results;
}

private List<List<?>> splitInput(List<?> input, int numSplits) {
List<List<?>> splits = new ArrayList<>();
int chunkSize = (int) Math.ceil((double) input.size() / numSplits);

                                for (int i = 0; i < input.size(); i += chunkSize) {
                                    int end = Math.min(i + chunkSize, input.size());
splits.add(input.subList(i, end));
}
return splits;
}

private Map<ReduceKey, List<MapValue>> groupByKey(
List<Pair<MapKey, MapValue>> mapOutputs,
Reducer<ReduceKey, MapKey, MapValue, ReduceValue> reducer) {

Map<ReduceKey, List<MapValue>> grouped = new HashMap<>();

for (Pair<MapKey, MapValue> pair : mapOutputs) {
ReduceKey key = reducer.getReduceKey(pair.key);
                                  grouped.computeIfAbsent(key, k -> new ArrayList<>())
.add(pair.value);
}

return grouped;
}
public void shutdown() {
```

```
executor.shutdown();
}

// Helper classes
static class Pair<K, V> {
K key;
V value;
Pair(K key, V value) { this.key = key; this.value = value; }
}

interface Mapper<K, V> {
List<Pair<K, V>> map(List<?> input);
}

interface Reducer<ReduceKey, MapKey, MapValue, ReduceValue> {
ReduceKey getReduceKey(MapKey mapKey);
Pair<ReduceKey, ReduceValue> reduce(ReduceKey key, List<MapValue> values);
}
}
// 2. Word Count Example
class WordCount {
public static void main(String[] args) throws Exception {
// Create input data
List<String> documents = Arrays.asList(
"the quick brown fox jumps over the lazy dog",
"the quick brown fox jumps over the lazy dog",
"hello world hello world",
"map reduce is great"
);

// Create MapReduce instance
MapReduce<String, Integer, String, Integer> mr =
new MapReduce<>(2, 2);

// Define mapper
                                MapReduce.Mapper<String, Integer> mapper = input -> {
List<MapReduce.Pair<String, Integer>> result = new ArrayList<>();
for (Object item : input) {
String doc = (String) item;
                                            for (String word : doc.split("\\s+")) {
result.add(new MapReduce.Pair<>(word, 1));
}
}
return result;
};

// Define reducer
MapReduce.Reducer<String, String, Integer, Integer> reducer =
new MapReduce.Reducer<>() {
public String getReduceKey(String mapKey) {
return mapKey; // same key
}

public MapReduce.Pair<String, Integer> reduce(
String key, List<Integer> values) {
int sum = values.stream().mapToInt(Integer::intValue).sum();
return new MapReduce.Pair<>(key, sum);
}
                                                        +--+
};

// Execute MapReduce
List<MapReduce.Pair<String, Integer>> results =
mr.execute(documents, mapper, reducer);
```

```
// Print results
                                    results.sort((a, b) -> b.value - a.value);
for (MapReduce.Pair<String, Integer> pair : results) {
                                   System.out.println(pair.key + ": " + pair.value);
}

mr.shutdown();
}
}
// 3. Parallel Stream Implementation (Java 8+)
class StreamMapReduce {
public static void main(String[] args) {
List<String> documents = Arrays.asList(
"the quick brown fox jumps over the lazy dog",
"the quick brown fox jumps over the lazy dog",
"hello world hello world",
"map reduce is great"
);

// Map phase (parallel)
Map<String, Integer> wordCounts = documents.parallelStream()
                                    .flatMap(doc -> Arrays.stream(doc.split("\\s+")))
                                   .map(word -> new AbstractMap.SimpleEntry<>(word, 1))
.collect(
Collectors.groupingByConcurrent(
Map.Entry::getKey,
Collectors.summingInt(Map.Entry::getValue)
)
);

// Print results
wordCounts.entrySet().stream()
                                    .sorted((a, b) -> b.getValue() - a.getValue())
                         .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));
}
}
// 4. CompletableFuture-based MapReduce
class CompletableFutureMapReduce {
private final ExecutorService executor =
Executors.newFixedThreadPool(10);

public CompletableFuture<Map<String, Long>> processFiles(
List<Path> files) {

// Map phase: process each file asynchronously
List<CompletableFuture<Map<String, Long>>> mapFutures = files.stream()
                                       .map(file -> CompletableFuture.supplyAsync(
                                             () -> processFile(file), executor))
.collect(Collectors.toList());

// Combine all map results (reduce phase)
CompletableFuture<Map<String, Long>> result =
CompletableFuture.completedFuture(new HashMap<>());

for (CompletableFuture<Map<String, Long>> future : mapFutures) {
result = result.thenCombine(future, this::mergeMaps);
}

return result;
}

private Map<String, Long> processFile(Path file) {
// Simulate file processing
```

```
Map<String, Long> counts = new HashMap<>();
                        counts.put("word" + file.hashCode(), 1L);
return counts;
}

private Map<String, Long> mergeMaps(
Map<String, Long> m1, Map<String, Long> m2) {
Map<String, Long> result = new HashMap<>(m1);
                    m2.forEach((k, v) -> result.merge(k, v, Long::sum));
return result;
}
}
```

When to Use MapReduce Pattern

Suitable Situations:

1. When processing extremely large datasets (terabytes+).
2. When the problem can be expressed as map and reduce operations.
3. When you need fault tolerance and distributed processing.
4. When you have many machines available.
5. For batch processing, not real-time.

Real-World Use Cases:

- **Search Engines:** Building inverted indexes
- **Log Analysis:** Processing server logs
- **Data Mining:** Finding patterns in large datasets
- **Machine Learning:** Training data preparation
- **ETL Pipelines:** Extract, transform, load operations

Advantages of MapReduce Pattern

1. **Scalability:** Can scale to thousands of machines.
2. **Fault Tolerance:** Tasks can be retried on failure.
3. **Simplicity:** Programmers only write map and reduce functions.

4. **Data Locality:** Moves computation to data.

5. **Load Balancing:** Automatically handles stragglers.

Disadvantages of MapReduce Pattern

1. **Batch Orientation:** Not for real-time processing.

2. **Rigid Structure:** Only map and reduce phases.

3. **Disk I/O Heavy:** Intermediate data written to disk.

4. **Latency:** High latency due to shuffling.

5. **Complexity:** Distributed systems are complex.

Comparison with Other Patterns

Pattern	Purpose	Difference
MapReduce	Large-scale distributed	Designed for clusters
Fork-Join	In-memory parallel	Single JVM
Fan-Out/Fan-In	Parallel independent tasks	Simpler, no shuffle
Pipeline	Sequential stages	Linear flow

MapReduce vs Fork-Join

- **MapReduce:** Distributed, disk-based, fault-tolerant
- **Fork-Join:** In-memory, single JVM, no fault tolerance

MapReduce vs Spark

- **MapReduce:** Writes intermediate results to disk
- **Spark:** Keeps data in memory (faster for iterative algorithms)

Common Pitfalls

1. **Too Many Small Files:** Overheads of many map tasks.

2. **Skewed Data:** Some reducers get much more data.

3. **Stragglers:** Slow tasks delay entire job.

4. **Resource Contention:** Competing for cluster resources.

5. **Debugging Difficulty:** Hard to debug distributed jobs.

Best Practices

1. **Use Combiners:** Reduce data shuffled.

2. **Balance Partitions:** Ensure even distribution.

3. **Handle Stragglers:** Speculative execution.

4. **Monitor Jobs:** Track progress and resource usage.

5. **Test with Sample Data:** Verify logic on smaller datasets.

6. **Consider Alternatives:** Use Spark for iterative algorithms.

[Page 26]

PART FIVE: EVENT HANDLING PATTERNS

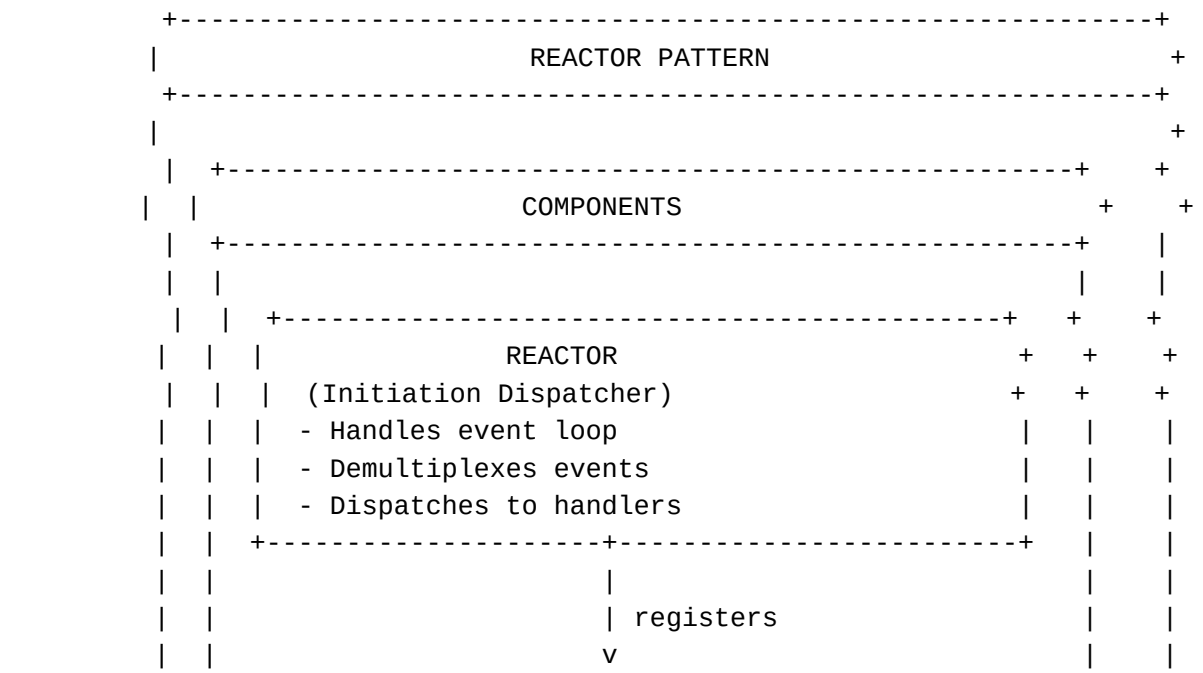
Chapter 24: Reactor Pattern

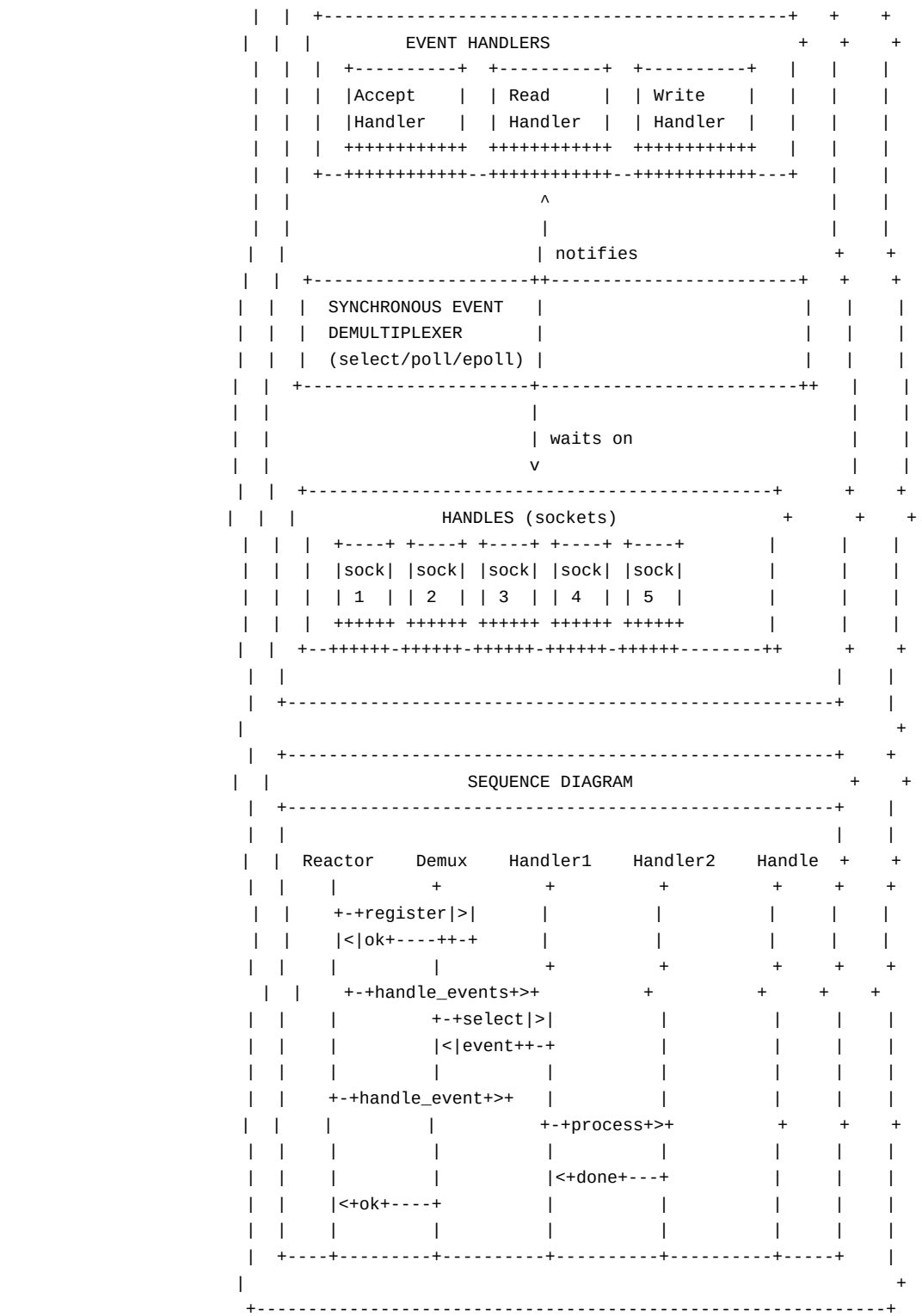
Detailed Explanation

The Reactor pattern handles service requests delivered concurrently to an application by one or more clients. It demultiplexes incoming requests and dispatches them synchronously to associated request handlers.

Understanding the Problem

In network servers, you need to handle multiple connections simultaneously. Traditional approaches:
- **Thread-per-connection:** Creates many threads, not scalable
- **Thread pool:** Still has context switching overhead
- **Polling:** Wastes CPU cycles
The challenge is to handle many connections efficiently with limited threads.
The Solution
The Reactor pattern uses:
1. **Event Demultiplexer:** Waits for events on multiple handles (select/poll/epoll)
2. **Reactor:** Demultiplexes events and dispatches to handlers
3. **Handlers:** Process specific events
Key Concepts
- **Handle:** Resource (socket, file descriptor) that can generate events
- **Synchronous Event Demultiplexer:** select(), poll(), epoll()
- **Reactor:** Demultiplexes and dispatches events
- **Event Handler:** Processes specific event types
- **Initiation Dispatcher:** Registers handlers and dispatches events
Diagram: Reactor Pattern





Implementation Example

```
import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.*;
import java.util.Iterator;
import java.util.Set;
import java.util.concurrent.*;
// 1. Event Handler Interface
interface EventHandler {
```

```
    void handleEvent(SelectionKey key) throws IOException;
}

// 2. Concrete Handlers
class AcceptHandler implements EventHandler {
    private final ServerSocketChannel serverChannel;

    public AcceptHandler(ServerSocketChannel serverChannel) {
        this.serverChannel = serverChannel;
    }

    @Override
    public void handleEvent(SelectionKey key) throws IOException {
        SocketChannel clientChannel = serverChannel.accept();
        clientChannel.configureBlocking(false);

        // Register read interest for new connection
        clientChannel.register(key.selector(), SelectionKey.OP_READ,
            new ReadHandler());

        System.out.println("Accepted connection from: " +
            clientChannel.getRemoteAddress());
    }
}

class ReadHandler implements EventHandler {
    private final ByteBuffer buffer = ByteBuffer.allocate(1024);

    @Override
    public void handleEvent(SelectionKey key) throws IOException {
        SocketChannel clientChannel = (SocketChannel) key.channel();

        int bytesRead = clientChannel.read(buffer);

        if (bytesRead == -1) {
            clientChannel.close();
            System.out.println("Connection closed");
            return;
        }

        if (bytesRead > 0) {
            buffer.flip();
            byte[] data = new byte[buffer.limit()];
            buffer.get(data);
            String message = new String(data);
            System.out.println("Received: " + message);

            // Echo back
            buffer.rewind();
            clientChannel.write(buffer);
            buffer.clear();
        }
    }
}

// 3. Reactor (Initiation Dispatcher)
class Reactor {
    private final Selector selector;
    private final ExecutorService workerPool;
    private volatile boolean running = true;

    public Reactor() throws IOException {
        this.selector = Selector.open();
        this.workerPool = Executors.newFixedThreadPool(10);
    }
```

```
public void registerChannel(SelectableChannel channel, int ops,
EventHandler handler) throws IOException {
channel.configureBlocking(false);
channel.register(selector, ops, handler);
}

public void run() {
try {
while (running) {
// Wait for events (synchronous event demultiplexing)
selector.select();

// Get ready events
Set<SelectionKey> selectedKeys = selector.selectedKeys();
Iterator<SelectionKey> iter = selectedKeys.iterator();

while (iter.hasNext()) {
SelectionKey key = iter.next();
iter.remove();

// Dispatch event to handler
if (key.isValid()) {
EventHandler handler = (EventHandler) key.attachment();

// Handle in worker thread for long operations
                                        if (key.isAcceptable() || key.isReadable()) {
                                                workerPool.submit(() -> {
try {
handler.handleEvent(key);
} catch (IOException e) {
e.printStackTrace();
}
});
} else {
handler.handleEvent(key);
}
}
}
}
} catch (IOException e) {
e.printStackTrace();
}
}

public void shutdown() {
running = false;
workerPool.shutdown();
try {
selector.close();
} catch (IOException e) {
e.printStackTrace();
}
}
}
// 4. Server Example
public class ReactorServer {
public static void main(String[] args) throws IOException {
Reactor reactor = new Reactor();

// Create server socket channel
ServerSocketChannel serverChannel = ServerSocketChannel.open();
serverChannel.bind(new InetSocketAddress(8080));

// Register with reactor for accept events
```

```
                reactor.registerChannel(serverChannel, SelectionKey.OP_ACCEPT,
new AcceptHandler(serverChannel));

System.out.println("Reactor server started on port 8080");

// Run reactor in separate thread
                            new Thread(() -> reactor.run()).start();

// Shutdown hook
                    Runtime.getRuntime().addShutdownHook(new Thread(() -> {
reactor.shutdown();
System.out.println("Server stopped");
}));
}
}
```

Simplified Reactor with Single Thread

```
class SimpleReactor {
private final Selector selector;
private final Map<SelectionKey, Handler> handlers = new HashMap<>();

public SimpleReactor() throws IOException {
this.selector = Selector.open();
}

public void registerChannel(SelectableChannel channel, int ops,
Handler handler) throws IOException {
channel.configureBlocking(false);
SelectionKey key = channel.register(selector, ops);
handlers.put(key, handler);
}

public void run() throws IOException {
while (true) {
selector.select();
Iterator<SelectionKey> iter = selector.selectedKeys().iterator();

while (iter.hasNext()) {
SelectionKey key = iter.next();
iter.remove();

Handler handler = handlers.get(key);
if (handler != null) {
handler.handle(key);
}
}
}
}

interface Handler {
void handle(SelectionKey key) throws IOException;
}
}
```

When to Use Reactor Pattern

Suitable Situations:

1. When handling many concurrent connections.

2. When connections are mostly idle (waiting for I/O).

3. When you need high scalability.

4. In network servers (HTTP, WebSocket, etc.).

5. When you want to minimize thread usage.

Real-World Use Cases:

- **Web Servers:** Nginx, Node.js (uses event loop similar to reactor)
- **Database Drivers:** Asynchronous database clients
- **Message Queues:** RabbitMQ, Kafka
- **Proxy Servers:** Handling many connections
- **Chat Servers:** Managing many client connections

Advantages of Reactor Pattern

1. **Scalability:** Handles thousands of connections with few threads.

2. **Efficiency:** No thread per connection overhead.

3. **Resource Usage:** Minimal threads.

4. **Performance:** Good for I/O-bound applications.

5. **Separation of Concerns:** Event handling separated from demultiplexing.

Disadvantages of Reactor Pattern

1. **Complexity:** More complex than thread-per-connection.

2. **Debugging Difficulty:** Event flow harder to trace.

3. **Blocking Operations:** Blocking in handler blocks reactor.

4. **CPU-Bound Tasks:** Not suitable for CPU-intensive work.

5. **Ordering Guarantees:** Events may be processed out of order.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Reactor	Sync event demux	Single-threaded demux
Proactor	Async event demux	Uses async I/O
Leader-Followers	Thread pool variant	Different thread model
Event Loop	Similar concept	Often single-threaded

Reactor vs Proactor

- **Reactor:** Synchronous I/O, notifies when ready
- **Proactor:** Asynchronous I/O, notifies when complete

Reactor vs Event Loop

- **Reactor:** Formal pattern with handlers
- **Event Loop:** Simpler, often used in JavaScript

Common Pitfalls

1. **Blocking Handlers:** Blocking in handler blocks reactor.

2. **Resource Leaks:** Not closing channels properly.

3. **Callback Hell:** Complex nested callbacks.

4. **Thread Safety:** Handlers may need synchronization.

5. **Error Handling:** Errors can leave reactor in bad state.

Best Practices

1. **Keep Handlers Non-blocking:** Use worker threads for long operations.

2. **Handle Errors Gracefully:** Don't crash reactor on handler errors.

3. **Use Timeouts:** Prevent hanging connections.

4. **Monitor Selector:** Watch for large select sets.

5. **Consider Hybrid Approaches:** Combine with thread pool for CPU work.

6. **Test with Many Connections:** Verify scalability.

[Page 27]

Chapter 25: Proactor Pattern

Detailed Explanation

The Proactor pattern handles asynchronous events by initiating asynchronous operations and then being notified upon their completion. It's the asynchronous counterpart to the Reactor pattern.

Understanding the Problem

While the Reactor pattern efficiently handles many connections, it still uses synchronous I/O operations. When a read operation is initiated, the thread must wait for data. This leads to:

1. **Thread Blocking:** Threads block on I/O
2. **Context Switching:** Switching between threads
3. **Complexity:** Managing read states

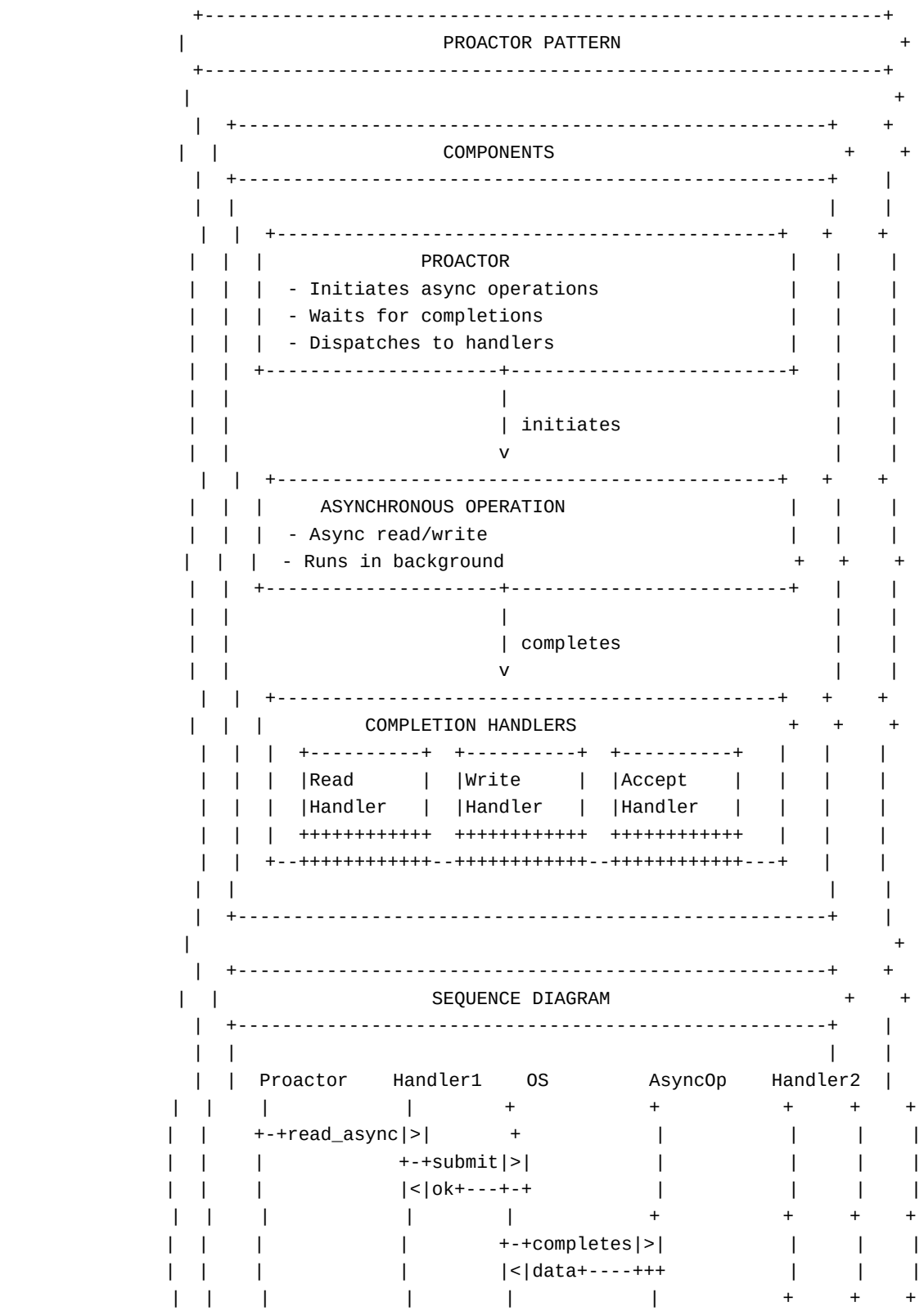
The Solution

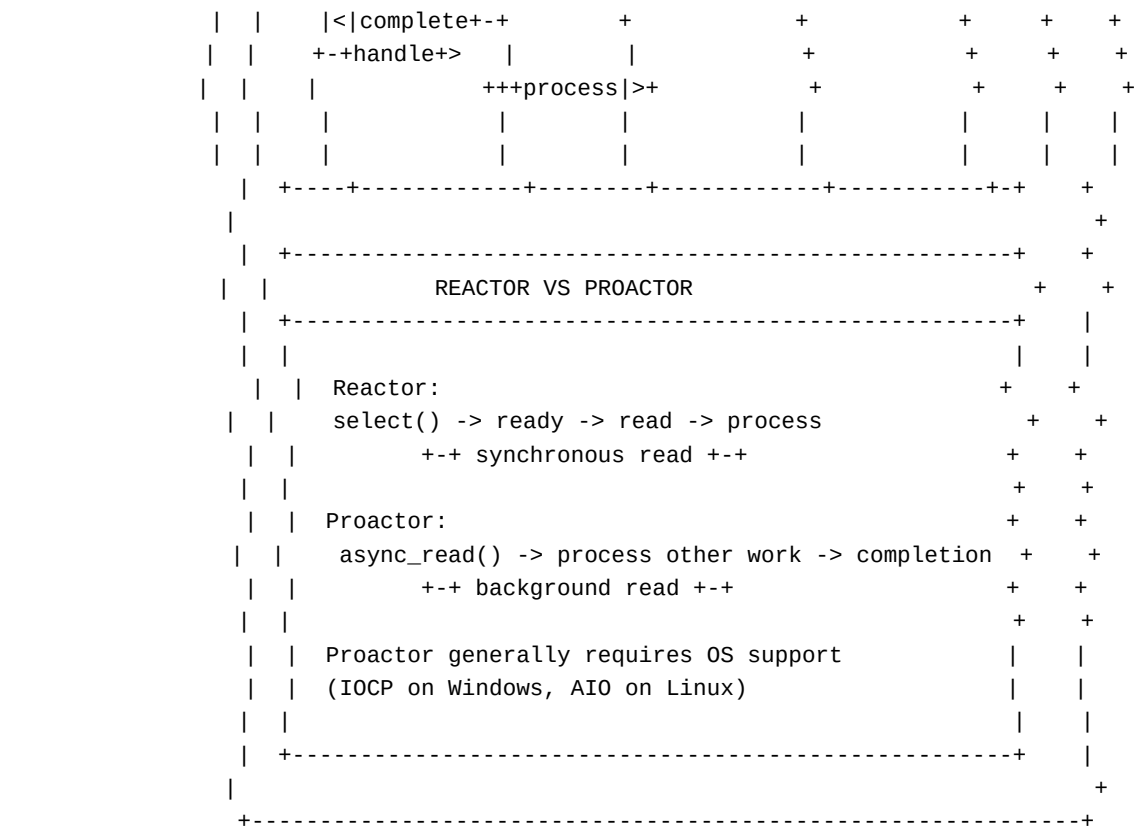
The Proactor pattern uses asynchronous I/O operations:

1. **Initiate Operation:** Start async read/write
2. **Continue Processing:** Thread doesn't block
3. **Completion Notification:** OS notifies when operation completes
4. **Completion Handler:** Processes completed operation

Key Concepts
- **Asynchronous Operation:** I/O operation that completes in background
- **Completion Handler:** Processes operation results
- **Proactor:** Initiates operations and dispatches completions
- **Completion Port:** Windows IOCP, Linux AIO, epoll with EPOLLET

Diagram: Proactor Pattern





Implementation Example (using Java AIO - NIO.2)

```
import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.*;
import java.util.concurrent.*;
// 1. Completion Handler Interface
interface CompletionHandler<V, A> {
void completed(V result, A attachment);
void failed(Throwable exc, A attachment);
}
// 2. Proactor Server
class ProactorServer {
private final AsynchronousServerSocketChannel serverChannel;
private final ExecutorService workerPool;

public ProactorServer(int port) throws IOException {
this.serverChannel = AsynchronousServerSocketChannel.open();
this.serverChannel.bind(new InetSocketAddress(port));
this.workerPool = Executors.newFixedThreadPool(10);
}

public void start() {
                        System.out.println("Proactor server started on port " +
serverChannel.getLocalAddress());

// Start accepting connections
acceptNext();

// Keep main thread alive
try {
Thread.currentThread().join();
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
```

```
}
}

private void acceptNext() {
serverChannel.accept(null, new CompletionHandler<AsynchronousSocketChannel, Void>() {
@Override
public void completed(AsynchronousSocketChannel clientChannel, Void attachment) {
// Accept next connection immediately
acceptNext();

// Handle this connection
handleConnection(clientChannel);
}

@Override
public void failed(Throwable exc, Void attachment) {
                                  System.err.println("Accept failed: " + exc.getMessage());
// Try again after delay
                                workerPool.schedule(() -> acceptNext(), 1, TimeUnit.SECONDS);
}
});
}

private void handleConnection(AsynchronousSocketChannel clientChannel) {
try {
                                       System.out.println("New connection from: " +
clientChannel.getRemoteAddress());
} catch (IOException e) {
e.printStackTrace();
}

// Create buffer for this connection
ByteBuffer buffer = ByteBuffer.allocate(1024);

// Start async read
readAsync(clientChannel, buffer);
}

private void readAsync(AsynchronousSocketChannel channel, ByteBuffer buffer) {
channel.read(buffer, buffer, new CompletionHandler<Integer, ByteBuffer>() {
@Override
public void completed(Integer bytesRead, ByteBuffer attachment) {
                                                  if (bytesRead == -1) {
// Connection closed
try {
channel.close();
System.out.println("Connection closed");
} catch (IOException e) {
e.printStackTrace();
}
return;
}

if (bytesRead > 0) {
attachment.flip();
byte[] data = new byte[attachment.limit()];
attachment.get(data);
String message = new String(data);
                                           System.out.println("Received: " + message);

// Echo back
attachment.rewind();
writeAsync(channel, attachment);
}
// Read next message
```

```
                attachment.clear();
                readAsync(channel, attachment);
            }

            @Override
            public void failed(Throwable exc, ByteBuffer attachment) {
                System.err.println("Read failed: " + exc.getMessage());
                try {
                    channel.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        });
    }

    private void writeAsync(AsynchronousSocketChannel channel, ByteBuffer buffer) {
        channel.write(buffer, buffer, new CompletionHandler<Integer, ByteBuffer>() {
            @Override
            public void completed(Integer bytesWritten, ByteBuffer attachment) {
                if (attachment.hasRemaining()) {
                    // Write remaining data
                    channel.write(attachment, attachment, this);
                }
            }

            @Override
            public void failed(Throwable exc, ByteBuffer attachment) {
                System.err.println("Write failed: " + exc.getMessage());
                try {
                    channel.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        });
    }

    public void shutdown() {
        try {
            serverChannel.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
        workerPool.shutdown();
    }

    public static void main(String[] args) throws IOException {
        ProactorServer server = new ProactorServer(8080);
        Runtime.getRuntime().addShutdownHook(new Thread(server::shutdown));
        server.start();
    }
}
```

Simplified Proactor with Futures

```
class SimpleProactor {
    private final AsynchronousChannelGroup channelGroup;

    public SimpleProactor() throws IOException {
        this.channelGroup = AsynchronousChannelGroup.withFixedThreadPool(
            10, Executors.defaultThreadFactory());
    }
```

```
        public Future<Integer> readAsync(AsynchronousSocketChannel channel,
                                         ByteBuffer buffer) {
            return channel.read(buffer);
        }

        public Future<Integer> writeAsync(AsynchronousSocketChannel channel,
                                          ByteBuffer buffer) {
            return channel.write(buffer);
        }

        public void processWithFuture(AsynchronousSocketChannel channel)
                throws Exception {
            ByteBuffer buffer = ByteBuffer.allocate(1024);

            // Start async read
            Future<Integer> readFuture = readAsync(channel, buffer);

            // Do other work while reading
            System.out.println("Reading in background...");

            // Wait for read to complete
            Integer bytesRead = readFuture.get(5, TimeUnit.SECONDS);

            if (bytesRead > 0) {
                buffer.flip();
                System.out.println("Read " + bytesRead + " bytes");

                // Echo back
                Future<Integer> writeFuture = writeAsync(channel, buffer);
                writeFuture.get(5, TimeUnit.SECONDS);
            }
        }
    }
```

When to Use Proactor Pattern

Suitable Situations:

1. When OS supports true asynchronous I/O (IOCP on Windows, AIO on Linux).

2. When you want maximum I/O scalability.

3. When you have many long-running I/O operations.

4. When you want to minimize thread usage.

5. In high-performance servers handling many concurrent connections.

Real-World Use Cases:

- **Windows Servers:** Using IOCP (I/O Completion Ports)
- **Database Systems:** Asynchronous query execution

- **File Servers:** Handling many file transfers
- **Proxy Servers:** Forwarding data between connections
- **High-Performance Computing:** Overlapping computation with I/O

Advantages of Proactor Pattern

1. **Maximum Concurrency:** Threads never block on I/O.

2. **Scalability:** Handles huge numbers of connections.

3. **Efficiency:** No thread blocking or context switching.

4. **Simplified Code:** No need to manage read states.

5. **Overlap:** Computation and I/O can overlap.

Disadvantages of Proactor Pattern

1. **OS Support:** Requires OS-level async I/O.

2. **Complexity:** More complex than reactor.

3. **Debugging:** Asynchronous flow harder to debug.

4. **Error Handling:** Errors reported asynchronously.

5. **Memory Management:** Buffers must stay alive until completion.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
Proactor	Async I/O demux	Notifies on completion
Reactor	Sync I/O demux	Notifies on readiness
Event Loop	Simple async	Usually sync I/O
AIO	OS async I/O	Underlying mechanism

Proactor vs Reactor

- **Proactor:** "Tell me when you're done"
- **Reactor:** "Tell me when you're ready"

Proactor vs AIO

- **Proactor:** Design pattern using AIO
- **AIO:** Operating system asynchronous I/O facility

Common Pitfalls

1. **Buffer Management:** Buffers must stay valid until completion.

2. **Completion Order:** Completions may come out of order.

3. **Resource Leaks:** Not releasing buffers after completion.

4. **Callback Hell:** Nested completion handlers.

5. **Thread Safety:** Completion handlers may run on arbitrary threads.

Best Practices

1. **Pool Buffers:** Reuse buffers to avoid allocation overhead.

2. **Handle Errors:** Always provide failure handlers.

3. **Limit Concurrency:** Don't initiate too many operations.

4. **Monitor Completion Queue:** Watch for backlog.

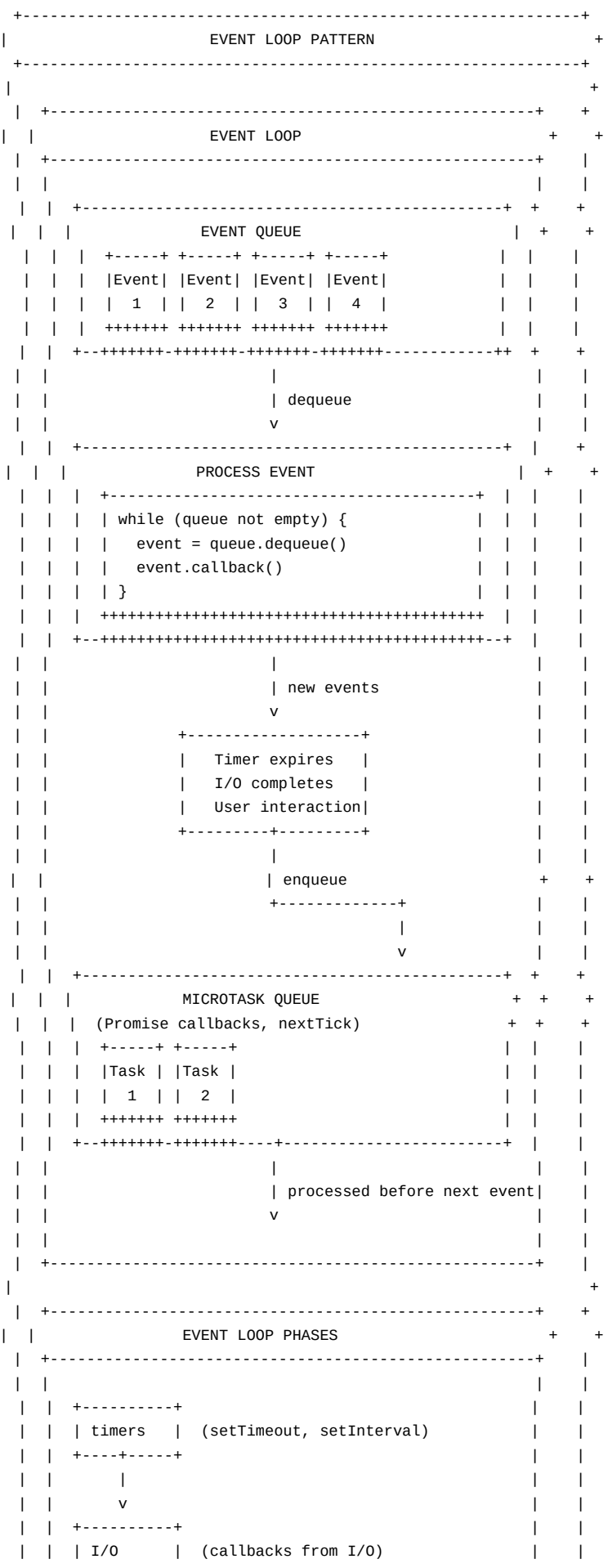
5. **Use Timeouts:** Prevent hanging operations.

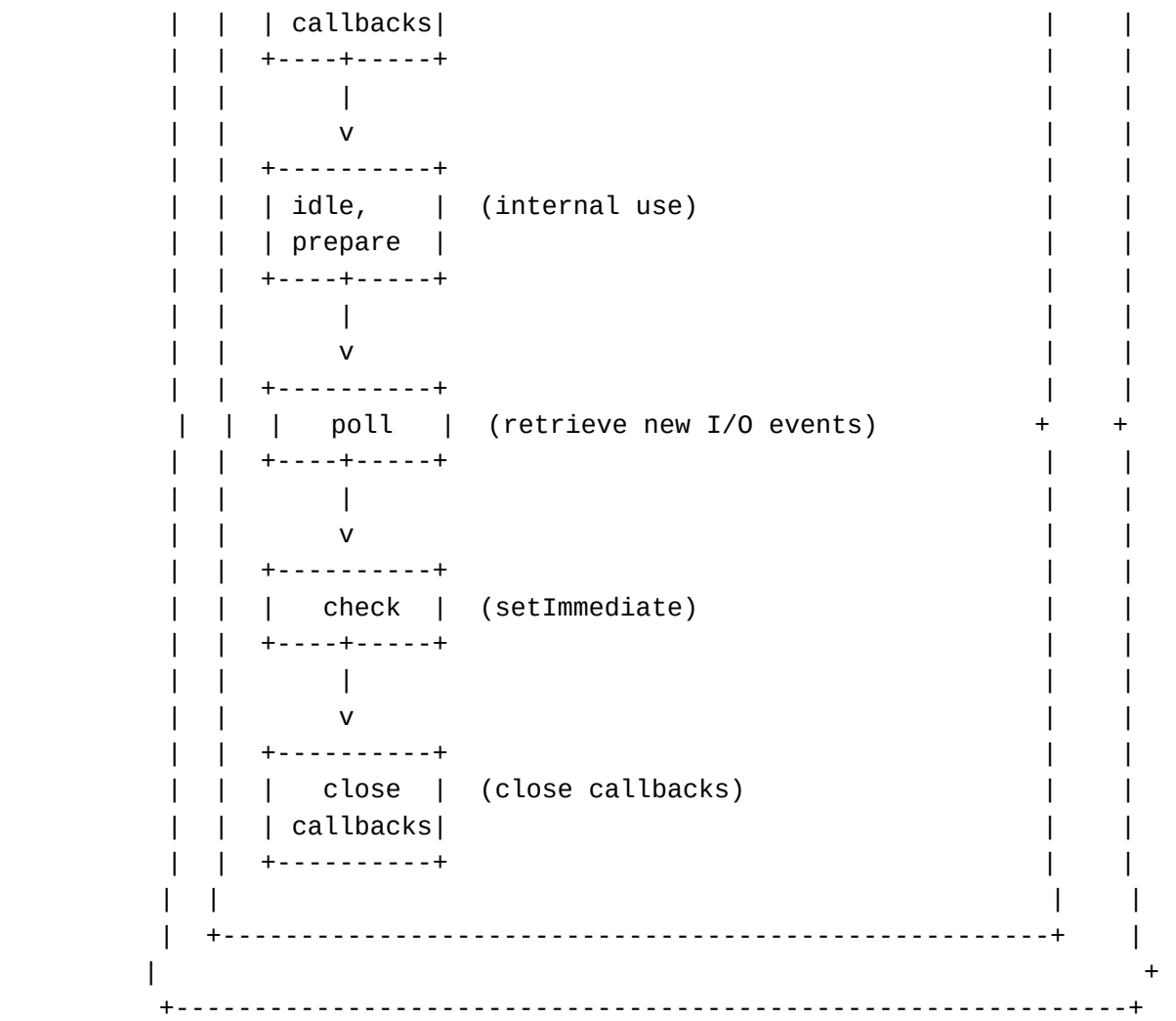
6. **Consider Hybrid:** Combine with thread pool for CPU work.

[Page 28]

Chapter 26: Event Loop Pattern

Detailed Explanation
The Event Loop pattern continuously waits for and dispatches events in a single thread. It's the foundation of many asynchronous systems, particularly in JavaScript/Node.js and GUI applications.
Understanding the Problem
Traditional threaded approaches have overhead:
1. **Thread Creation:** Expensive
2. **Context Switching:** CPU overhead
3. **Synchronization:** Complex locking
4. **Memory:** Each thread needs stack
For I/O-bound applications, much time is spent waiting. Why have many threads waiting when one thread can handle all waiting?
The Solution
The Event Loop pattern uses a single thread that:
1. **Waits** for events (I/O, timers, callbacks)
2. **Processes** events one at a time
3. **Queues** new events during processing
4. **Repeats** forever
Key Concepts
- **Event Queue:** Holds pending events
- **Event Loop:** Continuously checks queue and processes events
- **Callbacks:** Functions to execute when events occur
- **Non-blocking I/O:** Operations don't block the loop
- **Microtask Queue:** High-priority queue for promises
Diagram: Event Loop


EVENT LOOP PATTERN
EVENT LOOP
EVENT QUEUE
Event 1
Event 2
Event 3
Event 4
dequeue
PROCESS EVENT
while (queue not empty) {
event = queue.dequeue()
event.callback()
}
new events
Timer expires
I/O completes
User interaction
enqueue
MICROTASK QUEUE
(Promise callbacks, nextTick)
Task 1
Task 2
processed before next event
EVENT LOOP PHASES
timers
(setTimeout, setInterval)
I/O
(callbacks from I/O)



Implementation Examples

1. JavaScript/Node.js Event Loop

```
// JavaScript's event loop is built-in
console.log('Start');
setTimeout(() => {
console.log('Timeout 1');
}, 0);
Promise.resolve().then(() => {
console.log('Promise 1');
});
setTimeout(() => {
console.log('Timeout 2');
}, 0);
console.log('End');
// Output:
// Start
// End
// Promise 1
// Timeout 1
// Timeout 2
```

2. Simple Event Loop in Java

```
import java.util.concurrent.*;
```

```
import java.util.*;
public class SimpleEventLoop {
private final BlockingQueue<Runnable> taskQueue =
new LinkedBlockingQueue<>();
private final BlockingQueue<Runnable> microTaskQueue =
new LinkedBlockingQueue<>();
private volatile boolean running = true;

public void post(Runnable task) {
taskQueue.offer(task);
}

public void postMicro(Runnable task) {
microTaskQueue.offer(task);
}

public void run() {
while (running) {
try {
// Process all microtasks first
while (!microTaskQueue.isEmpty()) {
Runnable microTask = microTaskQueue.poll();
if (microTask != null) {
microTask.run();
}
}

// Take next task (blocks if none)
Runnable task = taskQueue.poll(100, TimeUnit.MILLISECONDS);
if (task != null) {
task.run();
}

// Check timers, I/O, etc. (simplified)
checkTimers();

} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}

public void stop() {
running = false;
}

private void checkTimers() {
// Check for expired timers and add them to queue
                            // Simplified - actual implementation would track timers
}

// Timer support
private final Map<Long, Runnable> timers = new ConcurrentHashMap<>();
private final ScheduledExecutorService timerExecutor =
Executors.newSingleThreadScheduledExecutor();

public void setTimeout(Runnable task, long delay, TimeUnit unit) {
long id = System.nanoTime();
timers.put(id, task);

                                        timerExecutor.schedule(() -> {
timers.remove(id);
post(task);
```

```
}, delay, unit);
}

public static void main(String[] args) {
SimpleEventLoop eventLoop = new SimpleEventLoop();

// Start event loop in separate thread
                    Thread loopThread = new Thread(() -> eventLoop.run());
loopThread.start();

// Post some tasks
                      eventLoop.post(() -> System.out.println("Task 1"));
                  eventLoop.postMicro(() -> System.out.println("Microtask 1"));
                      eventLoop.post(() -> System.out.println("Task 2"));

                                    eventLoop.setTimeout(() ->
System.out.println("Timeout"), 1, TimeUnit.SECONDS);

// Let it run for a while
try {
Thread.sleep(2000);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}

eventLoop.stop();
loopThread.interrupt();
}
}
**3. Generic Event Loop with Handlers**
import java.util.*;
import java.util.concurrent.*;
interface Event {
String getType();
Object getData();
}
interface EventHandler {
void handle(Event event);
}
class EventLoop {
private final BlockingQueue<Event> eventQueue =
new LinkedBlockingQueue<>();
private final Map<String, List<EventHandler>> handlers =
new ConcurrentHashMap<>();
private final Map<Long, Event> timerEvents = new ConcurrentHashMap<>();
private volatile boolean running = true;

public void registerHandler(String eventType, EventHandler handler) {
            handlers.computeIfAbsent(eventType, k -> new CopyOnWriteArrayList<>())
.add(handler);
}

public void post(Event event) {
eventQueue.offer(event);
}

public void postDelayed(Event event, long delay, TimeUnit unit) {
long id = System.nanoTime();
```

```
timerEvents.put(id, event);

ScheduledExecutorService executor = Executors.newSingleThreadScheduledExecutor();
                                                executor.schedule(() -> {
timerEvents.remove(id);
post(event);
executor.shutdown();
}, delay, unit);
}

public void run() {
while (running) {
try {
Event event = eventQueue.poll(100, TimeUnit.MILLISECONDS);
if (event != null) {
dispatch(event);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}

private void dispatch(Event event) {
List<EventHandler> eventHandlers = handlers.get(event.getType());
if (eventHandlers != null) {
for (EventHandler handler : eventHandlers) {
handler.handle(event);
}
}
}

public void stop() {
running = false;
}
}
// Example usage
class EventLoopExample {
public static void main(String[] args) {
EventLoop loop = new EventLoop();

// Register handlers
                                    loop.registerHandler("USER_LOGIN", event -> {
                                System.out.println("User logged in: " + event.getData());
});

                                    loop.registerHandler("DATA_UPDATE", event -> {
                                 System.out.println("Data updated: " + event.getData());
});

// Start loop
new Thread(loop::run).start();

// Post events
loop.post(new Event() {
                                     public String getType() { return "USER_LOGIN"; }
public Object getData() { return "john@example.com"; }
});

loop.postDelayed(new Event() {
                                    public String getType() { return "DATA_UPDATE"; }
public Object getData() { return "New data available"; }
}, 1, TimeUnit.SECONDS);
```

```
            // Let it run
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {}

            loop.stop();
        }
    }
```

When to Use Event Loop Pattern

Suitable Situations:

1. In I/O-bound applications (web servers, proxies).

2. In GUI applications (event-driven UI).

3. When you want simplicity of single-threaded programming.

4. When you have many concurrent connections.

5. In JavaScript/Node.js environments (it's built-in).

Real-World Use Cases:

- **Node.js:** JavaScript runtime
- **Web Browsers:** JavaScript event loop
- **GUI Frameworks:** Swing, JavaFX event dispatch thread
- **Game Engines:** Update loops
- **Redis:** Single-threaded event-driven server

Advantages of Event Loop Pattern

1. **Simplicity:** Single thread, no concurrency issues.

2. **No Race Conditions:** No locks needed.

3. **Predictable:** Events processed in order.

4. **Lightweight:** No thread overhead.

5. **Good for I/O:** Handles many connections efficiently.

Disadvantages of Event Loop Pattern

1. **CPU-Bound Tasks:** Block the loop.

2. **Single Core Only:** Can't utilize multiple cores.

3. **Callback Hell:** Nested callbacks become complex.

4. **Error Handling:** Errors can crash the loop.

5. **No Preemption:** Long-running tasks starve others.

Comparison with Other Patterns

Pattern	Purpose	Difference
Event Loop	Single-thread event processing	No parallelism
Reactor	Sync event demux	Can use thread pool
Proactor	Async event demux	OS async I/O
Actor	Message passing	Multiple actors

Event Loop vs Reactor

- **Event Loop:** Usually single-threaded
- **Reactor:** Can use thread pool for handlers

Event Loop vs Actor Model

- **Event Loop:** Single queue, single thread
- **Actor Model:** Multiple actors, each with own queue

Common Pitfalls

1. **Blocking Operations:** Never block the event loop.

2. **Too Many Callbacks:** Leads to callback hell.

3. **Error Propagation:** Errors must be caught.

4. **Zalgo Problem:** Mixing sync and async code.

5. **Memory Leaks:** Forgotten event listeners.

Best Practices

1. **Never Block:** Use async operations only.

2. **Keep Callbacks Small:** Break into smaller functions.

3. **Use Promises/Async-Await:** Avoid callback hell.

4. **Handle Errors:** Always have error handlers.

5. **Remove Listeners:** Clean up event listeners.

6. **Monitor Lag:** Watch for event loop lag.

7. **Use Worker Threads:** Offload CPU work.

[Page 29]

Chapter 27: Publish-Subscribe Pattern

Detailed Explanation

The Publish-Subscribe pattern decouples publishers (senders) from subscribers (receivers) through an intermediary message broker. Publishers emit messages without knowing who will receive them; subscribers express interest in certain messages without knowing who sent them.

Understanding the Problem

In distributed systems, direct communication between components creates tight coupling:

1. **Dependency:** Senders need to know receivers
2. **Scalability:** Hard to add new receivers
3. **Flexibility:** Changing receivers affects senders
4. **Reliability:** If receiver fails, sender may need to handle

The Solution

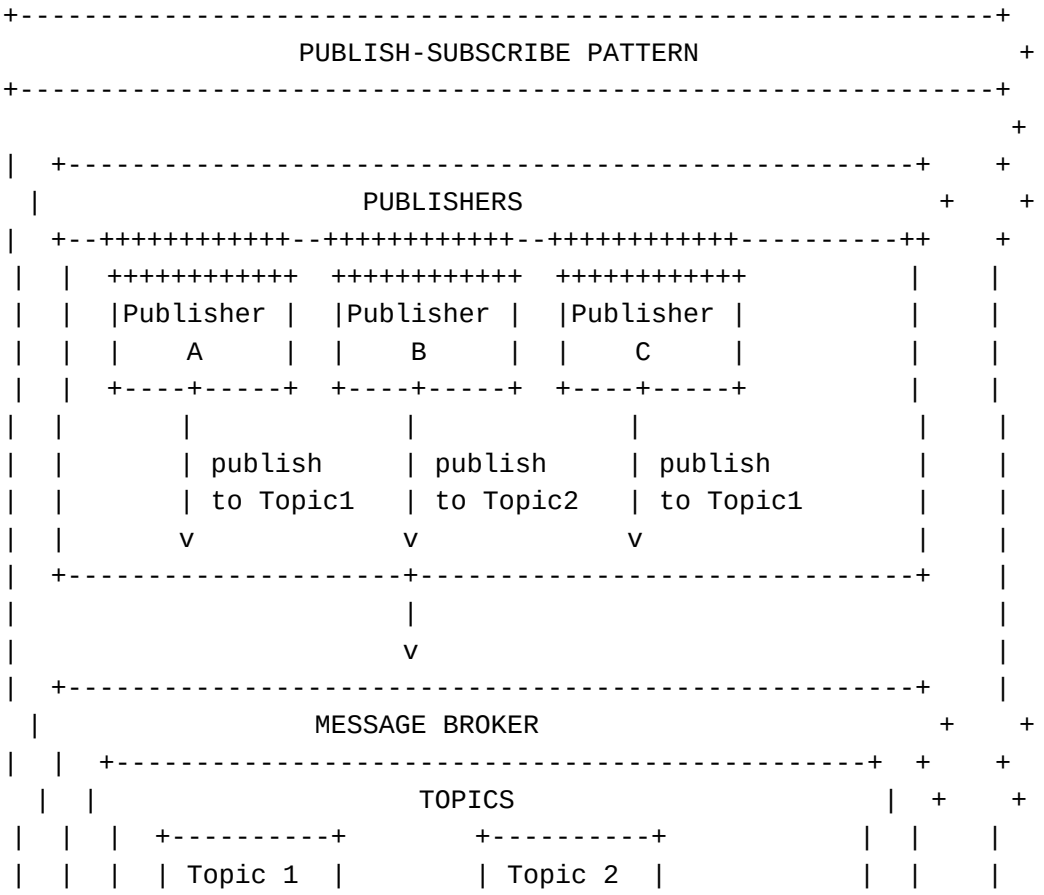
The Publish-Subscribe pattern introduces a message broker:

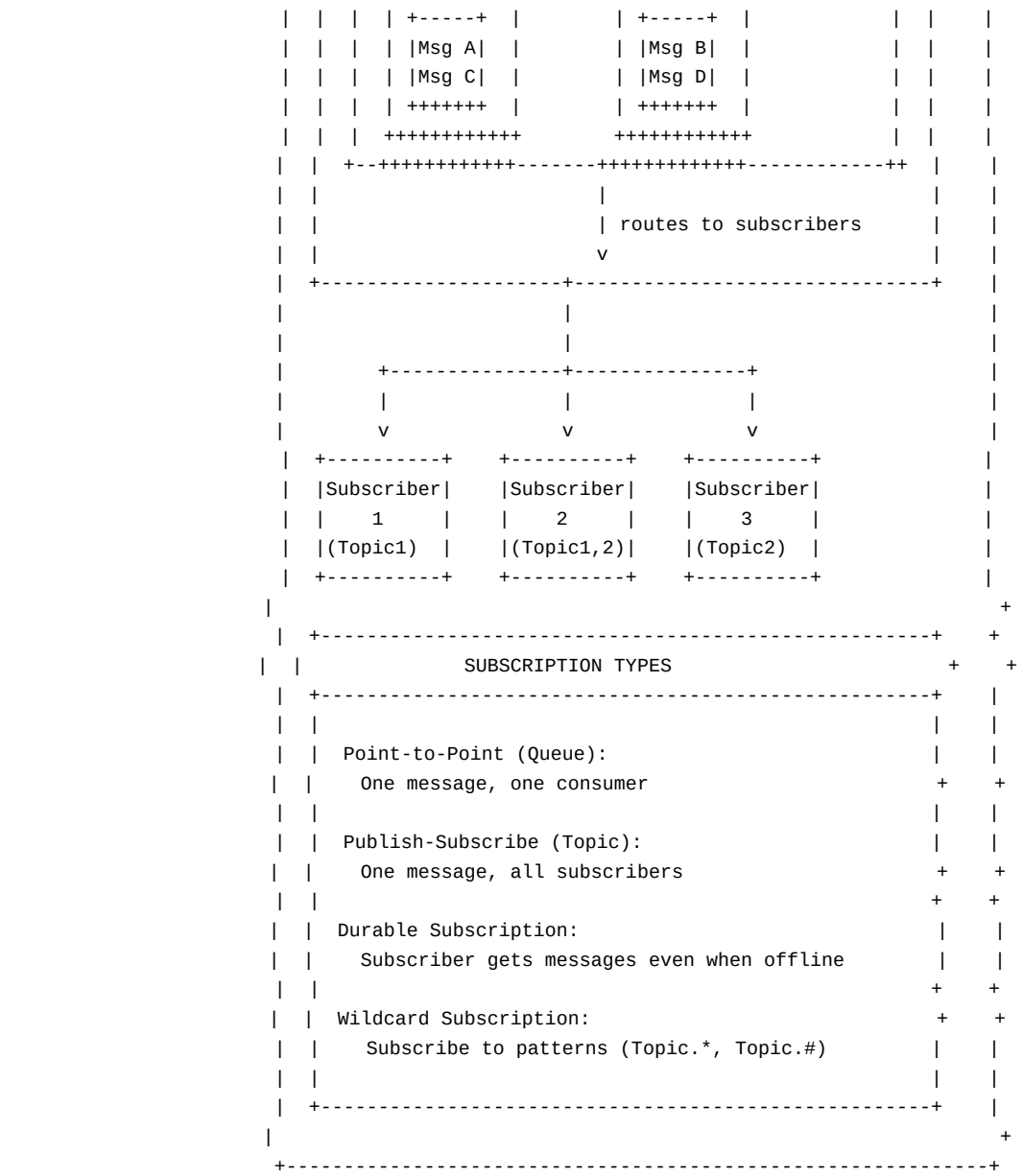
1. **Publishers:** Send messages to topics
2. **Subscribers:** Subscribe to topics of interest
3. **Broker:** Routes messages to all subscribers

Key Concepts

- **Publisher:** Produces messages
- **Subscriber:** Consumes messages
- **Topic/Channel:** Named destination for messages
- **Broker:** Routes messages to subscribers
- **Subscription:** Expression of interest in a topic

Diagram: Publish-Subscribe





Implementation Example

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
// 1. Message class
class Message {
private final String topic;
private final Object payload;
private final long timestamp;

public Message(String topic, Object payload) {
this.topic = topic;
this.payload = payload;
this.timestamp = System.currentTimeMillis();
}

public String getTopic() { return topic; }
public Object getPayload() { return payload; }
public long getTimestamp() { return timestamp; }
}
```

```
// 2. Subscriber interface
interface Subscriber {
void onMessage(Message message);
String getId();
}
// 3. Simple In-Memory PubSub Broker
class PubSubBroker {
private final Map<String, List<Subscriber>> topics =
new ConcurrentHashMap<>();
private final Map<String, BlockingQueue<Message>> topicQueues =
new ConcurrentHashMap<>();
private final ExecutorService executor =
Executors.newCachedThreadPool();

public void subscribe(String topic, Subscriber subscriber) {
                            topics.computeIfAbsent(topic, k -> new CopyOnWriteArrayList<>())
.add(subscriber);
                                System.out.println("Subscriber " + subscriber.getId() +
                                                " subscribed to " + topic);
}

public void unsubscribe(String topic, Subscriber subscriber) {
List<Subscriber> subscribers = topics.get(topic);
if (subscribers != null) {
subscribers.remove(subscriber);
                                  System.out.println("Subscriber " + subscriber.getId() +
                                                " unsubscribed from " + topic);
}
}

public void publish(String topic, Object payload) {
Message message = new Message(topic, payload);
publish(message);
}

public void publish(Message message) {
List<Subscriber> subscribers = topics.get(message.getTopic());

if (subscribers != null && !subscribers.isEmpty()) {
for (Subscriber subscriber : subscribers) {
// Deliver asynchronously
                                                      executor.submit(() -> {
try {
subscriber.onMessage(message);
} catch (Exception e) {
                                                 System.err.println("Error delivering to " +
                                                  subscriber.getId() + ": " + e.getMessage());
}
});
}
} else {
                                     System.out.println("No subscribers for topic: " +
message.getTopic());
}
}

public void shutdown() {
executor.shutdown();
try {
if (!executor.awaitTermination(5, TimeUnit.SECONDS)) {
executor.shutdownNow();
}
} catch (InterruptedException e) {
executor.shutdownNow();
```

```
        }
    }
}

// 4. Example Subscribers
class LoggingSubscriber implements Subscriber {
    private final String id;

    public LoggingSubscriber(String id) {
        this.id = id;
    }

    @Override
    public void onMessage(Message message) {
        System.out.println("[" + id + "] Received: " +
            message.getPayload() + " on topic " + message.getTopic());
    }

    @Override
    public String getId() { return id; }
}

class CountingSubscriber implements Subscriber {
    private final String id;
    private final AtomicInteger count = new AtomicInteger(0);

    public CountingSubscriber(String id) {
        this.id = id;
    }

    @Override
    public void onMessage(Message message) {
        int total = count.incrementAndGet();
        System.out.println("[" + id + "] Message count: " + total);
    }

    @Override
    public String getId() { return id; }
}

// 5. Pattern Matching Topic (Wildcard)
class PatternPubSub {
    private final Map<String, List<Subscriber>> exactTopics =
        new ConcurrentHashMap<>();
    private final List<PatternSubscription> patternSubscriptions =
        new CopyOnWriteArrayList<>();

    static class PatternSubscription {
        final String pattern;
        final Subscriber subscriber;

        PatternSubscription(String pattern, Subscriber subscriber) {
            this.pattern = pattern;
            this.subscriber = subscriber;
        }

        boolean matches(String topic) {
            // Simple wildcard: * matches one level, # matches multiple
            if (pattern.endsWith("#")) {
                String prefix = pattern.substring(0, pattern.length() - 1);
                return topic.startsWith(prefix);
            } else if (pattern.contains("*")) {
                // Simplified - in practice, use regex
                return topic.matches(pattern.replace("*", "[^.]+"));
            }
```

```
return pattern.equals(topic);
}
}

public void subscribe(String topic, Subscriber subscriber) {
                                if (topic.contains("*") || topic.contains("#")) {
patternSubscriptions.add(
new PatternSubscription(topic, subscriber));
} else {
exactTopics.computeIfAbsent(topic,
                                   k -> new CopyOnWriteArrayList<>()).add(subscriber);
}
}

public void publish(String topic, Object payload) {
// Send to exact matches
List<Subscriber> exact = exactTopics.get(topic);
if (exact != null) {
for (Subscriber s : exact) {
s.onMessage(new Message(topic, payload));
}
}

// Send to pattern matches
for (PatternSubscription ps : patternSubscriptions) {
if (ps.matches(topic)) {
ps.subscriber.onMessage(new Message(topic, payload));
}
}
}
}
// 6. Usage Example
public class PubSubExample {
public static void main(String[] args) throws InterruptedException {
PubSubBroker broker = new PubSubBroker();

// Create subscribers
Subscriber logger1 = new LoggingSubscriber("Logger1");
Subscriber logger2 = new LoggingSubscriber("Logger2");
Subscriber counter = new CountingSubscriber("Counter");

// Subscribe to topics
broker.subscribe("user.login", logger1);
broker.subscribe("user.logout", logger1);
broker.subscribe("user.login", logger2);
broker.subscribe("system.status", counter);
broker.subscribe("user.*", counter); // Wildcard example

// Publish messages
broker.publish("user.login", "User john logged in");
broker.publish("user.logout", "User john logged out");
broker.publish("system.status", "All systems operational");
broker.publish("database.error", "Connection failed");

// Wait for async delivery
Thread.sleep(1000);

broker.shutdown();
}
}
```

Distributed PubSub with Message Queue

```
// Simplified JMS-like interface
interface MessageQueue {
void publish(String topic, Message message);
void subscribe(String topic, MessageListener listener);
}
interface MessageListener {
void onMessage(Message message);
}
class DistributedPubSub implements MessageQueue {
private final Map<String, List<MessageListener>> listeners =
new ConcurrentHashMap<>();
private final ExecutorService executor =
Executors.newFixedThreadPool(10);
private final String brokerId;

public DistributedPubSub(String brokerId) {
this.brokerId = brokerId;
}

@Override
public void publish(String topic, Message message) {
// In distributed system, this would publish to a cluster
                                    executor.submit(() -> {
List<MessageListener> topicListeners = listeners.get(topic);
if (topicListeners != null) {
for (MessageListener listener : topicListeners) {
listener.onMessage(message);
}
}
});
}

@Override
public void subscribe(String topic, MessageListener listener) {
listeners.computeIfAbsent(topic,
                        k -> new CopyOnWriteArrayList<>()).add(listener);
}

// Cluster coordination
public void connectToBroker(String remoteBroker) {
// In real implementation, establish connection
            System.out.println("Connected to remote broker: " + remoteBroker);
}
}
```

When to Use Publish-Subscribe Pattern

Suitable Situations:

1. When you need to decouple message producers from consumers.
2. When multiple consumers need to receive the same messages.
3. When you don't know who will need the messages.

4. When the system needs to be extensible (add new consumers easily).

5. In event-driven architectures.

Real-World Use Cases:

- **Message Queues:** RabbitMQ, Apache Kafka, ActiveMQ
- **Event-Driven Microservices:** Service communication
- **Notification Systems:** Push notifications
- **Log Aggregation:** Centralized logging
- **Real-Time Data Feeds:** Stock tickers, news feeds

Advantages of Publish-Subscribe Pattern

1. **Loose Coupling:** Publishers and subscribers don't know each other.

2. **Scalability:** Can add many subscribers.

3. **Flexibility:** Subscribers can come and go.

4. **Asynchronous:** Publishers don't wait for subscribers.

5. **Multiple Receivers:** One message, many subscribers.

Disadvantages of Publish-Subscribe Pattern

1. **Complexity:** Broker adds complexity.

2. **Delivery Guarantees:** At-most-once, at-least-once, exactly-once.

3. **Ordering:** Messages may arrive out of order.

4. **Broker Bottleneck:** Broker can become bottleneck.

5. **Debugging Difficulty:** Harder to trace message flow.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Pub-Sub	Many-to-many	Broker-mediated
Observer	One-to-many	Subject knows observers
Producer-Consumer	Point-to-point	One consumer per message
Message Queue	Queue-based	Usually point-to-point

Pub-Sub vs Observer

- **Pub-Sub:** Publishers don't know subscribers
- **Observer:** Subject knows about observers

Pub-Sub vs Message Queue

- **Pub-Sub:** Each message to all subscribers
- **Message Queue:** Each message to one consumer

Common Pitfalls

1. **Message Loss:** No persistence, subscribers offline.

2. **Ordering Issues:** Messages arrive out of order.

3. **Broker Failure:** Single point of failure.

4. **Performance Bottleneck:** Broker can't keep up.

5. **Debugging Complexity:** Hard to trace messages.

Best Practices

1. **Use Durable Subscriptions:** For important messages.

2. **Implement Retry Logic:** Handle temporary failures.

3. **Monitor Queue Depth:** Alert on backlog.

4. **Design for Idempotency:** Handle duplicate messages.

5. **Use Message Schemas:** Ensure compatibility.

6. **Consider Partitioning:** For high throughput.

7. **Implement Dead Letter Queues:** For failed messages.

[Page 30]

Chapter 28: Half-Sync / Half-Async Pattern

Detailed Explanation

The Half-Sync/Half-Async pattern decouples synchronous and asynchronous processing in concurrent systems. It uses two layers: an asynchronous layer for I/O and a synchronous layer for processing, with a queue between them.

Understanding the Problem

Many systems have both asynchronous I/O requirements and synchronous processing needs:

1. **Asynchronous I/O:** Efficient for many connections
2. **Synchronous Processing:** Simpler for business logic
3. **Integration:** Need to bridge the two models

The Solution

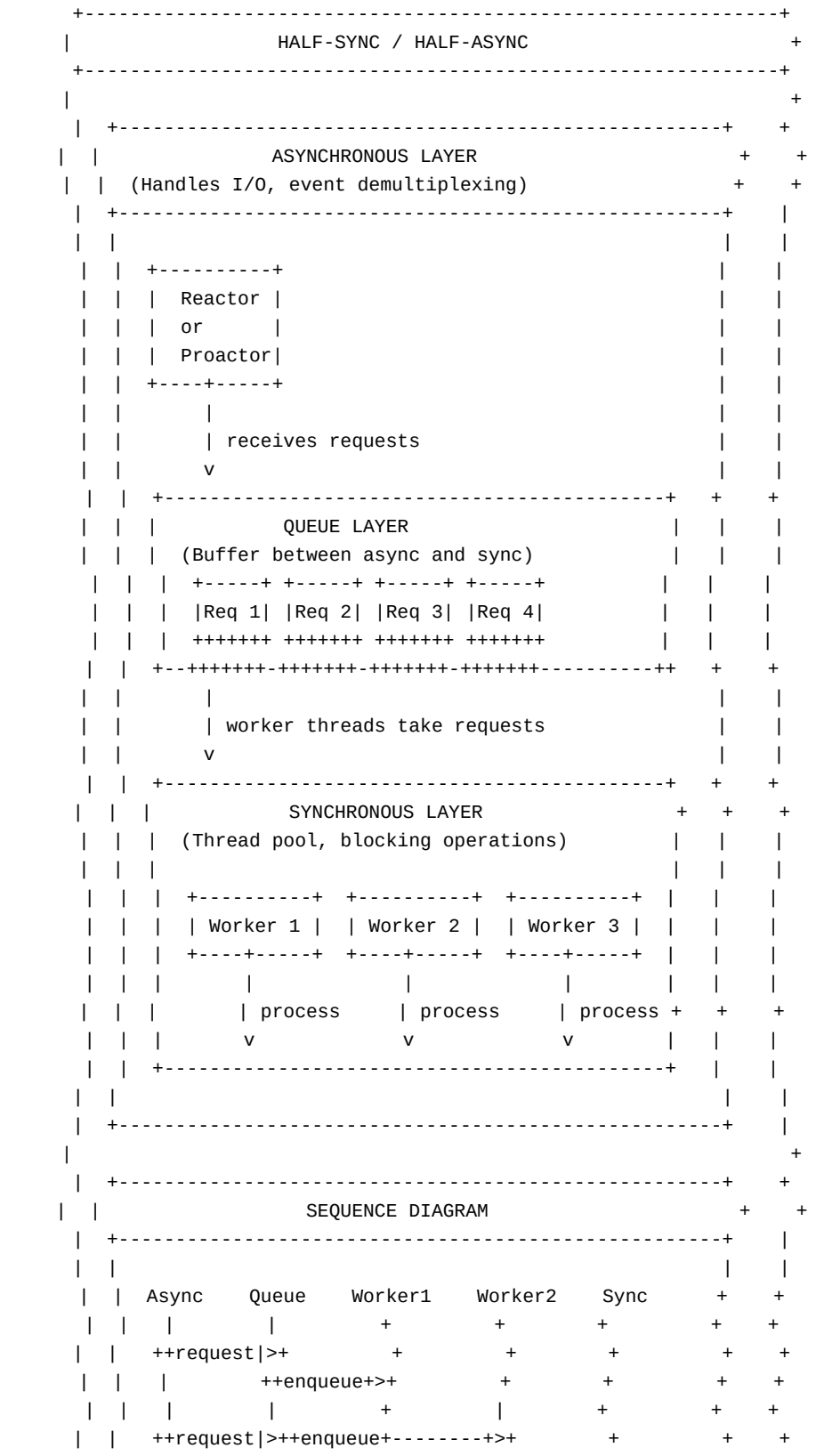
The Half-Sync/Half-Async pattern provides:

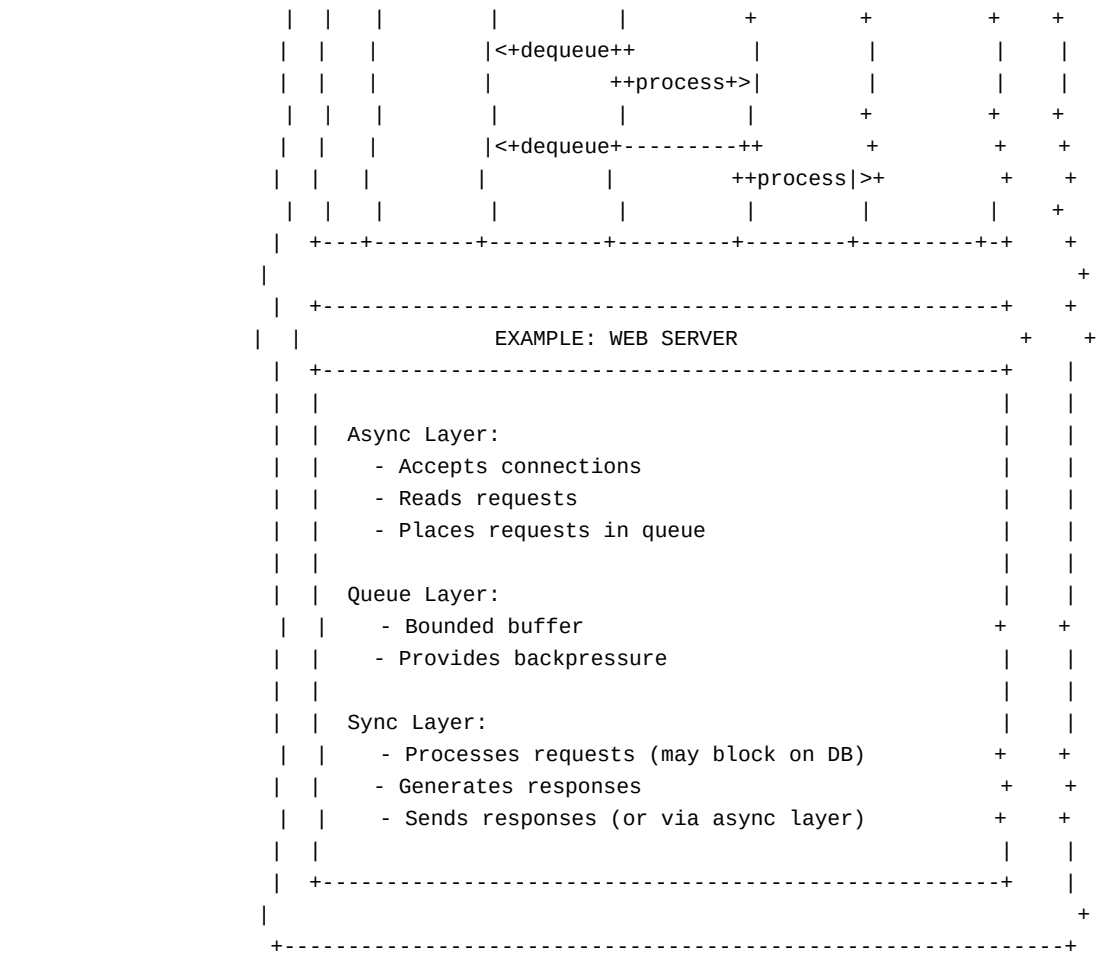
1. **Async Layer:** Handles I/O asynchronously
2. **Queue:** Buffers requests between layers
3. **Sync Layer:** Processes requests synchronously in thread pool
4. **Service Layer:** Interface to applications

Key Concepts

- **Async Layer:** Manages I/O, collects requests
- **Queue:** Holds pending requests
- **Sync Layer:** Thread pool processing requests
- **Demultiplexer:** Dispatches I/O events
- **Dispatcher:** Routes completed requests

Diagram: Half-Sync/Half-Async





Implementation Example

```
import java.util.concurrent.*;
import java.util.*;
// 1. Request/Response classes
class Request {
private final String id;
private final String data;
private final CompletableFuture<Response> future = new CompletableFuture<>();

public Request(String id, String data) {
this.id = id;
this.data = data;
}

public String getId() { return id; }
public String getData() { return data; }

public void complete(Response response) {
future.complete(response);
}

public Response getResponse() throws ExecutionException, InterruptedException {
return future.get();
}
}
class Response {
private final String requestId;
private final String result;
public Response(String requestId, String result) {
```

```
this.requestId = requestId;
this.result = result;
}

public String getRequestId() { return requestId; }
public String getResult() { return result; }
}
// 2. Async Layer (Reactor-like)
class AsyncLayer {
private final BlockingQueue<Request> requestQueue;
private volatile boolean running = true;

public AsyncLayer(BlockingQueue<Request> queue) {
this.requestQueue = queue;
}

public void start() {
// Simulate receiving requests
                                        Thread acceptor = new Thread(() -> {
int requestId = 0;
while (running) {
try {
// Simulate incoming request
                                         Request req = new Request("req-" + requestId++,
                                                       "Data-" + requestId);
                                      System.out.println("Async: Received " + req.getId());
requestQueue.put(req);

Thread.sleep(100); // Simulate request arrival rate
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
});
acceptor.setDaemon(true);
acceptor.start();
}

public void stop() {
running = false;
}
}
// 3. Synchronous Layer (Thread Pool)
class SyncLayer {
private final ExecutorService executor;
private final BlockingQueue<Request> requestQueue;
private volatile boolean running = true;

public SyncLayer(int threadCount, BlockingQueue<Request> queue) {
this.executor = Executors.newFixedThreadPool(threadCount);
this.requestQueue = queue;
}

public void start() {
                    for (int i = 0; i < Runtime.getRuntime().availableProcessors(); i++) {
                                        executor.submit(() -> {
while (running) {
try {
// Take request from queue (blocks if empty)
Request request = requestQueue.take();

// Process request synchronously (may block)
```

```
                        System.out.println("Sync: Processing " +
                            request.getId() + " on " +
                            Thread.currentThread().getName());

                        // Simulate processing (could be database, computation)
                        Thread.sleep(500);

                        // Create response
                        Response response = new Response(
                            request.getId(),
                            "Processed: " + request.getData()
                        );

                        // Complete the request's future
                        request.complete(response);

                        System.out.println("Sync: Completed " +
                            request.getId());

                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        break;
                    }
                }
            });
        }
    }

    public void stop() {
        running = false;
        executor.shutdown();
        try {
            executor.awaitTermination(5, TimeUnit.SECONDS);
        } catch (InterruptedException e) {
            executor.shutdownNow();
        }
    }
}

// 4. Response Handler (could be async or sync)
class ResponseHandler {
    public void handleResponse(Response response) {
        System.out.println("Response for " + response.getRequestId() +
            ": " + response.getResult());
    }
}

// 5. Complete Half-Sync/Half-Async System
class HalfSyncHalfAsyncSystem {
    private final BlockingQueue<Request> queue =
        new LinkedBlockingQueue<>(100); // Bounded queue
    private final AsyncLayer asyncLayer;
    private final SyncLayer syncLayer;
    private final ResponseHandler responseHandler;

    public HalfSyncHalfAsyncSystem(int threadCount) {
        this.asyncLayer = new AsyncLayer(queue);
        this.syncLayer = new SyncLayer(threadCount, queue);
        this.responseHandler = new ResponseHandler();
    }

    public void start() {
        asyncLayer.start();
        syncLayer.start();
        // Start response processing thread
```

```
                                        new Thread(() -> {
while (true) {
// In real system, responses would be collected differently
try {
Thread.sleep(1000);
} catch (InterruptedException e) {
break;
}
}
}).start();
}

public void stop() {
asyncLayer.stop();
syncLayer.stop();
}

// Client code to submit requests and wait for responses
public CompletableFuture<Response> submitRequest(String data) {
Request request = new Request(UUID.randomUUID().toString(), data);
try {
queue.put(request);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
CompletableFuture<Response> failed = new CompletableFuture<>();
failed.completeExceptionally(e);
return failed;
}
return request.future;
}

public static void main(String[] args) throws Exception {
HalfSyncHalfAsyncSystem system = new HalfSyncHalfAsyncSystem(4);
system.start();

// Submit some requests and wait for responses
                                    for (int i = 0; i < 10; i++) {
final int id = i;
                                     system.submitRequest("Data-" + i)
                                           .thenAccept(response -> {
                                      System.out.println("Got response for " +
response.getRequestId());
});
}

// Let system run
Thread.sleep(10000);
system.stop();
}
}
```

When to Use Half-Sync/Half-Async Pattern

Suitable Situations:

1. When you have both I/O-intensive and CPU-intensive operations.
2. When you want to separate concerns (I/O from processing).

3. When you need to scale I/O and processing independently.

4. In high-performance network servers.

5. When you need to use blocking APIs in an asynchronous system.

Real-World Use Cases:

- **Web Servers:** Accept connections async, process requests in thread pool
- **Database Drivers:** Async network, sync query execution
- **Gateway Systems:** Async I/O, sync processing
- **API Gateways:** Async request handling, sync service calls

Advantages of Half-Sync/Half-Async Pattern

1. **Separation of Concerns:** I/O and processing layers decoupled.

2. **Independent Scaling:** Each layer can be scaled separately.

3. **Simplicity:** Processing layer can use blocking APIs.

4. **Performance:** Combines async I/O efficiency with sync processing simplicity.

5. **Flexibility:** Can use different threading models per layer.

Disadvantages of Half-Sync/Half-Async Pattern

1. **Queue Management:** Queue sizing and overflow handling.

2. **Backpressure:** Must handle when queue fills.

3. **Complexity:** More complex than pure sync or async.

4. **Latency:** Queue adds latency.

5. **Debugging:** Harder to trace requests across layers.

Comparison with Other Patterns

Pattern	Purpose	Difference
Half-Sync/Half-Async	Bridge sync/async	Two-layer
Reactor	Sync event demux	Single layer
Proactor	Async event demux	Single layer
Leader-Followers	Thread pool	Different model

Half-Sync/Half-Async vs Reactor

- **Half-Sync/Half-Async:** Has processing layer separate from I/O
- **Reactor:** Handles processing in same thread or dispatches

Half-Sync/Half-Async vs Thread Pool

- **Half-Sync/Half-Async:** Has async I/O layer
- **Thread Pool:** May not have dedicated I/O layer

Common Pitfalls

1. **Queue Overflow:** Unbounded queues cause memory issues.

2. **Backpressure Not Implemented:** Producers overwhelm queue.

3. **Thread Pool Sizing:** Too few threads cause queuing.

4. **Response Path:** Sending responses back may need async handling.

5. **Error Propagation:** Errors across layers must be handled.

Best Practices

1. **Use Bounded Queue:** Prevent memory exhaustion.

2. **Implement Backpressure:** Slow producers when queue full.

3. **Monitor Queue Size:** Alert on high queue depth.

4. **Size Thread Pool Appropriately:** Based on workload.

5. **Handle Shutdown Gracefully:** Process remaining requests.

6. **Consider Response Path:** May need async return path.

[Page 31]

PART SIX: ADVANCED PATTERNS

Chapter 29: Balking Pattern

Detailed Explanation

The Balking pattern executes an operation only if the object is in an appropriate state. If not, it returns immediately without waiting. This pattern is useful when waiting would be pointless or would cause problems.

Understanding the Problem

Sometimes an operation should only be performed if certain conditions are met:

- Starting a service that's already started
- Saving data that hasn't changed
- Processing an event when already processing
- Updating a configuration that's locked

If the condition isn't met, waiting might be wasteful or dangerous.

The Solution

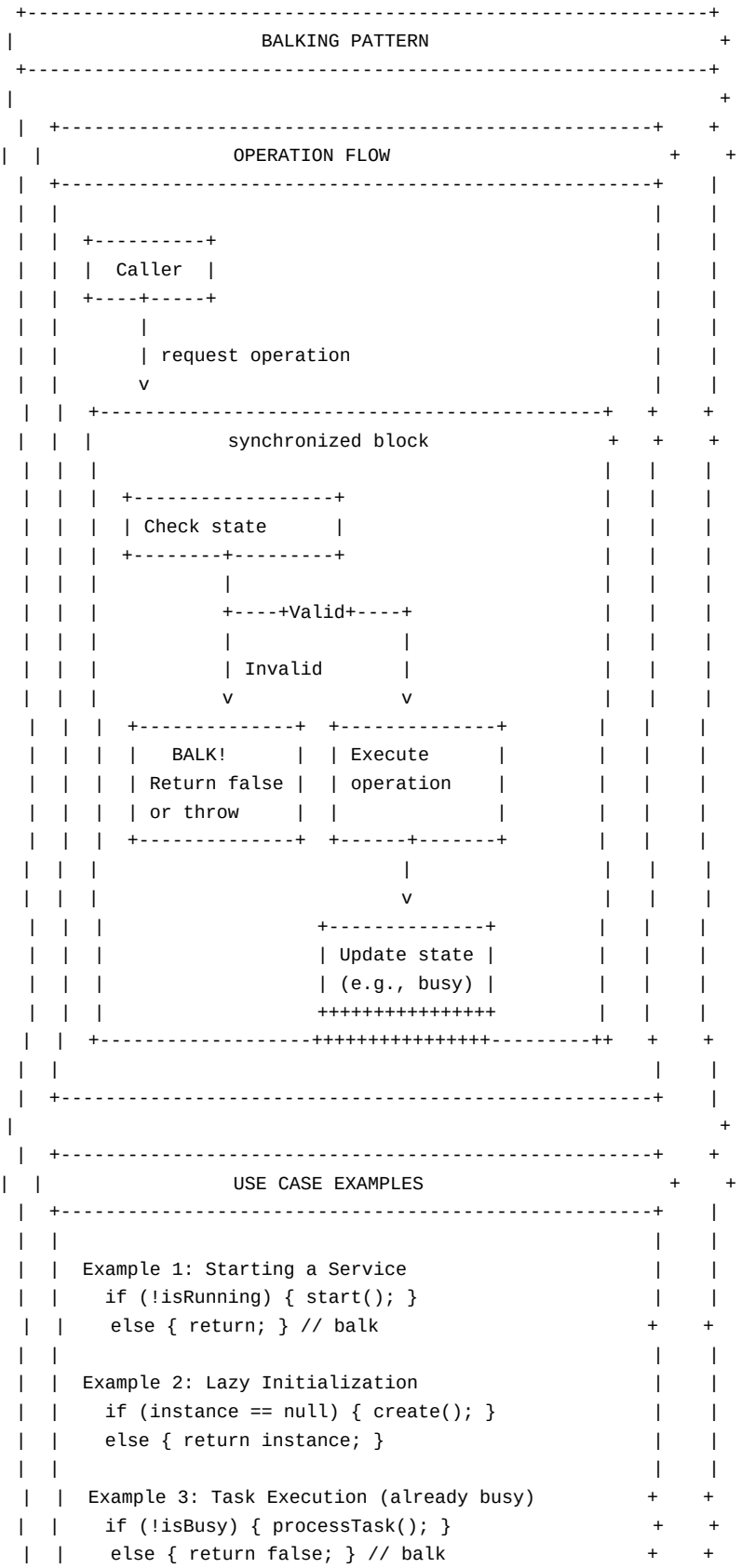
The Balking pattern checks the state before executing. If the state is invalid, the method returns immediately (or throws an exception). This is like a "guard clause" for concurrency.

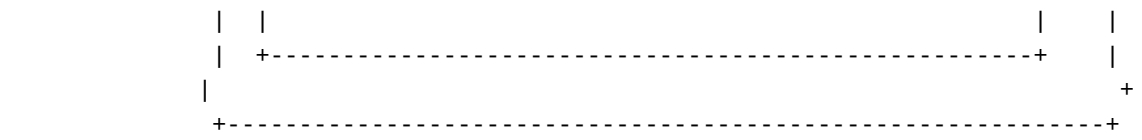
Key Concepts

- **Guard:** Condition that must be true to proceed

- **Balk:** Return immediately when guard fails
- **State Check:** Check condition without blocking
- **Atomic Check:** Must be done atomically with state change

Diagram: Balking Pattern





Implementation Example

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
// 1. Service that can only be started once
class StartableService {
private volatile boolean started = false;
private final Object lock = new Object();

public boolean start() {
synchronized (lock) {
if (started) {
                            System.out.println("Service already started - balking");
return false; // Balk
}

System.out.println("Starting service...");
// Simulate startup
try {
Thread.sleep(100);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
return false;
}
started = true;
return true;
}
}

public boolean isStarted() {
return started;
}
}
// 2. Task Processor that bails if already busy
class TaskProcessor {
private final AtomicBoolean busy = new AtomicBoolean(false);
private final ExecutorService executor = Executors.newSingleThreadExecutor();

public boolean processTask(Runnable task) {
// Try to acquire busy flag atomically
if (!busy.compareAndSet(false, true)) {
                              System.out.println("Processor busy - balking");
return false; // Balk
}

// Execute task
                                        executor.submit(() -> {
try {
task.run();
} finally {
busy.set(false); // Release busy flag
}
});

return true;
}
```

```
public void shutdown() {
executor.shutdown();
}
}
// 3. Cached Data with Balking Refresh
class DataCache {
private String data;
private long lastUpdated;
private final AtomicBoolean refreshing = new AtomicBoolean(false);
private final long cacheTimeout = 5000; // 5 seconds

public String getData() {
long now = System.currentTimeMillis();
                            if (data == null || (now - lastUpdated) > cacheTimeout) {
// Data is stale, try to refresh
if (refreshing.compareAndSet(false, true)) {
// Start refresh in background
refreshData();
}
// Return stale data (or null) while refreshing
}
return data;
}

private void refreshData() {
                                                    new Thread(() -> {
try {
System.out.println("Refreshing data...");
Thread.sleep(1000); // Simulate network call
                                    data = "Fresh data at " + System.currentTimeMillis();
lastUpdated = System.currentTimeMillis();
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
} finally {
refreshing.set(false);
}
}).start();
}
}
// 4. File Writer with Balking on Open
class FileWriter {
private boolean open = false;
private final Object lock = new Object();
private String fileName;

public boolean open(String fileName) {
synchronized (lock) {
if (open) {
                                     System.out.println("File already open - balking");
return false;
}

this.fileName = fileName;
                                    System.out.println("Opening file: " + fileName);
open = true;
return true;
}
}

public boolean write(String content) {
synchronized (lock) {
if (!open) {
                                     System.out.println("File not open - cannot write");
```

```
return false;
}
                            System.out.println("Writing to " + fileName + ": " + content);
return true;
}
}

public void close() {
synchronized (lock) {
if (!open) return;
                                    System.out.println("Closing file: " + fileName);
open = false;
}
}
}
// 5. Usage Example
public class BalkingExample {
public static void main(String[] args) throws InterruptedException {
// Example 1: Startable Service
System.out.println("=== Startable Service ===");
StartableService service = new StartableService();
                                      service.start(); // First call - starts
                                      service.start(); // Second call - balks

// Example 2: Task Processor
System.out.println("\n=== Task Processor ===");
TaskProcessor processor = new TaskProcessor();

                                         // First task - will execute
                                        processor.processTask(() -> {
System.out.println("Executing long task...");
try { Thread.sleep(2000); } catch (InterruptedException e) {}
});

                                    // Second task - will balk (processor busy)
                                   boolean accepted = processor.processTask(() ->
System.out.println("This task won't run"));
                                 System.out.println("Task accepted? " + accepted);

// Wait for first task to complete
Thread.sleep(3000);

                                       // Third task - will execute now
                        processor.processTask(() -> System.out.println("Third task runs"));

// Example 3: Data Cache
System.out.println("\n=== Data Cache ===");
DataCache cache = new DataCache();
                               System.out.println("First get: " + cache.getData());
                               System.out.println("Second get: " + cache.getData());
Thread.sleep(2000);
                              System.out.println("After delay: " + cache.getData());

// Example 4: File Writer
System.out.println("\n=== File Writer ===");
FileWriter writer = new FileWriter();
                                    writer.write("test"); // Fails - not open
writer.open("test.txt");
writer.write("Hello");
writer.write("World");
writer.close();
writer.write("After close"); // Fails

processor.shutdown();
```

```
            }
        }
```

When to Use Balking Pattern

Suitable Situations:

1. When an operation should only be performed once (like initialization).

2. When an object can only handle one operation at a time.

3. When waiting would be pointless or dangerous.

4. When you want fail-fast behavior.

5. In resource managers where operations can be rejected.

Real-World Use Cases:

- **Service Lifecycle:** Preventing multiple start/stop operations
- **Task Execution:** Rejecting tasks when processor is busy
- **Connection Pools:** Balking when no connections available
- **Cache Refresh:** Only one refresh at a time
- **File Operations:** Preventing concurrent writes

Advantages of Balking Pattern

1. **Simplicity:** No waiting, no complex state management.

2. **Fail-Fast:** Immediate feedback to caller.

3. **No Blocking:** Threads never wait.

4. **Resource Protection:** Prevents concurrent operations.

5. **Predictable Behavior:** Clear failure path.

Disadvantages of Balking Pattern

1. **May Not Fit:** Some operations must succeed.

2. **Caller Responsibility:** Caller must handle balk.

3. **Retry Logic:** Caller may need to implement retry.

4. **State Consistency:** Must ensure state is correct.

5. **Not Always Appropriate:** For operations that must eventually succeed.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Balking	Return if not ready	No waiting
Guarded Suspension	Wait until ready	Blocks
Retry	Retry on failure	Automatic retry
Circuit Breaker	Stop calling failing	State machine

Balking vs Guarded Suspension

- **Balking:** Returns immediately if condition not met
- **Guarded Suspension:** Waits until condition is met

Balking vs Retry

- **Balking:** Returns failure immediately
- **Retry:** Automatically tries again

Common Pitfalls

1. **Not Using Atomic Check:** Check and action not atomic.

2. **Incorrect State Management:** State may become inconsistent.

3. **No Retry Strategy:** Caller may need retry logic.

4. **Hidden Failures:** Caller may not handle balk.

5. **Race Conditions:** Check-then-act issues.

Best Practices

1. **Use Atomic Operations:** Compare-and-set for flags.

2. **Document Balking:** Make clear when method balks.

3. **Return Useful Status:** Indicate why operation failed.

4. **Consider Callback:** For async notification.

5. **Use Timeouts:** For operations that can wait.

6. **Test Concurrent Scenarios:** Verify atomic behavior.

[Page 32]

Chapter 30: Thread-Local Storage Pattern

Detailed Explanation

The Thread-Local Storage pattern provides each thread with its own copy of a variable, eliminating the need for synchronization. Each thread sees its own independent value.

Understanding the Problem

In concurrent programming, shared mutable state is a major source of complexity:

1. **Synchronization Overhead:** Locks add overhead
2. **Deadlock Risk:** Complex lock ordering
3. **Race Conditions:** Hard to debug
4. **Performance:** Contention reduces scalability

Sometimes you need data that is thread-specific:

- Transaction contexts

- User sessions
- Database connections
- Request-scoped data

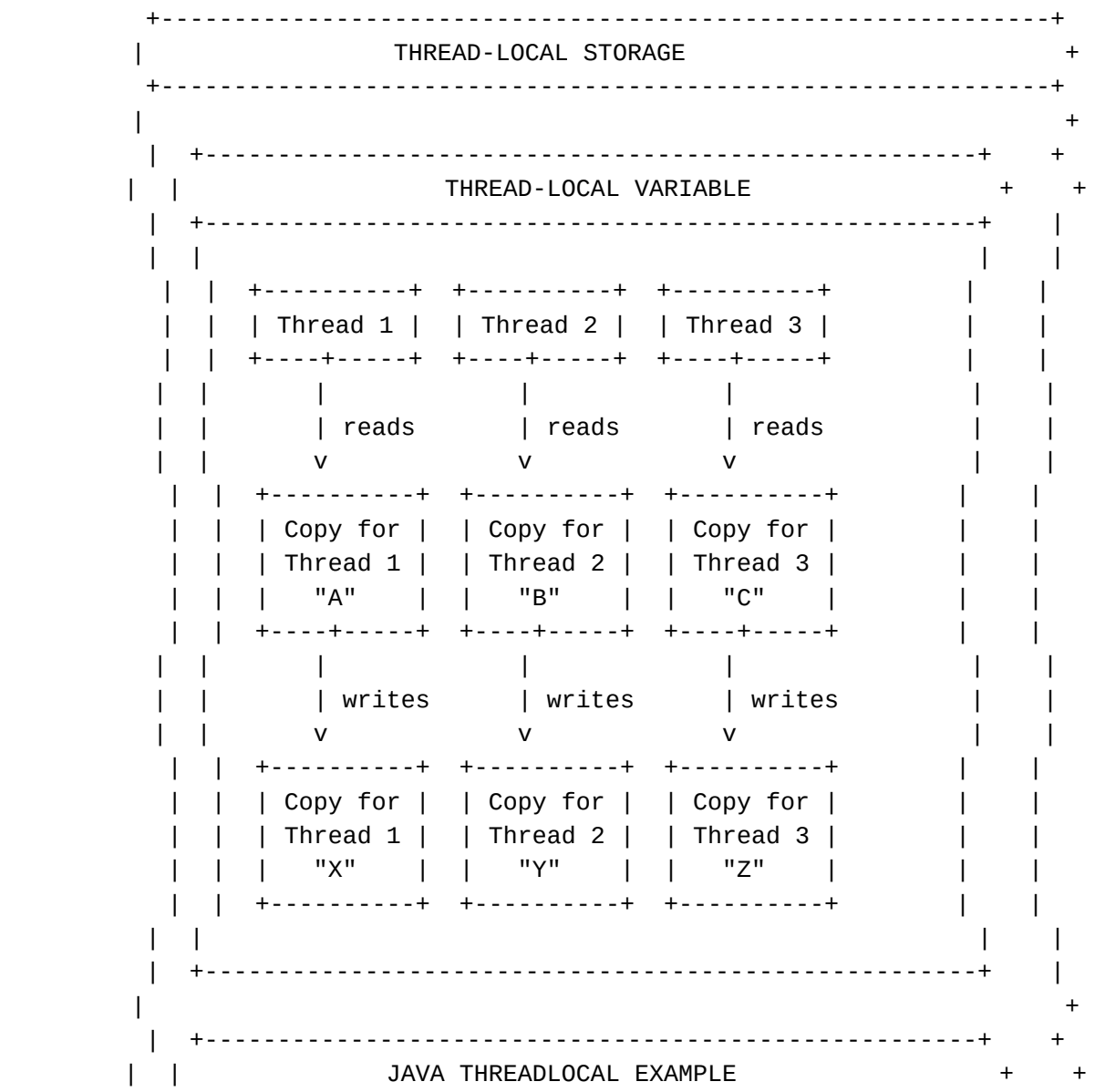
The Solution

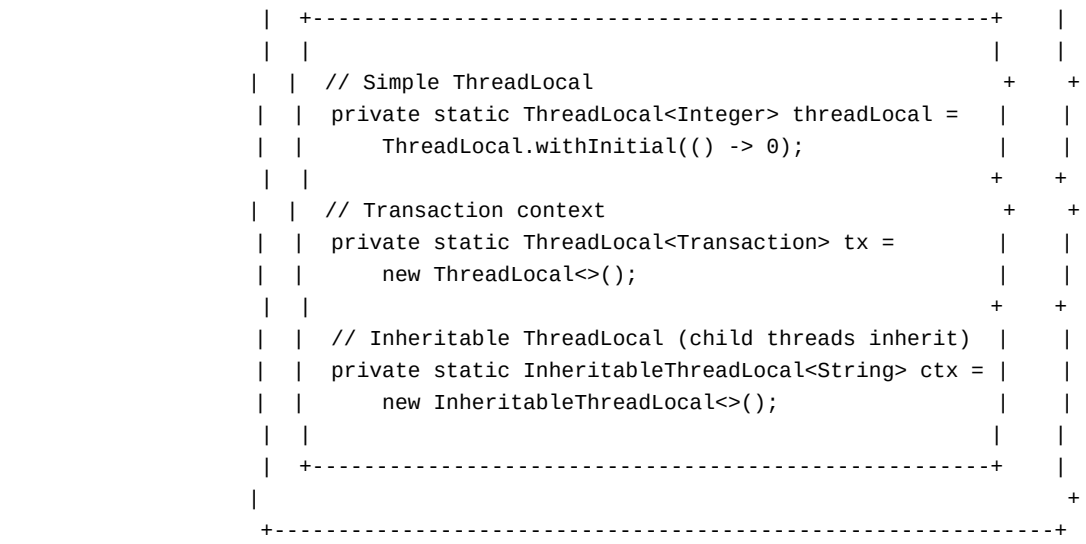
Thread-Local Storage provides a variable that appears to be global but actually has separate storage for each thread. Threads can access their own copy without synchronization.

Key Concepts

- **Thread-Local Variable:** Separate copy per thread
- **Initial Value:** Default value for new threads
- **Inheritance:** Child threads may inherit parent's values
- **Cleanup:** Resources must be cleaned up

Diagram: Thread-Local Storage





Implementation Examples

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
// 1. Basic ThreadLocal
class ThreadLocalExample {
// Simple ThreadLocal with initial value
private static final ThreadLocal<Integer> threadLocal =
                                    ThreadLocal.withInitial(() -> 0);

// ThreadLocal with custom initializer
private static final ThreadLocal<String> userId =
                                  ThreadLocal.withInitial(() -> "anonymous");

// Inheritable ThreadLocal (child threads inherit)
private static final InheritableThreadLocal<String> sessionId =
new InheritableThreadLocal<>();

public static void main(String[] args) throws Exception {
// Create multiple threads
                                     for (int i = 0; i < 3; i++) {
final int threadId = i;
                                            new Thread(() -> {
// Each thread has its own copy
Integer value = threadLocal.get();
                                    System.out.println("Thread " + threadId +
                                               " initial: " + value);

                                          // Modify thread-local value
                                      threadLocal.set(value + threadId + 1);
                                    System.out.println("Thread " + threadId +
                                        " after set: " + threadLocal.get());

                                             // Thread-specific user
                                         userId.set("user-" + threadId);
                                    System.out.println("Thread " + threadId +
                                             " userId: " + userId.get());

// Child thread inherits sessionId
                                      sessionId.set("session-" + threadId);
                                               new Thread(() -> {
                                   System.out.println("Child thread inherits session: " +
sessionId.get());
}).start();
```

```
            }).start();
        }

        Thread.sleep(1000);

        // Clean up (important to avoid memory leaks)
        threadLocal.remove();
        userId.remove();
        sessionId.remove();
    }
}

// 2. Transaction Context Manager
class TransactionManager {
    private static final ThreadLocal<Transaction> currentTransaction =
        new ThreadLocal<>();

    public static void beginTransaction() {
        if (currentTransaction.get() != null) {
            throw new IllegalStateException("Transaction already active");
        }
        currentTransaction.set(new Transaction());
    }

    public static void commit() {
        Transaction tx = currentTransaction.get();
        if (tx == null) {
            throw new IllegalStateException("No active transaction");
        }
        tx.commit();
        currentTransaction.remove(); // Clean up
    }

    public static void rollback() {
        Transaction tx = currentTransaction.get();
        if (tx != null) {
            tx.rollback();
            currentTransaction.remove();
        }
    }

    public static Transaction getCurrentTransaction() {
        return currentTransaction.get();
    }

    static class Transaction {
        private final long id = counter.incrementAndGet();
        private static final AtomicLong counter = new AtomicLong(0);

        void commit() {
            System.out.println("Committing transaction " + id);
        }

        void rollback() {
            System.out.println("Rolling back transaction " + id);
        }
    }
}

// 3. Request Context (like in web frameworks)
class RequestContext {
    private static final ThreadLocal<RequestContext> context =
        new ThreadLocal<>();

    private final String requestId;
```

```
private final String userId;
private final long startTime;

private RequestContext(String requestId, String userId) {
this.requestId = requestId;
this.userId = userId;
this.startTime = System.currentTimeMillis();
}

public static RequestContext get() {
return context.get();
}

public static void set(RequestContext ctx) {
context.set(ctx);
}

public static void clear() {
context.remove();
}

public static void startRequest(String requestId, String userId) {
set(new RequestContext(requestId, userId));
}

public static void endRequest() {
RequestContext ctx = get();
if (ctx != null) {
                            long duration = System.currentTimeMillis() - ctx.startTime;
                                  System.out.println("Request " + ctx.requestId +
                                              " took " + duration + "ms");
clear();
}
}

// Getters
public String getRequestId() { return requestId; }
public String getUserId() { return userId; }
public long getStartTime() { return startTime; }
}
// 4. ThreadLocal with ExecutorService
class ThreadLocalExecutor {
private static final ThreadLocal<String> threadLocal =
                                      ThreadLocal.withInitial(() -> "default");

public static void main(String[] args) {
ExecutorService executor = Executors.newFixedThreadPool(2);

                                    // Submit tasks that set thread-local values
                                           for (int i = 0; i < 4; i++) {
                                           final String value = "task-" + i;
                                                executor.submit(() -> {
                                                 // Set thread-local value
threadLocal.set(value);

// Value persists across tasks in same thread
                                    System.out.println(Thread.currentThread().getName() +
                                                        " set to: " + value);

// Simulate work
try { Thread.sleep(100); } catch (Exception e) {}

// Read back
                                    System.out.println(Thread.currentThread().getName() +
```

```
                    " has: " + threadLocal.get());

                // IMPORTANT: Clean up to prevent memory leaks
                threadLocal.remove();
            });
        }

        executor.shutdown();
    }
}

// 5. Safe ThreadLocal with AutoCloseable
class ThreadLocalCleanup {
    private static final ThreadLocal<Resource> resource =
        ThreadLocal.withInitial(Resource::new);

    public static Resource getResource() {
        return resource.get();
    }

    public static void cleanup() {
        Resource res = resource.get();
        if (res != null) {
            res.close();
            resource.remove();
        }
    }

    static class Resource implements AutoCloseable {
        private static final AtomicLong counter = new AtomicLong();
        private final long id = counter.incrementAndGet();

        Resource() {
            System.out.println("Creating resource " + id);
        }

        public void use() {
            System.out.println("Using resource " + id);
        }

        @Override
        public void close() {
            System.out.println("Closing resource " + id);
        }
    }

    // Usage with try-finally
    public static void main(String[] args) {
        try {
            getResource().use();
        } finally {
            cleanup();
        }
    }
}
```

When to Use Thread-Local Storage

Suitable Situations:

1. When each thread needs its own copy of a variable.

2. For per-thread context (user sessions, transactions).

3. When you want to avoid synchronization.

4. For thread-specific resources (connections, formatters).

5. In frameworks that manage request scope.

Real-World Use Cases:

- **Web Frameworks:** Request/response context
- **Transaction Management:** Per-thread transaction context
- **Date Formatting:** Thread-unsafe SimpleDateFormat
- **Database Connections:** Per-thread connections
- **Logging Context:** MDC (Mapped Diagnostic Context)

Advantages of Thread-Local Storage

1. **No Synchronization:** No locks, no contention.

2. **Thread Safety:** Each thread has its own copy.

3. **Performance:** Fast access, no locking overhead.

4. **Clean Context:** Pass context without parameters.

5. **Resource Management:** Thread-specific resources.

Disadvantages of Thread-Local Storage

1. **Memory Leaks:** Values persist if not cleaned up.

2. **Pooled Threads:** Values may leak across tasks.

3. **Debugging Difficulty:** Hard to track values across threads.

4. **Hidden Dependencies:** Coupling through thread-locals.

5. **Not Suitable for Async:** Thread-local doesn't work with async (fiber) concurrency.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Thread-Local Storage	Per-thread data	No synchronization
Synchronized	Shared data	Locks required
Copy-On-Write	Immutable copies	Different purpose
Scoped Value	Java 21+	Better for async

Thread-Local vs ScopedValue (Java 21+)

Java 21 introduced ScopedValue for async-friendly thread-local:

```
// ScopedValue is better for structured concurrency
ScopedValue<String> SCOPE = ScopedValue.newInstance();

ScopedValue.runWhere(SCOPE, "value", () -> {
    System.out.println(SCOPE.get()); // "value"
});
```

Common Pitfalls

1. **Not Removing:** Memory leaks in thread pools.

2. **Inheritance Issues:** Child threads may not inherit.

3. **Async Problems:** Thread-local doesn't work with async.

4. **Testing Difficulty:** Hard to test code with thread-locals.

5. **Overuse:** Using when parameters would be better.

Best Practices

1. **Always Clean Up:** Use try-finally with remove().

2. **Use with try-with-resources:** For AutoCloseable resources.

3. **Avoid in Thread Pools:** Clean up before returning thread.

4. **Prefer Parameters:** For simple cases.

5. **Document Usage:** Make clear thread-locals are used.

6. **Test with Multiple Threads:** Verify thread isolation.

[Page 33]

Chapter 31: Scheduler Pattern

Detailed Explanation

The Scheduler pattern controls the order in which threads execute by managing a queue of ready threads and selecting the next one to run. It's particularly useful for real-time systems and cooperative multitasking.

Understanding the Problem

In many systems, you need to control when tasks execute:

- **Real-time constraints:** Tasks must complete by deadlines
- **Priority inversion:** High-priority tasks blocked by low-priority ones
- **Resource sharing:** Some tasks need exclusive access
- **Periodic tasks:** Need to run at fixed intervals

The Solution

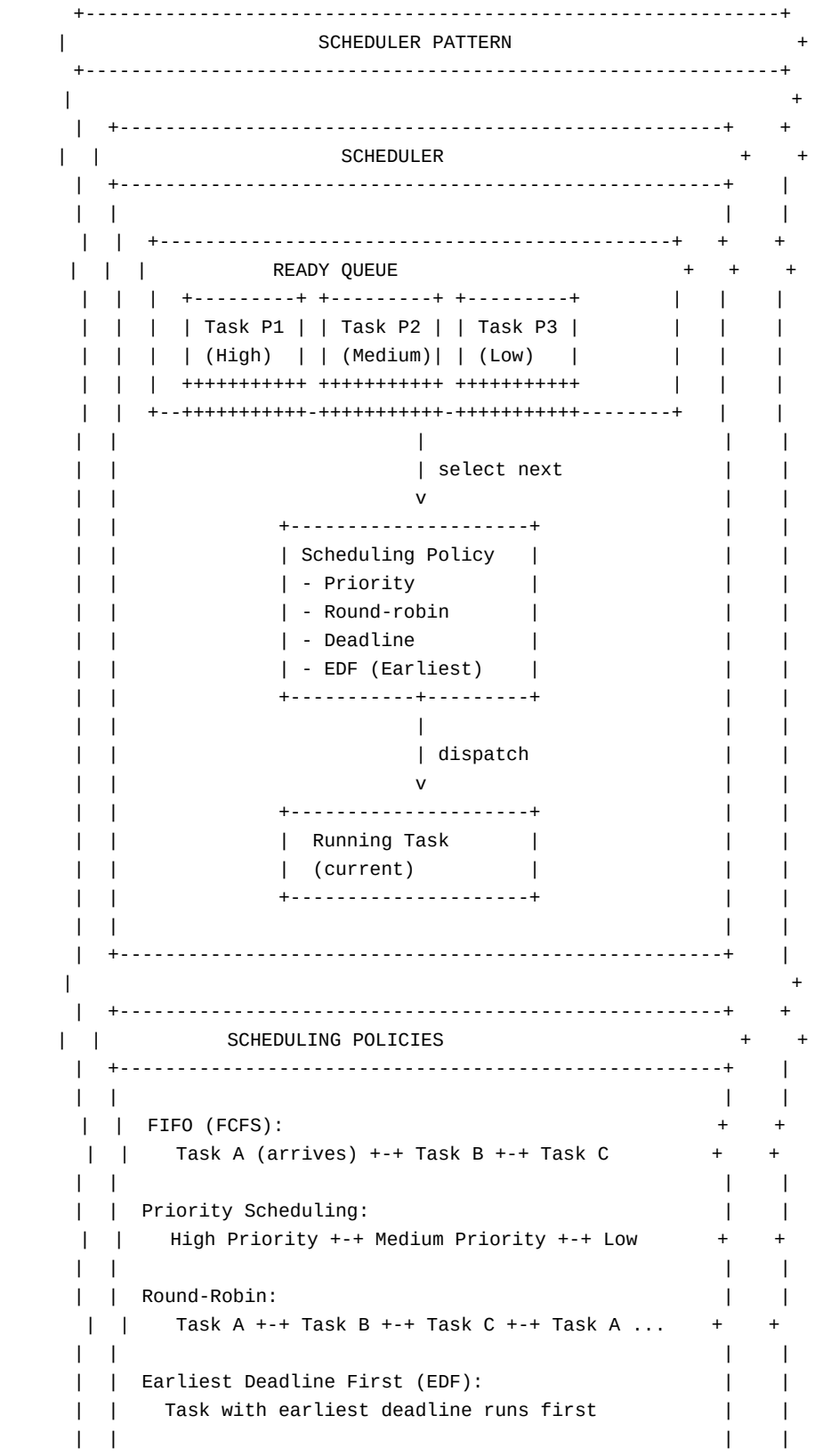
The Scheduler pattern provides a central scheduling entity that:

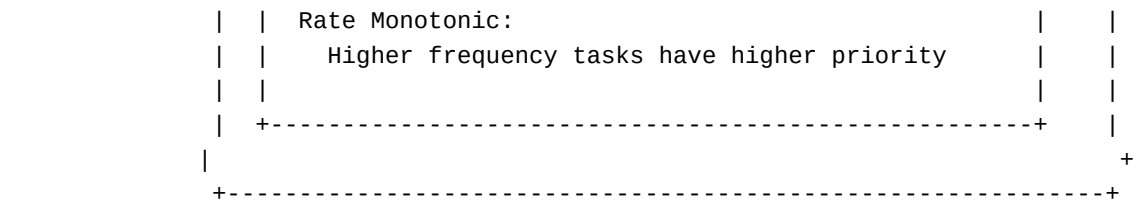
1. Maintains a queue of ready tasks
2. Uses a scheduling policy (FIFO, priority, round-robin)
3. Selects the next task to run
4. Manages preemption and context switching

Key Concepts

- **Scheduler:** Decides which thread runs next
- **Task:** Unit of work with priority and deadline
- **Scheduling Policy:** Algorithm for selection
- **Preemptive:** Scheduler can interrupt running tasks
- **Cooperative:** Tasks must yield voluntarily

Diagram: Scheduler Pattern





Implementation Example

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicLong;
// 1. Task class with priority and deadline
class ScheduledTask implements Comparable<ScheduledTask> {
private static final AtomicLong idGenerator = new AtomicLong();
private final long id = idGenerator.incrementAndGet();
private final Runnable task;
private final int priority; // Higher = more important
private final long deadline; // Optional deadline (0 = no deadline)
private final long creationTime;

public ScheduledTask(Runnable task, int priority) {
this(task, priority, 0);
}

public ScheduledTask(Runnable task, int priority, long deadline) {
this.task = task;
this.priority = priority;
this.deadline = deadline;
this.creationTime = System.currentTimeMillis();
}

public void execute() {
task.run();
}

@Override
public int compareTo(ScheduledTask other) {
// Higher priority first
int priorityCompare = Integer.compare(other.priority, this.priority);
if (priorityCompare != 0) return priorityCompare;

// If deadlines exist, earlier deadline first
if (this.deadline > 0 && other.deadline > 0) {
return Long.compare(this.deadline, other.deadline);
}

// Otherwise FIFO
return Long.compare(this.creationTime, other.creationTime);
}

public long getId() { return id; }
public int getPriority() { return priority; }
public long getDeadline() { return deadline; }
}
// 2. Priority Scheduler
class PriorityScheduler {
private final PriorityBlockingQueue<ScheduledTask> readyQueue;
private final ExecutorService executor;
private volatile boolean running = true;
private ScheduledTask currentTask;
```

```
public PriorityScheduler(int threadCount) {
this.readyQueue = new PriorityBlockingQueue<>();
this.executor = Executors.newFixedThreadPool(threadCount);
startScheduler();
}

public void schedule(Runnable task, int priority) {
readyQueue.offer(new ScheduledTask(task, priority));
}

public void scheduleWithDeadline(Runnable task, int priority, long deadline) {
readyQueue.offer(new ScheduledTask(task, priority, deadline));
}

private void startScheduler() {
                                                executor.submit(() -> {
while (running) {
try {
// Take next task (blocks if none)
currentTask = readyQueue.take();

// Execute task
currentTask.execute();

                                    System.out.println("Completed task " + currentTask.getId() +
                                           " (priority: " + currentTask.getPriority() + ")");

currentTask = null;
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
});
}

public void shutdown() {
running = false;
executor.shutdown();
}

public int getQueueSize() {
return readyQueue.size();
}
}
// 3. Round-Robin Scheduler
class RoundRobinScheduler {
private final Queue<Runnable>[] priorityQueues;
private final int priorities;
private int currentPriority = 0;
private final ExecutorService executor;
private volatile boolean running = true;

@SuppressWarnings("unchecked")
public RoundRobinScheduler(int priorityLevels, int threadCount) {
this.priorities = priorityLevels;
this.priorityQueues = new Queue[priorityLevels];
                                       for (int i = 0; i < priorityLevels; i++) {
priorityQueues[i] = new ConcurrentLinkedQueue<>();
}
this.executor = Executors.newFixedThreadPool(threadCount);
startScheduler();
}
public void schedule(Runnable task, int priority) {
```

```
                                    if (priority >= priorities || priority < 0) {
throw new IllegalArgumentException("Invalid priority");
}
priorityQueues[priority].offer(task);
}

private void startScheduler() {
                                                executor.submit(() -> {
while (running) {
                                               // Round-robin through priorities
                                        for (int i = 0; i < priorities && running; i++) {
                                         int queueIndex = (currentPriority + i) % priorities;
Runnable task = priorityQueues[queueIndex].poll();
if (task != null) {
task.run();
                                             currentPriority = (queueIndex + 1) % priorities;
break;
}
}

// If no tasks, yield
Thread.yield();
}
});
}

public void shutdown() {
running = false;
executor.shutdown();
}
}
// 4. Time-Slicing Scheduler (Preemptive simulation)
class TimeSlicingScheduler {
private final BlockingQueue<Runnable> readyQueue;
private final ExecutorService executor;
private final int timeSlice; // milliseconds
private volatile boolean running = true;

public TimeSlicingScheduler(int threadCount, int timeSlice) {
this.readyQueue = new LinkedBlockingQueue<>();
this.timeSlice = timeSlice;
this.executor = Executors.newFixedThreadPool(threadCount);
startScheduler();
}

public void schedule(Runnable task) {
readyQueue.offer(task);
}

private void startScheduler() {
                        for (int i = 0; i < Runtime.getRuntime().availableProcessors(); i++) {
                                            executor.submit(() -> {
while (running) {
try {
Runnable task = readyQueue.poll(timeSlice, TimeUnit.MILLISECONDS);
if (task != null) {
                                                   // Simulate time-sliced execution
executeWithTimeSlice(task);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
```

```
            });
        }
    }

    private void executeWithTimeSlice(Runnable task) {
        // In real preemptive scheduler, this would be handled by OS
        // Here we simulate by running in a separate thread with timeout
        Thread t = new Thread(task);
        t.start();
        try {
            t.join(timeSlice);
            if (t.isAlive()) {
                // Task exceeded time slice - would be preempted
                System.out.println("Task preempted after " + timeSlice + "ms");
                // In real system, would put back in queue
                readyQueue.offer(task);
                t.interrupt();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    public void shutdown() {
        running = false;
        executor.shutdown();
    }
}

// 5. Deadline Scheduler (EDF - Earliest Deadline First)
class DeadlineScheduler {
    private final PriorityQueue<DeadlineTask> readyQueue;
    private final ExecutorService executor;
    private volatile boolean running = true;

    static class DeadlineTask implements Comparable<DeadlineTask> {
        final Runnable task;
        final long deadline;
        final long id;
        private static AtomicLong counter = new AtomicLong();

        DeadlineTask(Runnable task, long deadline) {
            this.task = task;
            this.deadline = deadline;
            this.id = counter.incrementAndGet();
        }

        @Override
        public int compareTo(DeadlineTask o) {
            // Earliest deadline first
            return Long.compare(this.deadline, o.deadline);
        }
    }

    public DeadlineScheduler(int threadCount) {
        this.readyQueue = new PriorityQueue<>();
        this.executor = Executors.newFixedThreadPool(threadCount);
        startScheduler();
    }

    public void schedule(Runnable task, long deadline) {
        synchronized (readyQueue) {
            readyQueue.offer(new DeadlineTask(task, deadline));
            readyQueue.notify();
        }
```

```
}

private void startScheduler() {
                                                executor.submit(() -> {
while (running) {
DeadlineTask task = null;
synchronized (readyQueue) {
while (running && readyQueue.isEmpty()) {
try {
readyQueue.wait();
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
return;
}
}
task = readyQueue.poll();
}

if (task != null) {
long now = System.currentTimeMillis();
if (now > task.deadline) {
                                        System.out.println("Deadline missed for task " + task.id);
} else {
task.task.run();
                                System.out.println("Task " + task.id + " completed before deadline");
}
}
}
});
}

public void shutdown() {
running = false;
executor.shutdown();
}
}
// 6. Usage Example
public class SchedulerExample {
public static void main(String[] args) throws InterruptedException {
// Priority Scheduler
System.out.println("=== Priority Scheduler ===");
PriorityScheduler priorityScheduler = new PriorityScheduler(1);

                                            priorityScheduler.schedule(() -> {
System.out.println("Low priority task");
}, 1);

                                            priorityScheduler.schedule(() -> {
System.out.println("High priority task");
}, 10);

                                            priorityScheduler.schedule(() -> {
System.out.println("Medium priority task");
}, 5);

Thread.sleep(1000);
priorityScheduler.shutdown();

                                                // Round-Robin Scheduler
                                System.out.println("\n=== Round-Robin Scheduler ===");
RoundRobinScheduler rrScheduler = new RoundRobinScheduler(3, 1);

                                              for (int i = 0; i < 3; i++) {
final int id = i;
```

```
                                    rrScheduler.schedule(() -> {
                                System.out.println("Task " + id + " executing");
}, id);
}

Thread.sleep(500);
rrScheduler.shutdown();

// Deadline Scheduler
System.out.println("\n=== Deadline Scheduler ===");
DeadlineScheduler deadlineScheduler = new DeadlineScheduler(1);

long now = System.currentTimeMillis();
                                deadlineScheduler.schedule(() -> {
System.out.println("Task with 2s deadline");
                                        }, now + 2000);

                                deadlineScheduler.schedule(() -> {
System.out.println("Task with 1s deadline");
                                        }, now + 1000);

Thread.sleep(3000);
deadlineScheduler.shutdown();
}
}
```

When to Use Scheduler Pattern

Suitable Situations:

1. When tasks have different priorities.
2. When you need to meet real-time deadlines.
3. For cooperative multitasking systems.
4. When you need fair CPU time distribution.
5. In embedded and real-time systems.

Real-World Use Cases:

- **Operating Systems:** Process schedulers
- **Real-Time Systems:** Meeting deadlines
- **Workflow Engines:** Task orchestration
- **Batch Processing:** Job schedulers
- **Game Engines:** Update loops with priority

Advantages of Scheduler Pattern

1. **Control:** Fine-grained control over task execution.

2. **Fairness:** Ensure all tasks get CPU time.

3. **Priority:** Important tasks run first.

4. **Deadline Management:** Meet real-time constraints.

5. **Resource Management:** Prevent starvation.

Disadvantages of Scheduler Pattern

1. **Complexity:** Scheduler itself adds complexity.

2. **Overhead:** Scheduling decisions take time.

3. **Priority Inversion:** High-priority tasks blocked by low.

4. **Deadline Misses:** Hard to guarantee in non-real-time.

5. **Tuning:** Need to tune parameters.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
---	---	---
Scheduler	Task ordering	Central scheduling
Thread Pool	Reuse threads	No scheduling logic
Executor	Task execution	Simple queue
Work Stealing	Load balancing	Dynamic distribution

Scheduler vs ExecutorService

- **Scheduler:** Custom scheduling policy
- **ExecutorService:** Fixed policies (FIFO, priority)

Common Pitfalls

1. **Priority Inversion:** Low-priority tasks holding locks.

2. **Starvation:** Low-priority tasks never run.

3. **Deadline Misses:** Not meeting deadlines.

4. **Context Switching:** Too many switches reduce throughput.

5. **Tuning Difficulty:** Optimal parameters hard to find.

Best Practices

1. **Choose Appropriate Policy:** Match workload characteristics.

2. **Monitor Performance:** Track scheduling overhead.

3. **Avoid Priority Inversion:** Use priority inheritance.

4. **Test Under Load:** Verify behavior with real workloads.

5. **Consider Real-Time:** Use RTOS for hard real-time.

[Page 34]

Chapter 32: Supervisor Pattern

Detailed Explanation

The Supervisor pattern manages the lifecycle of worker processes or threads, automatically restarting them when they fail. It's a key pattern for building resilient, self-healing systems.

Understanding the Problem

In complex systems, failures are inevitable:

- **Process crashes:** Unexpected exceptions

- **Resource exhaustion:** Out of memory, file handles
- **Network issues:** Connection failures
- **Bugs:** Logic errors causing crashes

Without supervision, a single failure can bring down the entire system. Manual intervention for each failure is not feasible.

The Solution

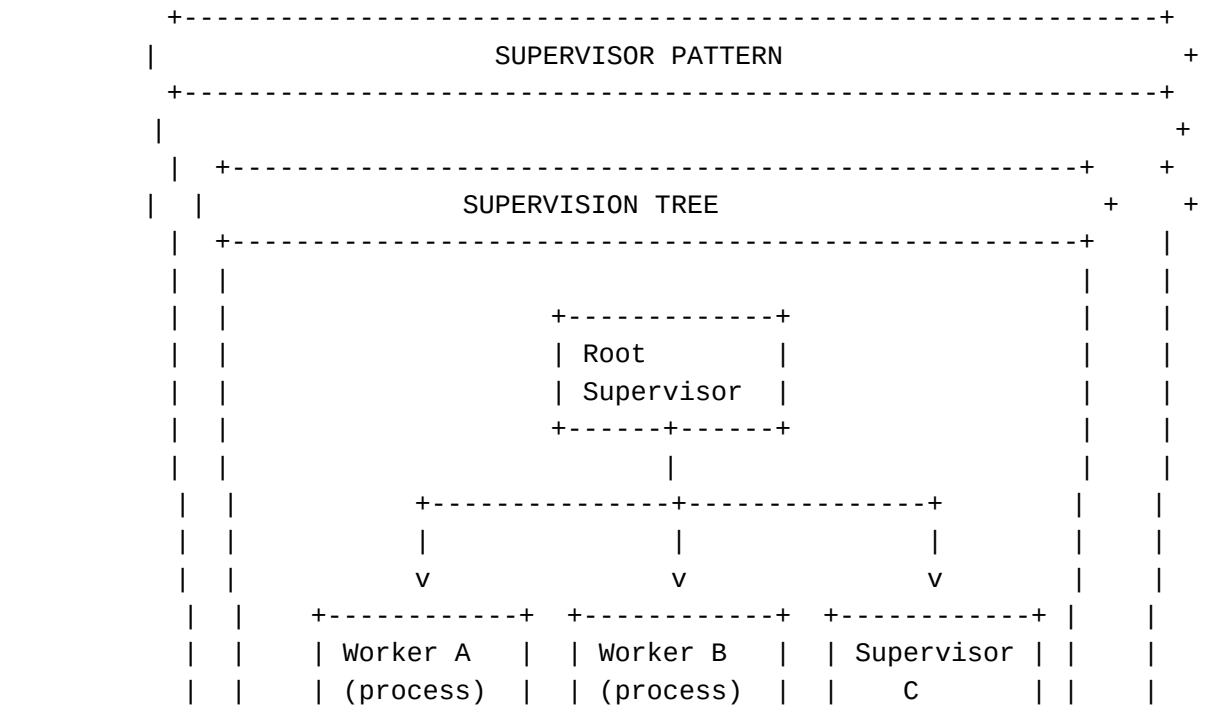
The Supervisor pattern monitors workers and takes action when they fail:

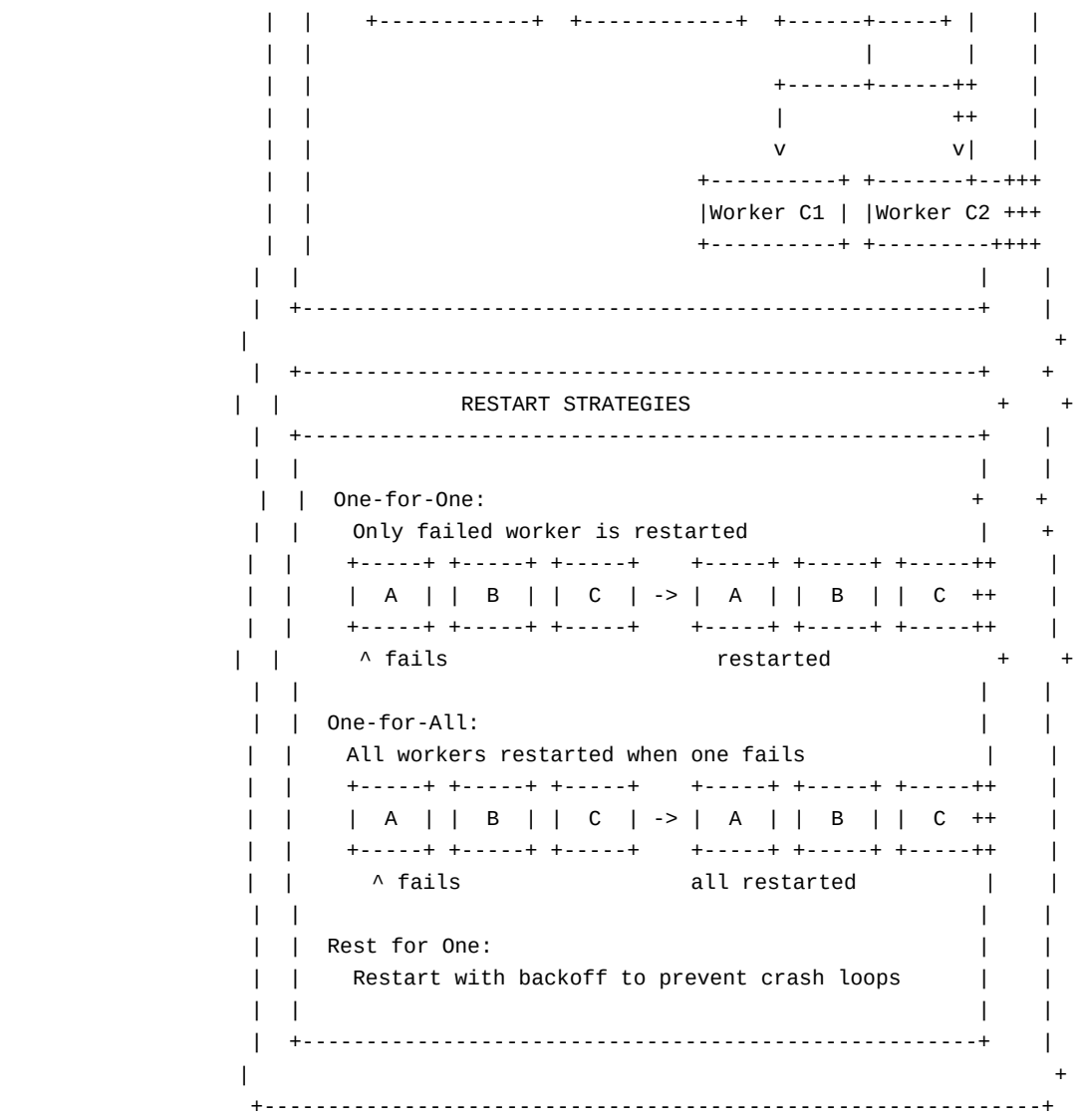
1. **Monitoring:** Detects worker failures
2. **Restart Strategy:** Determines what to do (restart, stop, escalate)
3. **Recovery:** Restarts workers with appropriate state
4. **Supervision Tree:** Hierarchical supervision

Key Concepts

- **Supervisor:** Monitors and manages workers
- **Worker:** Child process/thread being supervised
- **Restart Strategy:** How to restart (one-for-one, one-for-all)
- **Supervision Tree:** Hierarchy of supervisors
- **Error Propagation:** Failure handling up the tree

Diagram: Supervisor Pattern





Implementation Example

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.*;
// 1. Worker interface
interface Worker extends Runnable {
String getId();
boolean isRunning();
void stop();
}
// 2. Simple Worker Implementation
class SimpleWorker implements Worker {
private final String id;
private final Runnable task;
private volatile boolean running = true;
private Thread thread;
private final AtomicInteger failureCount = new AtomicInteger(0);

public SimpleWorker(String id, Runnable task) {
this.id = id;
this.task = task;
}

@Override
```

```
public void run() {
thread = Thread.currentThread();
try {
task.run();
} catch (Exception e) {
failureCount.incrementAndGet();
                        System.err.println("Worker " + id + " failed: " + e.getMessage());
} finally {
running = false;
}
}

@Override
public String getId() { return id; }

@Override
public boolean isRunning() { return running; }

@Override
public void stop() {
running = false;
if (thread != null) {
thread.interrupt();
}
}

public int getFailureCount() { return failureCount.get(); }
}
// 3. Supervisor with restart strategies
class Supervisor {
private final Map<String, Worker> workers = new ConcurrentHashMap<>();
private final ExecutorService executor;
private final RestartStrategy restartStrategy;
private final int maxRestarts;
private final long restartWindowMs;
private final Map<String, List<Long>> restartHistory = new ConcurrentHashMap<>();

public enum RestartStrategy {
                                ONE_FOR_ONE,    // Restart only failed worker
                                   ONE_FOR_ALL,    // Restart all workers
PERMANENT,      // Always restart (with backoff)
TEMPORARY       // Never restart
}

public Supervisor(RestartStrategy strategy, int maxRestarts, long windowMs) {
this.restartStrategy = strategy;
this.maxRestarts = maxRestarts;
this.restartWindowMs = windowMs;
this.executor = Executors.newCachedThreadPool();
}

public void addWorker(Worker worker) {
workers.put(worker.getId(), worker);
startWorker(worker);
}

private void startWorker(Worker worker) {
                                            executor.submit(() -> {
try {
worker.run();
} finally {
handleWorkerFailure(worker);
}
});
```

```
}

private void handleWorkerFailure(Worker worker) {
String id = worker.getId();
                    System.out.println("Worker " + id + " failed. Checking restart policy...");

switch (restartStrategy) {
case TEMPORARY:
                                System.out.println("Worker " + id + " will NOT be restarted");
workers.remove(id);
break;

case PERMANENT:
if (canRestart(id)) {
restartWorker(worker);
} else {
                                  System.out.println("Worker " + id + " max restarts exceeded");
workers.remove(id);
}
break;

                                                        case ONE_FOR_ONE:
if (canRestart(id)) {
restartWorker(worker);
} else {
workers.remove(id);
}
break;

                                                        case ONE_FOR_ALL:
if (canRestart(id)) {
restartAllWorkers();
} else {
System.out.println("Too many restarts, stopping all");
stopAllWorkers();
}
break;
}
}

private boolean canRestart(String workerId) {
List<Long> history = restartHistory.computeIfAbsent(workerId,
                                                    k -> new ArrayList<>());

long now = System.currentTimeMillis();
// Remove old entries
                                  history.removeIf(t -> (now - t) > restartWindowMs);

if (history.size() >= maxRestarts) {
return false;
}

history.add(now);
return true;
}

private void restartWorker(Worker worker) {
                               System.out.println("Restarting worker " + worker.getId());

// Create new worker (old one is dead)
SimpleWorker newWorker = new SimpleWorker(worker.getId(),
((SimpleWorker) worker)::run);

workers.put(worker.getId(), newWorker);
startWorker(newWorker);
```

```
    }

    private void restartAllWorkers() {
        System.out.println("Restarting all workers");
        Map<String, Worker> copy = new HashMap<>(workers);

        for (Worker w : copy.values()) {
            if (w.isRunning()) {
                w.stop();
            }
        }

        workers.clear();

        for (Worker w : copy.values()) {
            SimpleWorker newWorker = new SimpleWorker(w.getId(),
                ((SimpleWorker) w)::run);
            workers.put(w.getId(), newWorker);
            startWorker(newWorker);
        }
    }

    private void stopAllWorkers() {
        System.out.println("Stopping all workers");
        for (Worker w : workers.values()) {
            w.stop();
        }
        workers.clear();
        executor.shutdown();
    }

    public void shutdown() {
        stopAllWorkers();
    }

    public Map<String, Worker> getWorkers() {
        return Collections.unmodifiableMap(workers);
    }
}

// 4. Akka-like Actor Supervisor (simplified)
class ActorSupervisor {
    private final Map<String, Actor> actors = new ConcurrentHashMap<>();
    private final ExecutorService executor = Executors.newCachedThreadPool();

    interface Actor extends Runnable {
        String getName();
        void onFailure(Throwable t);
        void postRestart();
    }

    static class ChildActor implements Actor {
        private final String name;
        private final Runnable behavior;

        ChildActor(String name, Runnable behavior) {
            this.name = name;
            this.behavior = behavior;
        }

        @Override
        public void run() {
            behavior.run();
        }
        @Override
```

```
public String getName() { return name; }

@Override
public void onFailure(Throwable t) {
                        System.err.println("Actor " + name + " failed: " + t.getMessage());
}

@Override
public void postRestart() {
                                System.out.println("Actor " + name + " restarted");
}
}

public void createActor(String name, Runnable behavior) {
ChildActor actor = new ChildActor(name, behavior);
actors.put(name, actor);
startActor(actor);
}

private void startActor(ChildActor actor) {
                                            executor.submit(() -> {
try {
actor.run();
} catch (Exception e) {
actor.onFailure(e);
// Apply supervision strategy
handleActorFailure(actor);
}
});
}

private void handleActorFailure(ChildActor actor) {
                                        // Simple one-for-one restart
                            System.out.println("Restarting actor " + actor.getName());
ChildActor newActor = new ChildActor(actor.getName(), actor.behavior);
actors.put(actor.getName(), newActor);
newActor.postRestart();
startActor(newActor);
}

public void shutdown() {
executor.shutdown();
}
}
// 5. Usage Example
public class SupervisorExample {
public static void main(String[] args) throws InterruptedException {
// Flaky worker that fails after a few runs
AtomicInteger counter = new AtomicInteger(0);
                                        Runnable flakyTask = () -> {
int run = counter.incrementAndGet();
                                    System.out.println("Running task " + run);
if (run % 3 == 0) {
                                throw new RuntimeException("Task " + run + " failed!");
}
try {
Thread.sleep(1000);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
};

                                    // Supervisor with ONE_FOR_ONE strategy
                            System.out.println("=== ONE-FOR-ONE SUPERVISOR ===");
```

```
Supervisor oneForOne = new Supervisor(
                            Supervisor.RestartStrategy.ONE_FOR_ONE,
5, 10000);

SimpleWorker worker1 = new SimpleWorker("worker1", flakyTask);
                SimpleWorker worker2 = new SimpleWorker("worker2", () -> {
System.out.println("Worker2: steady work");
try { Thread.sleep(2000); } catch (InterruptedException e) {}
});

oneForOne.addWorker(worker1);
oneForOne.addWorker(worker2);

Thread.sleep(8000);
oneForOne.shutdown();

// Actor Supervisor
System.out.println("\n=== ACTOR SUPERVISOR ===");
ActorSupervisor actorSupervisor = new ActorSupervisor();

                        actorSupervisor.createActor("flaky", () -> {
                                for (int i = 0; i < 3; i++) {
                          System.out.println("Flaky actor running " + i);
if (i == 2) {
throw new RuntimeException("Flaky actor failed!");
}
try { Thread.sleep(500); } catch (Exception e) {}
}
});

                        actorSupervisor.createActor("steady", () -> {
                                for (int i = 0; i < 5; i++) {
System.out.println("Steady actor working");
try { Thread.sleep(500); } catch (Exception e) {}
}
});

Thread.sleep(5000);
actorSupervisor.shutdown();
}
}
```

When to Use Supervisor Pattern

Suitable Situations:

1. When building resilient, self-healing systems.
2. For managing long-running processes.
3. In actor-based systems (Akka, Erlang).
4. When failures should be handled automatically.
5. For microservices that need restart on failure.

Real-World Use Cases:

- **Erlang/OTP:** Supervision trees
- **Akka:** Actor supervision
- **Kubernetes:** Pod restarts
- **Systemd:** Service supervision
- **Process Managers:** PM2, supervisord

Advantages of Supervisor Pattern

1. **Self-Healing:** Automatic recovery from failures.

2. **Fault Isolation:** Failures contained to workers.

3. **Structured Recovery:** Clear restart strategies.

4. **Hierarchical:** Complex failure handling.

5. **Observability:** Clear failure history.

Disadvantages of Supervisor Pattern

1. **Complexity:** Additional layer to manage.

2. **State Loss:** Workers may lose state on restart.

3. **Crash Loops:** Continuous restarts if underlying issue persists.

4. **Debugging:** Hard to debug across restarts.

5. **Overhead:** Monitoring adds overhead.

Comparison with Other Patterns

Pattern	**Purpose**	**Difference**
Supervisor	Failure recovery	Restarts workers
Circuit Breaker	Stop failing calls	Prevents calls

Pattern	Purpose	Difference
Retry	Retry operations	Transient failures
Health Check	Detect failures	Monitoring

Supervisor vs Circuit Breaker

- **Supervisor:** Restarts workers on failure
- **Circuit Breaker:** Prevents calls to failing service

Supervisor vs Health Check

- **Supervisor:** Takes action on failure
- **Health Check:** Only detects failure

Common Pitfalls

1. **Crash Loops:** Unlimited restarts without backoff.

2. **State Loss:** Not persisting critical state.

3. **Over-supervision:** Supervising non-fault-tolerant components.

4. **Restart Storm:** Many restarts overwhelming system.

5. **Circular Dependencies:** Supervisors depending on each other.

Best Practices

1. **Use Backoff:** Exponential backoff for restarts.

2. **Limit Restarts:** Maximum restarts within time window.

3. **Persist State:** Store state externally when needed.

4. **Separate Concerns:** Only supervise fault-prone components.

5. **Monitor Restarts:** Alert on frequent restarts.

6. **Test Failure Scenarios:** Verify recovery behavior.

[Page 35]

Chapter 33: Rate Limiting Pattern

Detailed Explanation

The Rate Limiting pattern controls the rate at which operations are performed to prevent system overload, ensure fair resource distribution, and protect against abuse.

Understanding the Problem

In distributed systems, uncontrolled request rates can cause:

1. **Resource Exhaustion:** Overwhelming databases, services
2. **Denial of Service:** Malicious or accidental overload
3. **Cost Spikes:** Cloud costs based on usage
4. **Unfairness:** Some clients consuming all resources
5. **Cascading Failures:** Downstream services overwhelmed

The Solution

The Rate Limiting pattern limits the number of operations within a time window:

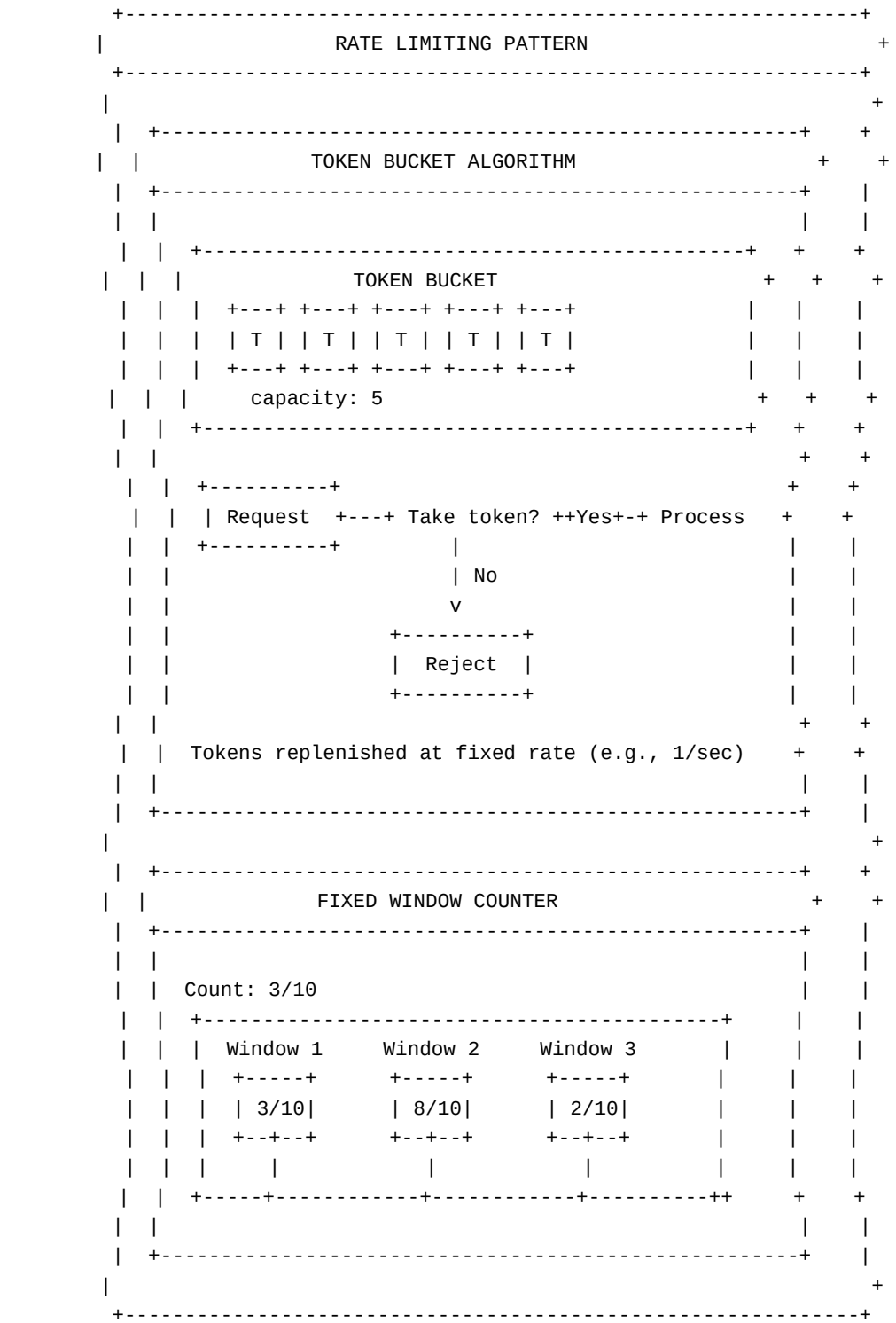
1. **Leaky Bucket:** Requests processed at fixed rate
2. **Token Bucket:** Tokens replenished at fixed rate
3. **Fixed Window:** Count per time window
4. **Sliding Window:** Rolling time window

Key Concepts

- **Rate:** Operations per time unit (e.g., 100 requests/second)
- **Burst Capacity:** Maximum allowed in a short burst
- **Queue:** Temporary storage for excess requests
- **Throttling:** Rejecting or delaying requests

- **Backpressure:** Signaling to slow down senders

Diagram: Rate Limiting Algorithms



Implementation Examples

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
// 1. Token Bucket Rate Limiter
class TokenBucket {
private final long capacity;
private final long refillRate; // tokens per second
private final AtomicLong availableTokens;
private final AtomicLong lastRefillTime;
```

```
    public TokenBucket(long capacity, long refillRate) {
        this.capacity = capacity;
        this.refillRate = refillRate;
        this.availableTokens = new AtomicLong(capacity);
        this.lastRefillTime = new AtomicLong(System.nanoTime());
    }

    public boolean tryAcquire() {
        refill();
        long current = availableTokens.get();
        while (current > 0 && !availableTokens.compareAndSet(current, current - 1)) {
            current = availableTokens.get();
        }
        return current > 0;
    }

    private void refill() {
        long now = System.nanoTime();
        long lastRefill = lastRefillTime.get();
        long elapsed = now - lastRefill;

        if (elapsed > 0) {
            long tokensToAdd = elapsed * refillRate / 1_000_000_000L;
            if (tokensToAdd > 0) {
                long newTokens = Math.min(capacity,
                    availableTokens.get() + tokensToAdd);
                availableTokens.set(newTokens);
                lastRefillTime.set(now);
            }
        }
    }
}

// 2. Leaky Bucket Rate Limiter
class LeakyBucket {
    private final long capacity;
    private final long leakRate; // tokens per second
    private final AtomicLong waterLevel;
    private final AtomicLong lastLeakTime;

    public LeakyBucket(long capacity, long leakRate) {
        this.capacity = capacity;
        this.leakRate = leakRate;
        this.waterLevel = new AtomicLong(0);
        this.lastLeakTime = new AtomicLong(System.nanoTime());
    }

    public boolean tryAcquire() {
        leak();
        long current = waterLevel.get();
        if (current >= capacity) {
            return false;
        }
        return waterLevel.compareAndSet(current, current + 1);
    }

    private void leak() {
        long now = System.nanoTime();
        long lastLeak = lastLeakTime.get();
        long elapsed = now - lastLeak;

        if (elapsed > 0) {
            long leaked = elapsed * leakRate / 1_000_000_000L;
            if (leaked > 0) {
                long newLevel = Math.max(0, waterLevel.get() - leaked);
```

```
                waterLevel.set(newLevel);
                lastLeakTime.set(now);
            }
        }
    }
}

// 3. Fixed Window Counter
class FixedWindowCounter {
    private final long windowSizeMs;
    private final long maxRequests;
    private final AtomicLong count;
    private final AtomicLong windowStart;

    public FixedWindowCounter(long windowSizeMs, long maxRequests) {
        this.windowSizeMs = windowSizeMs;
        this.maxRequests = maxRequests;
        this.count = new AtomicLong(0);
        this.windowStart = new AtomicLong(System.currentTimeMillis());
    }

    public boolean tryAcquire() {
        long now = System.currentTimeMillis();
        long start = windowStart.get();

        if (now - start >= windowSizeMs) {
            // New window
            if (windowStart.compareAndSet(start, now)) {
                count.set(0);
            }
        }

        long current = count.get();
        if (current >= maxRequests) {
            return false;
        }
        return count.compareAndSet(current, current + 1);
    }
}

// 4. Sliding Window Counter (more accurate)
class SlidingWindowCounter {
    private final long windowSizeMs;
    private final long maxRequests;
    private final ConcurrentLinkedQueue<Long> timestamps = new ConcurrentLinkedQueue<>();
    private final AtomicLong count = new AtomicLong(0);

    public SlidingWindowCounter(long windowSizeMs, long maxRequests) {
        this.windowSizeMs = windowSizeMs;
        this.maxRequests = maxRequests;
    }

    public boolean tryAcquire() {
        long now = System.currentTimeMillis();
        long windowStart = now - windowSizeMs;

        // Remove old timestamps
        while (!timestamps.isEmpty() && timestamps.peek() < windowStart) {
            timestamps.poll();
            count.decrementAndGet();
        }

        if (count.get() >= maxRequests) {
            return false;
        }
```

```
timestamps.offer(now);
count.incrementAndGet();
return true;
}
}
// 5. Distributed Rate Limiter using Redis (simulated)
class DistributedRateLimiter {
private final String key;
private final long maxRequests;
private final long windowSizeMs;
// In real implementation, would use Redis
private final Map<String, SlidingWindowCounter> counters = new ConcurrentHashMap<>();

public DistributedRateLimiter(String key, long maxRequests, long windowSizeMs) {
this.key = key;
this.maxRequests = maxRequests;
this.windowSizeMs = windowSizeMs;
}

public boolean tryAcquire(String clientId) {
                                    String clientKey = key + ":" + clientId;
SlidingWindowCounter counter = counters.computeIfAbsent(clientKey,
                                k -> new SlidingWindowCounter(windowSizeMs, maxRequests));
return counter.tryAcquire();
}
}
// 6. Rate Limiter with Queue and Backpressure
class RateLimiterWithQueue {
private final TokenBucket tokenBucket;
private final BlockingQueue<Runnable> queue;
private final ExecutorService executor;
private final int queueSize;

public RateLimiterWithQueue(long rate, long capacity, int queueSize) {
this.tokenBucket = new TokenBucket(capacity, rate);
this.queue = new LinkedBlockingQueue<>(queueSize);
this.queueSize = queueSize;
this.executor = Executors.newSingleThreadExecutor();
startProcessor();
}

private void startProcessor() {
                                                executor.submit(() -> {
while (true) {
try {
Runnable task = queue.take();
if (tokenBucket.tryAcquire()) {
task.run();
} else {
// Wait and retry
Thread.sleep(100);
queue.offer(task);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
});
}

public boolean submit(Runnable task) {
return queue.offer(task);
```

```
}

public void shutdown() {
executor.shutdown();
}
}
// 7. Usage Example
public class RateLimitingExample {
public static void main(String[] args) throws InterruptedException {
// Token Bucket Example
System.out.println("=== Token Bucket Rate Limiter ===");
TokenBucket tokenBucket = new TokenBucket(5, 1); // 5 tokens, 1 per second

                                    for (int i = 0; i < 10; i++) {
boolean allowed = tokenBucket.tryAcquire();
                                System.out.println("Request " + i + ": " +
(allowed ? "ACCEPTED" : "REJECTED"));
Thread.sleep(200); // 5 requests per second (1 per 200ms)
}

// Fixed Window Example
System.out.println("\n=== Fixed Window Rate Limiter ===");
FixedWindowCounter window = new FixedWindowCounter(1000, 3);

                                    for (int i = 0; i < 10; i++) {
boolean allowed = window.tryAcquire();
                                System.out.println("Request " + i + ": " +
(allowed ? "ACCEPTED" : "REJECTED"));
Thread.sleep(100);
}

// Sliding Window Example
System.out.println("\n=== Sliding Window Rate Limiter ===");
SlidingWindowCounter sliding = new SlidingWindowCounter(1000, 3);

                                    for (int i = 0; i < 10; i++) {
boolean allowed = sliding.tryAcquire();
                                System.out.println("Request " + i + ": " +
(allowed ? "ACCEPTED" : "REJECTED"));
Thread.sleep(100);
}

// Rate Limiter with Queue
System.out.println("\n=== Rate Limiter with Queue ===");
RateLimiterWithQueue queued = new RateLimiterWithQueue(2, 5, 10);

                                    for (int i = 0; i < 20; i++) {
final int id = i;
                                  boolean queued_ = queued.submit(() -> {
                                System.out.println("Processing request " + id);
try { Thread.sleep(100); } catch (Exception e) {}
});
                        System.out.println("Request " + i + " queued: " + queued_);
}

Thread.sleep(5000);
queued.shutdown();
}
}
```
When to Use Rate Limiting Pattern

Suitable Situations:

1. For API endpoints to prevent abuse.

2. To protect downstream services from overload.

3. For fair resource distribution among tenants.

4. To control costs in cloud environments.

5. For rate-limited external services (Twitter, GitHub APIs).

Real-World Use Cases:

- **API Gateways:** Rate limiting per client
- **Web Servers:** Prevent DDoS attacks
- **Database Proxies:** Protect databases from high load
- **Message Queues:** Control processing rates
- **Cloud Services:** AWS throttling, API Gateway

Advantages of Rate Limiting Pattern

1. **Protection:** Prevents system overload.

2. **Fairness:** Equal resource distribution.

3. **Cost Control:** Limits usage costs.

4. **Stability:** Smooths traffic spikes.

5. **Abuse Prevention:** Mitigates DDoS and brute force.

Disadvantages of Rate Limiting Pattern

1. **Complexity:** Need to choose and implement algorithm.

2. **Latency:** Checking rate limits adds overhead.

3. **User Experience:** Legitimate users may be throttled.

4. **Distributed Coordination:** Hard to enforce across nodes.

5. **Tuning:** Finding right limits requires experimentation.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Rate Limiting	Control operation rate	Limits frequency
Circuit Breaker	Stop failing calls	Based on errors
Bulkhead	Isolate resources	Limits concurrency
Backpressure	Slow down producers	Signal to upstream

Rate Limiting vs Circuit Breaker

- **Rate Limiting:** Prevents overload proactively
- **Circuit Breaker:** Reacts to failures

Rate Limiting vs Backpressure

- **Rate Limiting:** Centralized control
- **Backpressure:** Distributed control from consumer

Common Pitfalls

1. **Single Node Limit:** Distributed systems need coordination.

2. **Burst Handling:** Too strict limits hurt legitimate users.

3. **Window Boundary Bursts:** Fixed window can allow double burst.

4. **Resource Exhaustion:** Rate limiter itself can be bottleneck.

5. **Poor User Feedback:** Users don't know why requests fail.

Best Practices

1. **Use Sliding Window:** More accurate than fixed window.

2. **Return Headers:** Include limit, remaining, reset information.

3. **Exponential Backoff:** Client retry with backoff.

4. **Distributed Coordination:** Use Redis for shared state.

5. **Monitor:** Track rate limiting events.

6. **Granular Limits:** Different limits for different clients.

7. **Queue Bursts:** Allow small bursts with queuing.

[Page 36]

Chapter 34: Singleflight Pattern

Detailed Explanation

The Singleflight pattern prevents duplicate concurrent operations from being executed multiple times. When multiple threads request the same operation simultaneously, only one executes, and the others wait for the result.

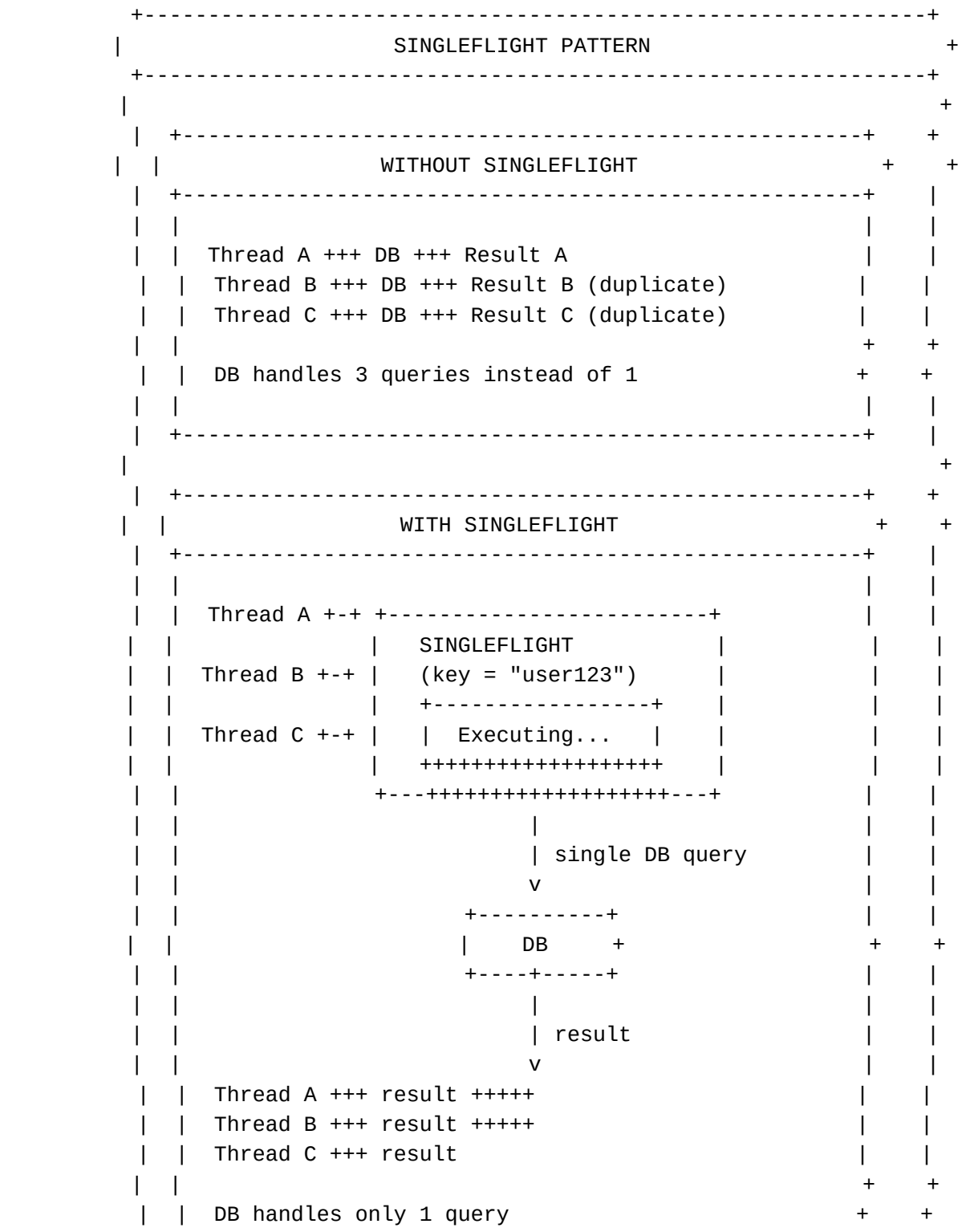
Understanding the Problem

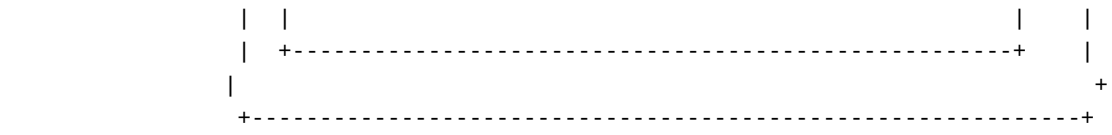
Consider a cache miss scenario:

1. Multiple threads simultaneously request a value that's not in cache
2. All threads go to the database to fetch the value
3. Database gets hammered with duplicate queries
4. Wasted resources, potential overload

Similarly for:
- Expensive computations
- External API calls
- File loads

The Solution
The Singleflight pattern ensures that for a given key, only one operation runs at a time. Other requests for the same key wait for the first to complete and share its result.
Key Concepts
- **Key:** Identifier for the operation group
- **In-flight:** Operations currently executing
- **Waiters:** Threads waiting for result
- **Deduplication:** Eliminating duplicate concurrent operations
Diagram: Singleflight Pattern





Implementation Examples

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.*;
// 1. Simple Singleflight Implementation
class SimpleSingleflight {
private final ConcurrentMap<String, Future<?>> inflight = new ConcurrentHashMap<>();
private final ExecutorService executor = Executors.newCachedThreadPool();

public <T> CompletableFuture<T> doSingle(String key, Callable<T> operation) {
// Check if operation already in flight
Future<?> existing = inflight.get(key);
if (existing != null) {
// Wait for existing operation to complete
                                    return CompletableFuture.supplyAsync(() -> {
try {
return (T) existing.get();
} catch (Exception e) {
throw new CompletionException(e);
}
});
}

// Create new future for this operation
CompletableFuture<T> future = new CompletableFuture<>();

// Try to register as the executing operation
Future<?> previous = inflight.putIfAbsent(key, future);
if (previous != null) {
                                       // Someone else started - wait for them
                                    return CompletableFuture.supplyAsync(() -> {
try {
return (T) previous.get();
} catch (Exception e) {
throw new CompletionException(e);
}
});
}

                                // We are the chosen one - execute the operation
                                            executor.submit(() -> {
try {
T result = operation.call();
future.complete(result);
// Propagate result to waiting futures (they're already waiting)
} catch (Exception e) {
future.completeExceptionally(e);
} finally {
inflight.remove(key, future);
}
});

return future;
}

public void shutdown() {
```

```
executor.shutdown();
}
}
// 2. Go-style Singleflight Implementation
class SingleflightGroup {
private final ConcurrentMap<String, Call> calls = new ConcurrentHashMap<>();

private static class Call {
final CompletableFuture<Object> future = new CompletableFuture<>();
final AtomicInteger count = new AtomicInteger(1);

void addWaiter() {
count.incrementAndGet();
}

int getCount() {
return count.get();
}
}

public <T> T doSingle(String key, Callable<T> operation) throws Exception {
                            Call call = calls.computeIfAbsent(key, k -> new Call());

if (call == null) {
// Already completed or error
call = calls.get(key);
}

if (call.getCount() > 1) {
// Already in flight, add waiter
call.addWaiter();
} else {
// We are the first, execute
try {
T result = operation.call();
call.future.complete(result);
return result;
} catch (Exception e) {
call.future.completeExceptionally(e);
throw e;
} finally {
calls.remove(key);
}
}

// Wait for result
try {
return (T) call.future.get();
} catch (ExecutionException e) {
throw (Exception) e.getCause();
}
}
}
// 3. Singleflight with Cache Integration
class SingleflightCache<K, V> {
private final ConcurrentMap<K, V> cache = new ConcurrentHashMap<>();
private final ConcurrentMap<K, CompletableFuture<V>> inflight = new ConcurrentHashMap<>();
private final Function<K, V> loader;

public SingleflightCache(Function<K, V> loader) {
this.loader = loader;
}
public V get(K key) throws Exception {
```

```
// Check cache first
V cached = cache.get(key);
if (cached != null) {
return cached;
}

// Check if already loading
CompletableFuture<V> future = inflight.get(key);
if (future != null) {
return future.get();
}

// Start loading
                    CompletableFuture<V> newFuture = CompletableFuture.supplyAsync(() -> {
try {
return loader.apply(key);
} catch (Exception e) {
throw new CompletionException(e);
}
});

// Try to register
CompletableFuture<V> existing = inflight.putIfAbsent(key, newFuture);
if (existing != null) {
return existing.get();
}

try {
V result = newFuture.get();
cache.put(key, result);
return result;
} catch (ExecutionException e) {
throw (Exception) e.getCause();
} finally {
inflight.remove(key);
}
}

public void evict(K key) {
cache.remove(key);
}

public void clear() {
cache.clear();
}
}
// 4. Singleflight with Timeout
class TimeoutSingleflight {
private final ConcurrentMap<String, Call> calls = new ConcurrentHashMap<>();
private final long timeoutMs;

private static class Call {
final CompletableFuture<Object> future = new CompletableFuture<>();
final long startTime;

Call() {
this.startTime = System.currentTimeMillis();
}

boolean isExpired(long timeoutMs) {
                            return System.currentTimeMillis() - startTime > timeoutMs;
}
}
public TimeoutSingleflight(long timeoutMs) {
```

```
this.timeoutMs = timeoutMs;
}

public <T> CompletableFuture<T> doSingle(String key, Callable<T> operation) {
// Clean expired calls
cleanExpired();

Call call = calls.get(key);
if (call != null && !call.isExpired(timeoutMs)) {
return (CompletableFuture<T>) call.future;
}

// Create new call
Call newCall = new Call();
Call existing = calls.putIfAbsent(key, newCall);

if (existing != null && !existing.isExpired(timeoutMs)) {
return (CompletableFuture<T>) existing.future;
}

// Execute operation
                        CompletableFuture<T> future = CompletableFuture.supplyAsync(() -> {
try {
return operation.call();
} catch (Exception e) {
throw new CompletionException(e);
}
});

// Link to call
                                        future.whenComplete((result, ex) -> {
if (ex != null) {
newCall.future.completeExceptionally(ex);
} else {
newCall.future.complete(result);
}
calls.remove(key, newCall);
});

return future;
}

private void cleanExpired() {
                    calls.entrySet().removeIf(entry -> entry.getValue().isExpired(timeoutMs));
}
}
// 5. Usage Example
public class SingleflightExample {
private static final AtomicInteger dbCalls = new AtomicInteger(0);

static class Database {
static String fetchUser(String userId) throws InterruptedException {
dbCalls.incrementAndGet();
                              System.out.println("Executing database query for " + userId);
Thread.sleep(1000); // Simulate DB latency
                                        return "User data for " + userId;
}
}

public static void main(String[] args) throws Exception {
System.out.println("=== Singleflight Example ===");

                                    // Without Singleflight - multiple DB calls
System.out.println("\nWithout Singleflight:");
```

```
dbCalls.set(0);
ExecutorService executor = Executors.newFixedThreadPool(5);
List<CompletableFuture<String>> futures = new ArrayList<>();

                                for (int i = 0; i < 10; i++) {
                            futures.add(CompletableFuture.supplyAsync(() -> {
try {
return Database.fetchUser("user123");
} catch (InterruptedException e) {
throw new RuntimeException(e);
}
}, executor));
}

CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
                        System.out.println("DB calls made: " + dbCalls.get());

                               // With Singleflight - only one DB call
System.out.println("\nWith Singleflight:");
dbCalls.set(0);
SimpleSingleflight singleflight = new SimpleSingleflight();
futures.clear();

                                for (int i = 0; i < 10; i++) {
                           futures.add(singleflight.doSingle("user123", () ->
Database.fetchUser("user123")));
}

CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
                        System.out.println("DB calls made: " + dbCalls.get());
singleflight.shutdown();

// Singleflight with Cache
System.out.println("\nSingleflight with Cache:");
dbCalls.set(0);
          SingleflightCache<String, String> cache = new SingleflightCache<>(key -> {
try {
return Database.fetchUser(key);
} catch (InterruptedException e) {
throw new RuntimeException(e);
}
});

                                 // First call - goes to DB
String result1 = cache.get("user456");
                            System.out.println("First call: " + result1);

                                  // Second call - from cache
String result2 = cache.get("user456");
                           System.out.println("Second call: " + result2);

                        System.out.println("DB calls made: " + dbCalls.get());

executor.shutdown();
}
}
```

When to Use Singleflight Pattern

Suitable Situations:

1. When you have cache misses from multiple threads.

2. For expensive operations that should be deduplicated.

3. When calling external APIs with rate limits.

4. For loading shared resources (config, large files).

5. When multiple requests for same data can be collapsed.

Real-World Use Cases:

- **Cache Libraries:** Loading caches on miss
- **Database Access:** Reducing duplicate queries
- **API Gateways:** Collapsing identical requests
- **Configuration Loading:** Loading config once
- **File Loading:** Loading large files once

Advantages of Singleflight Pattern

1. **Resource Efficiency:** Eliminates duplicate work.

2. **Reduced Load:** Less pressure on databases and APIs.

3. **Consistency:** All waiters get same result.

4. **Fault Tolerance:** If first fails, all fail.

5. **Simplicity:** Transparent to callers.

Disadvantages of Singleflight Pattern

1. **Complexity:** Additional infrastructure.

2. **Memory Usage:** Tracking in-flight operations.

3. **Cascading Failures:** One failure affects all waiters.

4. **Timeout Handling:** Need to handle stuck operations.

5. **Distributed Coordination:** Hard across multiple nodes.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Singleflight	Deduplicate concurrent calls	Coalesces in-flight
Cache	Store results	Prevents future calls
Throttling	Limit rate	Different goal
Memoization	Cache function results	For repeated calls

Singleflight vs Cache

- **Singleflight:** Prevents concurrent duplicates
- **Cache:** Prevents future duplicates

Singleflight vs Throttling

- **Singleflight:** Deduplicates identical operations
- **Throttling:** Limits rate of operations

Common Pitfalls

1. **No Timeout:** Operation can hang forever.

2. **Memory Leaks:** In-flight map grows.

3. **No Cleanup:** Completed calls not removed.

4. **Key Collisions:** Different operations with same key.

5. **Distributed Use:** Doesn't work across nodes.

Best Practices

1. **Implement Timeouts:** Prevent hanging.

2. **Clean Up:** Remove completed operations.

3. **Use Weak References:** For cache integration.

4. **Distinguish Keys:** Use unique keys per operation type.

5. **Monitor In-flight:** Track waiting requests.

6. **Handle Errors:** Propagate exceptions correctly.

[Page 37]

PART SEVEN: COMPARISON AND INTERVIEW PREPARATION

Chapter 35: Pattern Comparison Table

Pattern	Category	Purpose	When to Use
---	---	---	---
Mutex	Coordination	Exclusive access	Shared data protection
Signaling	Coordination	Thread notification	Producer-consumer, wait conditions
Rendezvous	Coordination	Two-thread synchronization	Data exchange, meeting points
Barrier	Coordination	Multi-thread synchronization	Phase synchronization
Double-Checked Locking	Coordination	Optimized lazy init	Singleton, lazy loading
Read-Write Lock	Coordination	Shared reads, exclusive writes	Read-heavy data structures
Monitor Object	Coordination	Mutex + condition	Complete thread-safe objects
Guarded Suspension	Coordination	Wait for condition	Resource pools, blocking queues
Thread Pool	Task Distribution	Reuse threads	Many short-lived tasks
Worker Pool	Task Distribution	Parallel processing	Batch processing, task queues
Active Object	Task Distribution	Async method execution	GUI apps, decoupled systems
Future/Promise	Task Distribution	Async result	Async operations, callbacks
Generator	Task Distribution	Lazy sequences	Large or infinite sequences
Fork-Join	Task Distribution	Divide and conquer	Recursive parallel tasks
Leader-Followers	Task Distribution	Event handling	Network servers, event loops
Producer-Consumer	Data Flow	Decouple production	Buffering, load leveling
Pipeline	Data Flow	Sequential stages	Stream processing, ETL
Fan-Out/Fan-In	Data Flow	Parallel independent tasks	Data aggregation, parallel calls

Pattern	Category	Purpose	When to Use
MapReduce	Data Flow	Large-scale data	Distributed data processing
Reactor	Event Handling	Sync event demux	Network servers
Proactor	Event Handling	Async event demux	High-performance I/O
Event Loop	Event Handling	Single-thread events	Node.js, GUI frameworks
Publish-Subscribe	Event Handling	Decoupled messaging	Event-driven architectures
Half-Sync/Half-Async	Event Handling	Bridge sync/async	Web servers, gateways
Balking	Advanced	Skip if not ready	Initialization, one-time ops
Thread-Local Storage	Advanced	Per-thread data	Request context, connections
Scheduler	Advanced	Task ordering	Real-time systems, priorities
Supervisor	Advanced	Failure recovery	Resilient systems, actors
Rate Limiting	Advanced	Control operation rate	API protection, fairness
Singleflight	Advanced	Deduplicate calls	Cache loading, expensive ops

---

[Page 38]

Chapter 36: How to Answer Concurrency Questions

Understanding What Interviewers Look For

When interviewers ask about concurrency patterns, they want to see:

1. **Understanding of Thread Safety:** Can you identify race conditions?

2. **Synchronization Knowledge:** Do you know when and how to synchronize?

3. **Pattern Selection:** Can you choose the right pattern for the problem?

4. **Trade-off Awareness:** Do you know the downsides of each approach?

5. **Practical Experience:** Can you relate to real-world scenarios?

The STAR Method for Concurrency Questions

Situation: Describe the concurrent scenario.
Task: Explain what needs to be synchronized.

Application: Describe the pattern used.

Result: Explain the benefits and trade-offs.

Example: Producer-Consumer

"Situation: In our order processing system, orders came in faster than we could process them. During peak times, we were losing orders.

Task: We needed a buffer between order reception and processing that could handle spikes.

Application: We implemented the Producer-Consumer pattern with a bounded blocking queue. The web thread (producer) puts orders in the queue; worker threads (consumers) take orders and process them.

Result: This smoothed out load spikes, improved throughput, and provided backpressure to the web layer when processing fell behind."

Common Interview Questions by Pattern

Thread Coordination:

1. "What's the difference between mutex and semaphore?"
2. "How does the Read-Write Lock pattern improve performance?"
3. "When would you use Guarded Suspension vs Balking?"

Task Distribution:

4. "Explain the difference between Thread Pool and Fork-Join."
5. "How does the Future pattern help with asynchronous programming?"
6. "What problem does the Active Object pattern solve?"

Data Flow:

7. "How does the Pipeline pattern differ from Producer-Consumer?"
8. "When would you use Fan-Out/Fan-In instead of Fork-Join?"
9. "Explain MapReduce in simple terms."

Event Handling:

10. "What's the difference between Reactor and Proactor?"

11. "How does Node.js handle concurrency with its event loop?"

12. "When would you use Publish-Subscribe instead of direct calls?"

Advanced Patterns:

13. "What problem does Singleflight solve?"

14. "How would you implement rate limiting for an API?"

15. "Explain the Supervisor pattern and its use in resilient systems."

Scenario Questions:

16. "You have a cache that multiple threads access. How do you ensure thread safety while maintaining performance?"

17. "Your application needs to process a large dataset. How would you parallelize the work?"

18. "How would you implement a job queue that processes tasks in order but doesn't block the producer?"

19. "Your system is experiencing cascading failures. How would you make it more resilient?"

20. "How would you prevent duplicate database queries when multiple requests miss the cache simultaneously?"

[Page 39]

Chapter 37: Common Interview Questions with Examples

1. Q: What is a race condition and how do you prevent it?

A: A race condition occurs when multiple threads access shared data concurrently and the final result depends on the timing of their execution.

Example: Two threads incrementing a counter:

```
counter = 0
Thread A: read counter (0)
```

```
Thread B: read counter (0)
Thread A: write 1
Thread B: write 1
Result: 1 (should be 2)
```

Prevention: Use synchronization mechanisms like mutexes, atomic operations, or thread-local storage.

2. Q: Explain the difference between a mutex and a semaphore.

A:

- **Mutex:** Binary lock with ownership. Only the thread that locked it can unlock it. Used for mutual exclusion.

- **Semaphore:** Counter with multiple permits. Any thread can release permits. Used for signaling or controlling access to multiple resources.

3. Q: When would you use a read-write lock instead of a mutex?

A: Use read-write lock when reads are much more frequent than writes. Multiple readers can access simultaneously, while writers get exclusive access. Example: Configuration store that's read thousands of times per second but updated rarely.

4. Q: What problem does the Producer-Consumer pattern solve?

A: It decouples data production from consumption using a queue. Producers add data; consumers take data. Benefits:

- Handles rate mismatches

- Provides buffering

- Enables parallel processing

- Example: Web server accepting requests (producers) while workers process them (consumers)

5. Q: Explain the difference between Reactor and Proactor patterns.

A:

- **Reactor:** Synchronous I/O. Notifies when operation can be performed (ready). Thread then performs the operation.

- **Proactor:** Asynchronous I/O. Initiates operation and notifies when complete. OS handles the operation.

6. Q: How does the Fork-Join pattern work?

A: Divide-and-conquer parallel processing:

1. Fork: Split task into smaller subtasks

2. Recursively fork until tasks are small enough

3. Compute each subtask in parallel

4. Join: Combine results from subtasks

5. Uses work stealing for load balancing

7. Q: What is the Double-Checked Locking pattern and why was it broken?

A: Optimized lazy initialization:

```
if (instance == null) {
    synchronized(this) {
        if (instance == null) {
            instance = new Object();
        }
    }
}
```

It was broken in early Java due to instruction reordering (constructor could execute after assignment). Fixed with volatile in Java 5+.

8. Q: Explain the Singleflight pattern with a real-world example.

A: Singleflight prevents duplicate concurrent operations. Example: 100 requests for the same user data miss the cache simultaneously. Without Singleflight: 100 database queries. With Singleflight: 1 database query, all 100 wait for the result.

9. Q: How would you implement rate limiting for an API?

A: Several algorithms:

- **Token Bucket:** Tokens replenished at fixed rate, each request consumes a token

- **Leaky Bucket:** Requests processed at fixed rate

- **Sliding Window:** Counts requests in rolling window

- **Implementation:** Use Redis for distributed counters, store per client

10. Q: What's the difference between Guarded Suspension and Balking patterns?

A:

- **Guarded Suspension:** Wait until condition is true (blocks)
- **Balking:** Return immediately if condition not true (non-blocking)

Use Guarded Suspension when operation must succeed; use Balking when operation is optional or can be retried later.

11. Q: Explain the Thread-Local Storage pattern and when to use it.

A: Thread-Local Storage gives each thread its own copy of a variable, eliminating synchronization. Use for:

- Per-thread contexts (user sessions)
- Thread-unsafe objects (SimpleDateFormat)
- Database connections per thread
- Transaction contexts

12. Q: How does the Supervisor pattern make systems resilient?

A: Supervisors monitor workers and automatically restart them on failure. Features:

- One-for-one: restart only failed worker
- One-for-all: restart all workers
- Backoff: prevent crash loops
- Supervision hierarchy: failures escalate up

Example: Erlang/OTP, Akka actors.

13. Q: What problem does the Pipeline pattern solve?

A: Pipeline processes data through sequential stages in parallel. Each stage runs in its own thread. Benefits:

- Multiple items in flight simultaneously
- Higher throughput than sequential
- Example: Image processing (load $\rightarrow$ resize $\rightarrow$ filter $\rightarrow$ save)

14. Q: Explain the difference between Future and Promise.

A:

- **Future:** Read-only view of async result. Can check if done, wait for result, cancel.
- **Promise:** Writable counterpart. Can complete the future with a result or exception.

Java: CompletableFuture combines both.

15. Q: How does the Event Loop pattern work in Node.js?

A: Single-threaded loop that processes events:

1. Event queue holds callbacks
2. Loop dequeues and executes one callback
3. During execution, new events are queued
4. I/O operations are non-blocking
5. Good for I/O-bound, not CPU-bound work

[Page 40]

CONCLUSION

Key Takeaways

1. **Concurrency is Essential:** Modern applications must handle multiple tasks simultaneously.

2. **Patterns Provide Solutions:** Each pattern solves specific concurrency problems.

3. **No Silver Bullet:** Choose the right pattern based on your needs.

4. **Understand Trade-offs:** Every pattern has advantages and disadvantages.

5. **Start Simple:** Begin with simpler patterns before adding complexity.

6. **Test Thoroughly:** Concurrency bugs are hard to reproduce; test under load.

Your Interview Preparation Checklist

- [ ] Understand the problem each pattern solves
- [ ] Can explain each pattern in simple terms
- [ ] Know when to use each pattern
- [ ] Know the disadvantages and trade-offs
- [ ] Can compare similar patterns
- [ ] Have real-world examples ready
- [ ] Can write simple implementations
- [ ] Understand language-specific concurrency constructs

Final Tips

1. **Practice Out Loud:** Explain patterns to yourself or a friend.

2. **Draw Diagrams:** Visual explanations show deep understanding.

3. **Be Honest:** If you haven't used a pattern, say so, but explain what you know.

4. **Show Depth:** Mention trade-offs and alternatives.

5. **Stay Calm:** You know the material. Trust your preparation.

Resources for Further Study

- **Books:** "Java Concurrency in Practice", "Concurrency in Go"
- **Languages:** Study how concurrency works in Java, Go, Rust, Kotlin
- **Practice:** Implement patterns in toy projects
- **Production Systems:** Study how real systems handle concurrency

Good luck with your interview!

About This Book

This book was created specifically for developers preparing for technical interviews on concurrency patterns. Every pattern is explained in clear, simple English with

comprehensive examples and diagrams. The goal is not just to help you pass an interview, but to make you a better concurrent programmer.

May your threads always synchronize, your deadlocks be nonexistent, and your interviews successful.

[End of Document]